GREAT ZIMBABWE UNIVERSITY

FACULTY OF NATURAL SCIENCES

DEPARTMENT OF MATHEMATICS AND COMPUTER SCIENCE



NAME:	**MACDONALD K GUDO (M215524)**
LEVEL:	**4.2**
PROGRAMME:	**BACHELOR OF SCIENCE HONOURS COMPUTER SCIENCE**
COURSE CODE:	**HCS460**
PROJECT TITLE:	**AN INTELLIGENT DECISION SUPPORT SYSTEM FOR PHARMACEUTICAL SUPPLY CHAIN OPTIMISATION: A CASE OF PARIRENYATWA GROUP OF HOSPITALS**
SUPERVISOR:	**[SUPERVISOR'S NAME]**

A dissertation submitted in partial fulfilment of the requirements for the Bachelor of Science Honours Degree in Computer Science.

[MONTH YEAR]

DECLARATION

I, Macdonald K. Gudo (M215524), declare that this dissertation is my own work. It was carried out under the guidance of my supervisor. It has not been submitted for any other degree at this or any other university. All sources of information have been acknowledged and cited in APA style.

Declaration on the use of AI tools

Section 3.1.2 of the project guidelines requires students to declare the use of AI tools. I declare that AI tools were used during this project. They helped with drafting and editing parts of this document. They also helped with parts of the software. I reviewed, tested and edited all such output. I take full responsibility for the content, the code and the results reported here.

Student: Macdonald K. Gudo Signature: ..... Date: .....

Supervisor: [Supervisor's name] Signature: ..... Date: .....

ACKNOWLEDGEMENT

First, I thank God Almighty for the strength and wisdom to finish this work. Without His grace, this work would not have been possible.

I am deeply grateful to my supervisor, [Supervisor's name], for patient guidance throughout the year. Your questions and corrections shaped this work at every stage.

I also thank the lecturers of the Department of Mathematics and Computer Science. The knowledge they shared over four years is the foundation of this project.

My thanks go to the staff of the Pharmacy Department at Parirenyatwa Group of Hospitals. Their daily work in difficult conditions inspired this study.

Finally, I thank my family for their prayers, sacrifice and constant support. I am also grateful to my classmates, whose ideas and encouragement kept me going.

ABSTRACT

Zimbabwe's public hospitals often run out of essential medicines. At the same time, other stock expires on the shelves. This study developed an Intelligent Decision Support System for this problem. The case study was the Pharmacy Department of Parirenyatwa Group of Hospitals. The study followed Design Science Research. CRISP-DM guided the models and Agile Scrum guided the software. Four decision models were trained in Google Colab on simulated data for 20 essential medicines. A Random Forest model forecasts daily demand. It cut forecast error by 45.9% compared with a naive method. A second Random Forest model predicts stockouts before the next delivery. It caught 80.2% of real stockouts, with a PR-AUC of 0.81. A weighted score with K-Means clustering places twelve suppliers into three stable tiers. A first-expiry-first-out rule flags batches likely to expire unused. It agreed with 95.5% of labelled outcomes and outperformed a trained classifier. A decision engine combines the four models into one reorder recommendation per medicine. Each recommendation explains its reasons and needs a pharmacist's approval. The system was built with Django, React and PostgreSQL. It runs on the hospital network without internet access. It passed 35 automated tests, 10 model tests and 14 system tests. Testing also found and fixed thirteen defects. User acceptance testing with hospital staff remains to be done. The study shows that machine learning can help Zimbabwean public pharmacies reduce stockouts and waste.

Keywords: decision support system, machine learning, demand forecasting, pharmaceutical supply chain, Zimbabwe

TABLE OF CONTENTS

DECLARATION i

ACKNOWLEDGEMENT ii

ABSTRACT iii

LIST OF ABBREVIATIONS xv

CHAPTER ONE: INTRODUCTION 1

1.0 Introduction 1

1.1 Background of the Organisation 1

1.1.1 Structure and Services 1

1.1.2 Goals of the Organisation 2

1.1.3 The Pharmacy Department 2

1.1.4 Operational Environment 2

1.2 Background to the Proposed System 3

1.2.1 The Current System 3

1.2.2 Gaps in the Current System 3

1.2.3 The Opportunity 3

1.3 Statement of the Problem 4

1.4 Aim and Objectives of the Proposed System 4

1.4.1 Aim 4

1.4.2 Objectives 4

1.4.3 Research Questions 5

1.5 Definitions of Key Terms and Concepts 5

1.6 Significance of the Study 6

1.6.1 To the PGH Pharmacy Department 6

1.6.2 To Patients 6
1.6.3 To MoHCC and NatPharm 6
1.6.4 To the Academic Community 6
1.6.5 To the Researcher 6
1.6.6 Alignment with National and Global Frameworks 6
1.7 Theoretical Framework and Conceptual Model 7
1.7.1 Simon's Model of Decision Making 7
1.7.2 Inventory Control Theory 7
1.7.3 Machine Learning Theory 8
1.7.4 Conceptual Model 8
1.8 Assumptions 9
1.9 Ethical Considerations 9
1.10 Methodology 10
1.10.1 Research Methodology 10
1.10.2 Software Development Methodology 11
1.10.3 Data Collection Tools and Techniques 12
1.10.4 Development Tools and Technologies 12
1.11 Scope and Limitations 13
1.11.1 Scope 13
1.11.2 Limitations 14
1.12 Project Timeline 14
1.13 Justification of the Project 16
1.14 Summary 16
CHAPTER TWO: LITERATURE REVIEW AND FEASIBILITY STUDY 17
2.0 Introduction 17

2.1 What Has Been Done Locally 17
2.1.1 The State of Zimbabwe's Medicine Supply Chain 17
2.1.2 From Push Systems to the Zimbabwe Assisted Pull System 18
2.1.3 Digital Health Systems in Zimbabwe 18
2.1.4 Regional Advances 18
2.1.5 Successes, Failures and Limitations of Local Efforts 19
2.1.6 Government Policies and Initiatives 20
2.2 What Has Been Done Globally 20
2.2.1 Optimising the Pharmaceutical Supply Chain 20
2.2.2 Machine Learning for Demand Forecasting 21
2.2.3 Predicting Stockouts and Backorders 21
2.2.4 Supplier Evaluation and Selection 21
2.2.5 Commercial and Hospital Systems 22
2.2.6 Challenges and Lessons Learned 22
2.2.7 Summary of Related Studies 22
2.3 What New Idea Is This Project Bringing? 23
2.3.1 Comparison with Existing Systems 23
2.3.2 Originality and Novelty 23
2.3.3 Potential Contributions 24
2.4 Feasibility Study of the Proposed System 24
2.4.1 Technical Feasibility 24
2.4.2 Operational Feasibility 25
2.4.3 Schedule Feasibility 26
2.4.4 Economic Feasibility 27
2.4.5 Threats to Security and Controls 29

2.5 Summary 30
CHAPTER THREE: METHODOLOGY 32
3.0 Introduction 32
3.1 Research Methodology 32
3.1.1 Research Philosophy and Design 32
3.1.2 Design Science Approach 32
3.1.3 Experimental-Based Approach 33
3.1.4 Quantitative Approach 34
3.1.5 Qualitative Approach 34
3.1.6 Population and Sampling 34
3.1.7 Data Collection Instruments 34
3.1.8 Datasets for Model Development 35
3.1.9 Data Preparation and Feature Engineering 35
3.1.10 Model Development 36
3.1.11 Data Analysis 37
3.2 Test Plan for the Proposed Model 37
3.2.1 Test Objectives 37
3.2.2 Test Environment 37
3.2.3 Test Data 38
3.2.4 Performance Metrics 38
3.2.5 Test Cases 39
3.2.6 Test Results 40
3.2.7 Bug and Issue Reporting 41
3.3 Ethical Considerations 42
3.3.1 Originality and Referencing 42

3.3.2 Use of AI Tools 42
3.3.3 Consent and Data Privacy 42
3.3.4 Bias and Fairness in AI Models 43
3.3.5 Environmental and Social Responsibility 43
3.4 Summary 43
CHAPTER FOUR: SYSTEM ANALYSIS AND DESIGN 44
4.0 Introduction 44
4.0.1 Requirements Summary 44
4.1 Functional Modeling 45
4.1.1 Context Diagram 45
4.1.2 Data Flow Diagram (Level 1) 46
4.2 System Modeling 47
4.2.1 Use Case Diagram 47
4.2.2 Sequence Diagrams 49
4.2.3 State Diagrams 50
4.2.4 Activity Diagram 51
4.2.5 Architectural Design 52
4.2.6 Component and Module Design 54
4.3 Data Modeling 55
4.3.1 Entity Relationship Diagram 55
4.3.2 Class Diagram 56
4.3.3 Database Design 57
4.4 Interface Design 59
4.4.1 Design Principles 59
4.4.2 Pseudocode for Core Components 64

4.5 Security Design 66
4.5.1 Password Hashing 66
4.5.2 Data Encryption 66
4.5.3 Authentication and Authorisation Logic 66
4.5.4 User Access Roles 67
4.5.5 Secure Storage and Integrity 67
4.6 Hardware Design 67
4.7 Summary 68
CHAPTER FIVE: CODING AND TESTING 69
5.0 Introduction 69
5.1 Coding 69
5.1.1 Development Environment 69
5.1.2 Building the Machine Learning Models 69
5.1.3 Training and Testing the Expiry Model 70
5.1.4 Backend Implementation 70
5.1.5 Integrating the Models 71
5.1.6 Frontend Implementation 72
5.2 Testing 73
5.2.1 Testing Approach 73
5.2.2 Automated Unit and Integration Tests 73
5.2.3 Model Testing 73
5.2.4 System Functional Testing 75
5.2.5 Performance Testing 91
5.2.6 Defects Found and Fixed 91
5.2.7 User Acceptance Testing 92

5.3 Summary 93
CHAPTER SIX: RESULTS AND DISCUSSIONS 94
6.0 Introduction 94
6.1 Achievement of Objectives 94
6.1.1 Objective 1: Analyse Current Inventory and Procurement Practices 94
6.1.2 Objective 2: Forecast Daily Medicine Demand 95
6.1.3 Objective 3: Predict Stockout Risk 97
6.1.4 Objective 4: Score and Tier Suppliers 98
6.1.5 Objective 5: Flag Batches at Risk of Expiry 100
6.1.6 Objective 6: Integrate the Models into a Secure Web System 101
6.1.7 Objective 7: Evaluate the System 103
6.1.8 Answers to the Research Questions 104
6.2 Limitations and Constraints 104
6.3 Recommendations for Further Research 105
6.4 Conclusion 106
REFERENCES 107

LIST OF FIGURES

Figure 1.1: Conceptual model of the proposed IDSS (adapted from Simon, 1960) 8

Figure 1.2: Gantt chart showing the critical path of the project 15

Figure 2.1: Project Gantt chart showing concurrency and sequencing of tasks 27

Figure 4.1: Context diagram of the proposed system 46

Figure 4.2: Level 1 data flow diagram 47

Figure 4.3: Use case diagram 48

Figure 4.4: Sequence diagram for user login 49

Figure 4.5: Sequence diagram for running the decision engine and approving a reorder 50

Figure 4.6: State diagrams for (a) a recommendation and (b) a purchase order 51

Figure 4.7: Activity diagram for the reorder decision 52

Figure 4.8: System architecture 53

Figure 4.9: Component diagram 54

Figure 4.10: Entity relationship diagram 56

Figure 4.11: Class diagram 57

Figure 4.12: Login screen 60

Figure 4.13: Dashboard showing key indicators and decision intelligence 61

Figure 4.14: Inventory screen with model-based reorder points 62

Figure 4.15: Model Lab screen for the demand forecasting model 63

Figure 4.16: Reorder Decisions screen with the decision engine 64

Figure 5.1: Live re-test of the stockout model in the Model Lab 75

Figure 5.2: ST-02, login rejected with a wrong password 77

Figure 5.3: ST-01, dashboard after a successful login 78

Figure 5.4: ST-03, repeated login attempts blocked 79

Figure 5.5: ST-05, reduced menu for the stores staff role 80
Figure 5.6: ST-06, demand forecast with model details 81
Figure 5.7: ST-12, a medicine with no history 82
Figure 5.8: Decision engine results before the what-if dispense 84
Figure 5.9: ST-07, Paracetamol moves into the risk zone after dispensing..... 84
Figure 5.10: ST-13, a negative quantity is rejected 85
Figure 5.11: The four-model decision chain for a recommendation 86
Figure 5.12: ST-08, recommendation approved and recorded 86
Figure 5.13: ST-09, supplier performance page 87
Figure 5.14: ST-10, expiry risk page 88
Figure 5.15: ST-11, inventory report generated and downloaded 89
Figure 5.16: Stockout risk for Coartem before the what-if change 90
Figure 5.17: Stockout risk for Coartem after adding 5,000 units 90
Figure 6.1: Demand model re-tested on unseen data in the Model Lab 96
Figure 6.2: Stockout risk for Coartem, with the reasons behind it 97
Figure 6.3: Scoring a candidate supplier in the Model Lab 100
Figure 6.4: Supplier scores recomputed live from 6,000 orders 100
Figure 6.5: Expiry risk check and its test results 101
Figure 6.6: How the four models reached one reorder decision 102
Figure 6.7: Role-based access for a stores staff account 103

LIST OF TABLES

Table 0.1: List of abbreviations xv

Table 1.1: Definitions of key terms and concepts..... 5

Table 1.2: Design Science Research steps applied to this study..... 10

Table 1.3: Data collection tools and techniques 12

Table 1.4: Development tools and technologies 13

Table 1.5: Project tasks, durations, dependencies and slack..... 15

Table 2.1: Summary of related studies 23

Table 2.2: Comparison of existing systems with the proposed IDSS 23

Table 2.3: Technical resources required 25

Table 2.4: Estimated costs of the proposed system 28

Table 2.5: Estimated yearly benefits of the proposed system..... 28

Table 2.6: Three-year cash flow and NPV at a 12% discount rate 29

Table 2.7: Security threats and controls..... 30

Table 3.1: Research approaches mapped to research questions 32

Table 3.2: Data collection instruments 35

Table 3.3: Datasets used for model development 35

Table 3.4: Candidate models for each decision task..... 36

Table 3.5: Test environment 38

Table 3.6: Performance metrics 38

Table 3.7: Machine learning test cases 39

Table 3.8: System test cases..... 39

Table 3.9: Model 1 results on the held-out test set 40

Table 3.10: Model 2 results on the held-out test set 40

Table 3.11: Issue log from model development 42
Table 4.1: Functional requirements 44
Table 4.2: Non-functional requirements 45
Table 4.3: Use case description for running the decision engine 49
Table 4.4: Module responsibilities 55
Table 4.5: MEDICINE table 57
Table 4.6: BATCH table 58
Table 4.7: DAILY_DEMAND table 58
Table 4.8: RECOMMENDATION table 58
Table 4.9: PURCHASE_ORDER table 59
Table 4.10: Role permission matrix 67
Table 5.1: Expiry classifier compared with the depletion rule on 560 test batches 70
Table 5.2: Automated backend tests 73
Table 5.3: Machine learning test results 74
Table 5.4: System test results 76
Table 5.5: Performance test results 91
Table 5.6: Defect log 92
Table 5.7: User acceptance test plan 92
Table 6.1: Summary of objectives and their achievement 94
Table 6.2: Gaps in the current system and the features that address them 95
Table 6.3: Summary of the technical evaluation 103
Table 6.4: User acceptance test results (to be completed after testing at PGH) 104
Table 6.5: Answers to the research questions 104

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
ARI	Adjusted Rand Index
ARV	Antiretroviral
CRISP-DM	Cross-Industry Standard Process for Data Mining
CSV	Comma-Separated Values
DFD	Data Flow Diagram
DSR	Design Science Research
DSS	Decision Support System
EHR	Electronic Health Record
eLMIS	Electronic Logistics Management Information System
EOQ	Economic Order Quantity
ERD	Entity Relationship Diagram
FEFO	First Expiry, First Out
IDSS	Intelligent Decision Support System
JWT	JSON Web Token
MAE	Mean Absolute Error
MAPE	Mean Absolute Percentage Error
MCAZ	Medicines Control Authority of Zimbabwe
ML	Machine Learning
MoHCC	Ministry of Health and Child Care
NatPharm	National Pharmaceutical Company of Zimbabwe
NDS1	National Development Strategy 1
NPV	Net Present Value
PGH	Parirenyatwa Group of Hospitals
PR-AUC	Area Under the Precision-Recall Curve
REST	Representational State Transfer
RMSE	Root Mean Squared Error
ROC-AUC	Area Under the Receiver Operating Characteristic Curve
ROI	Return on Investment
ROP	Reorder Point
SDG	Sustainable Development Goal
SHAP	SHapley Additive exPlanations
TB	Tuberculosis
UAT	User Acceptance Testing
UML	Unified Modeling Language
ZAPS	Zimbabwe Assisted Pull System

Table 0.1: List of abbreviations

CHAPTER ONE: INTRODUCTION

1.0 Introduction

Medicines save lives only when they are on the shelf at the right time. A hospital pharmacy must therefore plan its stock with great care. It must order enough to meet patient demand. Yet it must not buy so much that stock expires before use. Getting this balance right is hard in Zimbabwe's public hospitals. Budgets are tight, lead times are long and demand shifts with the seasons.

This study is based at Parirenyatwa Group of Hospitals (PGH) in Harare. It focuses on the hospital's Pharmacy Department. The department keeps computer records of its stock. However, these records mostly describe what has already happened. They do not tell managers what is likely to happen next. Choices on what to order, how much, and from whom still rest on experience. This study proposes an Intelligent Decision Support System (IDSS) to close this gap. The system uses machine learning to forecast demand and flag stockout risk. It also scores suppliers and points out batches likely to expire unused.

This chapter opens with the background of the organisation. It then explains the current system and the problem it creates. The aim, objectives and research questions follow. Key terms, significance, theory, assumptions and ethics are then covered. The chapter closes with the methodology, scope, timeline and justification.

1.1 Background of the Organisation

Parirenyatwa Group of Hospitals is the largest referral hospital in Zimbabwe. It is a public institution located in Harare. The hospital falls under the Ministry of Health and Child Care (MoHCC). A Hospital Management Board oversees its day-to-day running. The group is named after Dr Tichafa Samuel Parirenyatwa. He is remembered as the first Black Zimbabwean to qualify as a medical doctor.

1.1.1 Structure and Services

PGH is not one building but a group of units. These include the main hospital and Mbuya Nehanda Maternity Hospital. Sekuru Kaguvi Eye Unit, a psychiatric unit and a radiotherapy

centre are also included. The group is the main teaching hospital for the University of Zimbabwe medical school. Clinics and district hospitals across the country refer patients to it. As a result, PGH treats a very wide range of conditions. These run from malaria and tuberculosis to HIV, diabetes and hypertension.

1.1.2 Goals of the Organisation

Like other central hospitals, PGH has three broad goals. First, it provides specialist care that lower-level facilities cannot offer. Second, it trains doctors, nurses, pharmacists and other health workers. Third, it supports medical research that informs national health policy. Each of these goals depends on a steady supply of safe and affordable medicines.

1.1.3 The Pharmacy Department

The Pharmacy Department is the focus of this study. It supplies medicines to wards, specialist clinics and outpatient services. Its core tasks are procurement, receiving, storage, stock control and dispensing. It also advises clinicians on the safe and rational use of medicines. Senior pharmacists lead the department. They work with pharmacy technicians, stores clerks and procurement officers. Every week, staff must decide which items to reorder and in what quantity.

1.1.4 Operational Environment

Most medicines reach PGH through the National Pharmaceutical Company of Zimbabwe (NatPharm). NatPharm is the state body that buys, stores and distributes medicines to public facilities. When NatPharm is out of stock, the hospital turns to other registered suppliers. These include local manufacturers such as CAPS, Varichem and Datlabs. Some products come from regional importers in South Africa and India. All products must be registered by the Medicines Control Authority of Zimbabwe (MCAZ).

The operating environment is far from easy. Foreign currency shortages often delay imports and push up prices. Supplier lead times can stretch to several weeks. Demand for antimalarials climbs sharply during the rainy season. Programme medicines such as antiretrovirals (ARVs) must never run out. A missed dose can harm the patient and weaken

long-term treatment. At the same time, power cuts and network outages disrupt computer work. Any new system must therefore be light, reliable and simple to use.

1.2 Background to the Proposed System

1.2.1 The Current System

At present, the pharmacy tracks stock with stock cards, ledgers and spreadsheets. Summary reports also feed into the national electronic Logistics Management Information System (eLMIS). In addition, public hospitals are adopting Impilo, the national electronic health record (EHR). These tools are a clear step up from paper alone. Staff can now see balances, receipts and issues much faster.

Even so, the current tools are transactional in nature. They store what has already taken place. They do not study past consumption to estimate future demand. Reorders are decided during periodic stock reviews. A pharmacist checks recent issues and then settles on an order quantity. This depends heavily on personal judgement. It also takes many hours when hundreds of items need review. When an experienced officer is away, the quality of decisions can drop.

1.2.2 Gaps in the Current System

A review of the current process reveals five main gaps.

- **No demand forecasting.** Seasonal peaks, such as malaria cases, are noticed late.
- **No early stockout warning.** Risk is seen only when shelves are nearly empty.
- **No objective supplier rating.** Late or short deliveries are not turned into scores.
- **Weak expiry tracking.** Slow-moving batches are often found just before they expire.
- No link from data to action. Reports exist, but they do not recommend anything.

1.2.3 The Opportunity

Artificial intelligence (AI) offers a practical way to fill these gaps. Machine learning models can learn patterns from past stock transactions. They can then estimate future demand and the chance of a stockout. Scoring models can rank suppliers on reliability, quality and price. A web dashboard can bring these results together for pharmacists. The pharmacist still makes

the final call. The system simply backs that call with evidence. This approach fits Education 5.0 and the Fourth Industrial Revolution (4IR) agenda. It also supports Vision 2030 (Government of Zimbabwe, 2018) and NDS1 (Government of Zimbabwe, 2020).

1.3 Statement of the Problem

The PGH Pharmacy Department suffers repeated stockouts of key medicines. At the same time, other medicines expire on the shelves before they are used. Both problems waste scarce public funds and put patients at risk. The current records do not forecast demand or warn of coming shortages. They also fail to rate suppliers or flag batches at risk of expiry. Pharmacists therefore make ordering decisions by hand, using experience and old reports. This is slow and hard to apply across hundreds of medicines. It also copes poorly with seasonal demand and uncertain supplier lead times. There is a need for a system that turns pharmacy data into timely recommendations.

1.4 Aim and Objectives of the Proposed System

1.4.1 Aim

The aim of this study is to develop an Intelligent Decision Support System. The system will improve medicine supply chain decisions at the PGH Pharmacy Department.

1.4.2 Objectives

The specific objectives of the study are:

1. To analyse current inventory and procurement practices at the PGH Pharmacy Department.
2. To develop a machine learning model that forecasts daily demand for each medicine.
3. To develop a classification model that predicts stockout risk before the next delivery.
4. To design a supplier performance scoring model using weighted scoring and clustering.
5. To develop an expiry risk module that flags batches likely to expire unused.
6. To integrate the models into a secure web-based system with role-based dashboards.
7. To evaluate the system using model accuracy metrics and user acceptance testing.

1.4.3 Research Questions

The study seeks to answer the following questions:

1. How does the PGH pharmacy currently plan and order its medicines?
2. How accurately can machine learning forecast daily medicine demand?
3. Can a classifier detect stockout risk early enough for staff to act?
4. How can suppliers be ranked fairly on delivery, quality and cost?
5. How can these models be combined into one system that pharmacists accept?

1.5 Definitions of Key Terms and Concepts

Table 1.1 explains the main terms used in this document.

Term	Meaning in this study
Decision Support System (DSS)	A computer system that helps managers make better choices. It does not replace them.
Intelligent DSS (IDSS)	A DSS that adds AI or machine learning to analyse data and suggest actions.
Pharmaceutical supply chain	All steps that move a medicine from maker to patient.
Machine learning (ML)	A branch of AI where computers learn patterns from data.
Demand forecasting	Estimating how much of a medicine will be used in a future period.
Stockout	A situation where a medicine is needed but none is in stock.
Lead time	The number of days between placing an order and receiving it.
Safety stock	Extra stock kept to cover sudden demand or late deliveries.
Reorder point (ROP)	The stock level at which a new order should be placed.
Economic Order Quantity (EOQ)	The order size that keeps ordering and holding costs lowest.
Expiry risk	The chance that a batch will pass its expiry date before it is used.
Random Forest	An ML method that combines many decision trees to give one prediction.
K-Means clustering	An ML method that sorts similar items into a set number of groups.
Mean Absolute Error (MAE)	The average size of forecast errors, in units of medicine.
PR-AUC	A score from 0 to 1 showing how well a model finds rare events.
Electronic Health Record (EHR)	A digital version of a patient's health record. Impilo is Zimbabwe's national EHR.
REST API	A standard way for the web screens to request data from the server.
JSON Web Token (JWT)	A signed digital pass that proves a user has logged in.

Table 1.1: Definitions of key terms and concepts

1.6 Significance of the Study

1.6.1 To the PGH Pharmacy Department

The system gives pharmacists a clear, daily view of stock risk. It shows which medicines to reorder, how much to order and from whom. Stock reviews that take hours could be done in minutes. Fewer stockouts mean fewer emergency purchases at high prices. Fewer expiries mean less money lost to destroyed stock.

1.6.2 To Patients

Patients are the final beneficiaries. Better stock planning means medicines are there when prescribed. Patients on ARVs and TB treatment can keep to their regimens without breaks. Families also spend less buying medicines from private pharmacies.

1.6.3 To MoHCC and NatPharm

Forecasts from central hospitals can improve national quantification. NatPharm could plan its own purchases with better demand signals. The ministry could also use supplier scores to guide tender decisions.

1.6.4 To the Academic Community

The study adds local evidence on machine learning in health supply chains. Most published work comes from high-income countries. This project shows how such tools can work under Zimbabwean conditions. Future students can build on its models, datasets and methods.

1.6.5 To the Researcher

The project lets the researcher apply skills gained during the degree. These include software engineering, databases, web development and machine learning. It also builds research and writing skills needed in industry.

1.6.6 Alignment with National and Global Frameworks

The study supports Vision 2030 and NDS1, which both prioritise health and digital growth. It contributes to Sustainable Development Goal 3 on good health (United Nations, 2015). It also supports SDG 9 on industry, innovation and infrastructure. Finally, it links to Aspiration

1 of Agenda 2063 (African Union Commission, 2015). That aspiration calls for healthy and well-nourished citizens.

1.7 Theoretical Framework and Conceptual Model

Three bodies of theory guide the design of the system. These are decision-making theory, inventory control theory and machine learning theory.

1.7.1 Simon's Model of Decision Making

Simon (1960) describes decision making in three phases. The first is intelligence, where the decision maker spots a problem. The second is design, where possible solutions are developed. The third is choice, where one option is selected. Power (2002) shows that a DSS can support each of these phases. In this study, the ML engine supports the intelligence phase. It detects rising demand, stockout risk and expiry risk early. The decision models support the design phase by proposing order options. The pharmacist then makes the choice by approving or rejecting each recommendation.

1.7.2 Inventory Control Theory

Classic inventory theory tells a store when to order and how much. The reorder point is the stock level that triggers an order (Silver et al., 2017). It is found with the formula below.

$$ROP = (d \times L) + SS$$

Here d is the average daily demand and L is the lead time in days. SS is the safety stock, which covers uncertainty in demand. Safety stock is found as follows.

$$SS = z \times \sigma d \times \sqrt{L}$$

In this formula, z is the service level factor. The symbol σd is the standard deviation of daily demand. The order size comes from the Economic Order Quantity model (Harris, 1913).

$$EOQ = \sqrt{(2DS / H)}$$

D is the annual demand and S is the cost of placing one order. H is the cost of holding one unit for a year. Traditional systems use simple averages for d. This study replaces the average with an ML forecast. This allows the reorder point to rise and fall with seasons and trends.

1.7.3 Machine Learning Theory

The study uses both supervised and unsupervised learning. In supervised learning, a model learns from examples with known answers. Demand forecasting is a regression task because the answer is a number. Stockout risk is a classification task because the answer is yes or no. Both tasks use Random Forest, which combines many decision trees (Breiman, 2001). In unsupervised learning, there is no known answer to learn from. Supplier tiers are therefore found with K-Means clustering. SHAP values are used to explain which inputs drive each prediction (Lundberg & Lee, 2017).

1.7.4 Conceptual Model

Figure 1.1 brings these theories together. Pharmacy data flows into the data layer and then into the ML engine. The engine feeds the decision models, which produce recommendations. The pharmacist reviews each one before any order is placed. New data from each cycle is used to retrain the models.

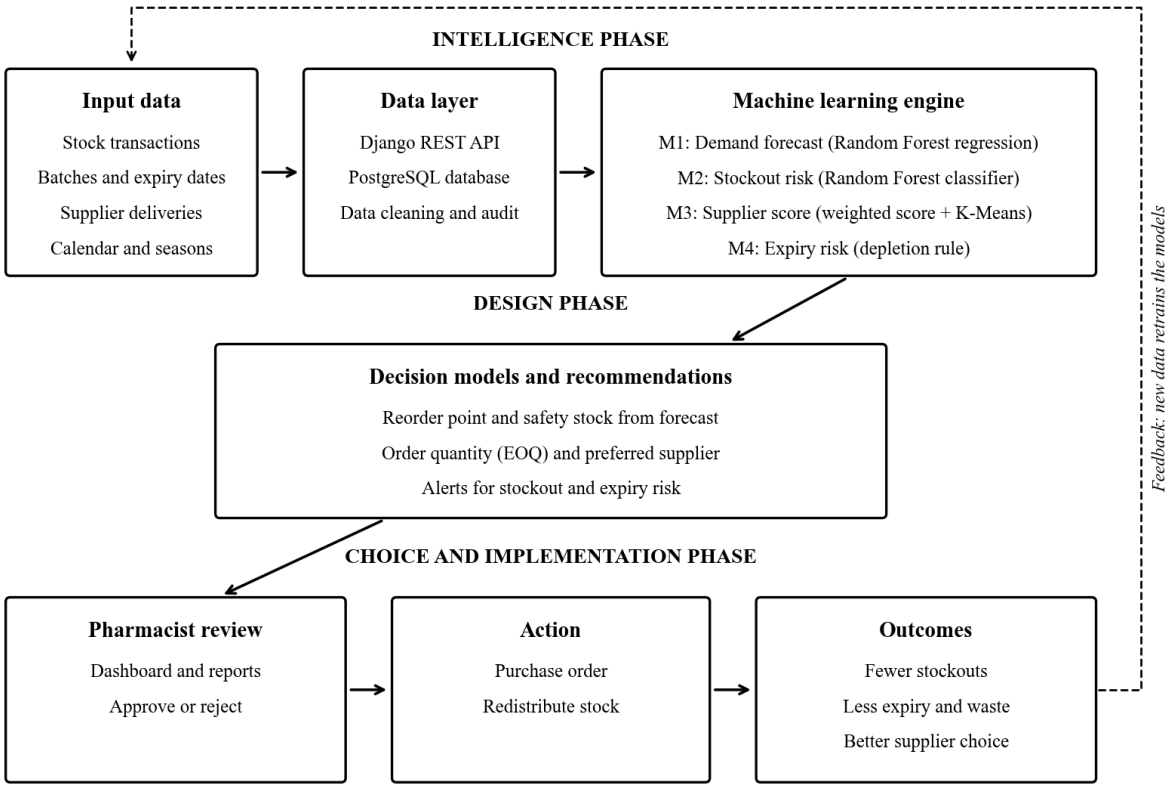


Figure 1.1: Conceptual model of the proposed IDSS (adapted from Simon, 1960)

1.8 Assumptions

The study is based on the following assumptions:

- Past consumption patterns are a fair guide to demand in the near future.
- Stock records captured by pharmacy staff are accurate and up to date.
- Suppliers will behave in future much as they have in the past.
- The hospital will provide computers and a stable local network.
- Pharmacists will review each recommendation before placing an order.
- Users have basic computer skills and can use a web browser.
- The simulated training data captures the main demand patterns found at PGH.

1.9 Ethical Considerations

Health data is sensitive, so the study follows clear ethical rules.

- **Permission.** Written approval will be sought from PGH management and the university first.
- **Informed consent.** Each participant will read a consent note before the questionnaire. Taking part is voluntary, and anyone may withdraw.
- **Anonymity.** No names or staff numbers will be recorded in the questionnaire.
- **Confidentiality.** The system stores medicine data only. It holds no patient names or diagnoses.
- **Data protection.** The study follows the Cyber and Data Protection Act (Government of Zimbabwe, 2021). Passwords are hashed and access is controlled by user role.
- **Honest use of data.** The models were trained on simulated data. Supplier scores are illustrations, not real judgements of any company.
- **Fairness.** Model accuracy is checked for each medicine category. Weak spots are reported openly.
- **Human oversight.** The system never places an order on its own. A pharmacist must approve every action.
- **Use of AI tools.** Any AI writing or coding aid will be declared, as the project guide requires.

- **Originality.** All sources are cited in APA style, and the work is checked with Turnitin.

1.10 Methodology

This section explains how the research was carried out. It also covers how the software was built and which tools were used.

1.10.1 Research Methodology

The study follows the Design Science Research (DSR) approach. DSR builds and tests an artefact that solves a real problem (Hevner et al., 2004). Here, the artefact is the IDSS. The six DSR steps of Peffers et al. (2007) were followed. Table 1.2 shows how each step was applied.

DSR step	Activity in this study
1. Identify the problem	Study stockouts, expiries and manual ordering at PGH pharmacy.
2. Define objectives	Set the seven objectives listed in Section 1.4.2.
3. Design and develop	Train four models in Colab. Build the Django and React system.
4. Demonstrate	Run the system with pharmacy data and sample reorder scenarios.
5. Evaluate	Measure model accuracy. Run user acceptance tests with pharmacy staff.
6. Communicate	Present the results in this dissertation and at the viva.

Table 1.2: Design Science Research steps applied to this study

Within DSR, a mixed methods design was used. The qualitative part explores how staff plan and order medicines. It answers research question one. The quantitative part is experimental. It tests how well each model performs on held-out data. It answers research questions two to four. The final question is answered through system testing and user feedback.

The machine learning work followed the CRISP-DM process (Wirth & Hipp, 2000). Its six stages are business understanding, data understanding and data preparation. These are followed by modelling, evaluation and deployment. Each dataset was first audited for missing values, duplicates and impossible values. Data was then split by date into training, validation and test sets. A 70, 15 and 15 percent split was used. The split was never shuffled, so no future data leaked into training. Five-fold time-series cross-validation then confirmed the ranking of models.

Several candidate models were compared for each task. For demand, these were a naive baseline, SARIMA and Linear Regression. Random Forest was also tested. So were XGBoost (Chen & Guestrin, 2016) and LightGBM (Ke et al., 2017). Forecasts were judged by MAE, RMSE and MAPE. For stockout risk, Logistic Regression, Random Forest, XGBoost and LightGBM were compared. Only about one check in ten was a stockout. Accuracy alone would therefore mislead. Precision, recall, F1 score and PR-AUC were used instead (Saito & Rehmsmeier, 2015). The best model was tuned with Optuna (Akiba et al., 2019). It was kept only if it beat the untuned version on the test set.

1.10.2 Software Development Methodology

The system was built using the Agile Scrum method (Schwaber & Sutherland, 2020). Work was split into two-week sprints. Each sprint delivered a working part of the system. The supervisor acted as the product owner and reviewed each sprint. Pharmacy staff gave feedback on screens as they were built.

The life cycle phases were applied within each sprint as follows.

- **Requirements gathering.** Needs were collected through questionnaires, interviews and observation. They were written as user stories in a product backlog.
- **Design.** Each sprint began with design of the screens, database tables and API endpoints involved.
- **Implementation.** Models were coded in Google Colab. The backend used Django and the frontend used React.
- **Testing.** Unit and integration tests ran at the end of each sprint. User acceptance testing followed the final sprint.
- **Deployment.** The system runs as a web application on the hospital network.
- **Maintenance.** Models are versioned, so a better model can replace an old one safely.

Agile suits this project for four reasons. First, ML work is experimental, and results often change the plan. Second, user needs became clearer only after staff saw early screens. Third, a working version was always ready to show the supervisor. Fourth, the Waterfall model was rejected because it handles change poorly. In Waterfall, a late change to requirements is slow and costly.

1.10.3 Data Collection Tools and Techniques

Data for this study came from people and from records. Table 1.3 lists the tools used.

Tool	How it is used	Type of data
Questionnaire (Google Forms)	Sent online to pharmacists, technicians and stores staff.	Quantitative and qualitative
Semi-structured interviews	Face-to-face talks with senior pharmacists and procurement officers.	Qualitative, unstructured
Observation	Watching stock reviews, receiving and dispensing in the pharmacy.	Qualitative
Document review	Studying stock cards, delivery notes, requisitions and expiry records.	Quantitative, structured
Transaction datasets	Daily demand, reorder checks, supplier orders and batch records.	Quantitative, structured
System logs	Usage and error logs captured during testing.	Quantitative, structured

Table 1.3: Data collection tools and techniques

Purposive sampling was used to select respondents. Only staff who handle medicine stock were included. Their knowledge is directly relevant to the problem.

Real dispensing records could not be released because of confidentiality rules. A structured, simulated dataset was therefore built for model training. It was shaped around 20 essential medicines on the hospital's list. It also reflects Zimbabwe's rainy, cold and transition seasons. The demand file holds 7,320 daily records for the year 2024. The reorder file holds 6,000 risk checks. The supplier file holds 6,000 orders from 12 suppliers. A further 6,000 batch records support the expiry module.

Several steps were taken to keep the data reliable and valid.

- The questionnaire was pilot tested with three people before release.
- The same interview guide was used with every respondent.
- Results from questionnaires, interviews and records were compared to confirm findings.
- Each dataset was audited for gaps, duplicates and impossible values before modelling.
- Models were tested only on data they had never seen during training.

1.10.4 Development Tools and Technologies

Table 1.4 lists the tools used to build the system.

Category	Tool or technology	Purpose

Category	Tool or technology	Purpose
ML environment	Google Colab, Python 3	Free cloud notebooks for training models.
Data handling	pandas, NumPy	Loading, cleaning and shaping data.
ML libraries	scikit-learn, XGBoost, LightGBM, statsmodels	Training and comparing models.
Tuning and explanation	Optuna, SHAP	Tuning settings and explaining predictions.
Visualisation	Matplotlib, Seaborn, Recharts	Charts in notebooks and on dashboards.
Model storage	joblib, JSON metadata	Saving versioned model files for the backend.
Backend	Django 5.2, Django REST Framework	Business logic and REST API.
Security	Simple JWT, role-based permissions	Login, tokens and access control.
Database	PostgreSQL (SQLite in development)	Storing medicines, stock, suppliers and results.
Reports	ReportLab	Producing PDF reports.
Frontend	React 19, TypeScript, Vite	Building the user interface.
UI and data fetching	TanStack Router and Query, Tailwind CSS, shadcn/ui	Pages, styling and server data.
Version control	Git, GitHub, Lovable	Tracking code changes and backups.
Editor and diagrams	Visual Studio Code, draw.io	Writing code and drawing UML diagrams.

Table 1.4: Development tools and technologies

1.11 Scope and Limitations

1.11.1 Scope

The system covers the main stores and dispensary of the PGH Pharmacy Department. It includes the following modules:

- User login with four roles: administrator, manager, pharmacist and staff.
- Medicine and inventory records, including batches and stock movements.
- Demand forecasting for each medicine.
- Stockout risk alerts and reorder recommendations with suggested quantities.
- Supplier performance scores and tiers.
- Expiry risk monitoring for batches.

- Dashboards, notifications, PDF and CSV reports, and an audit trail.

Some areas fall outside the scope. The system does not handle patient records or prescriptions. It does not make payments or send orders to suppliers automatically. Direct links to eLMIS, Impilo and NatPharm are left for future work. A mobile app and IoT sensors are also not included.

1.11.2 Limitations

- **Data access.** Real records were confidential, so models were trained on simulated data. Results must be confirmed with live hospital data.
- **Short history.** One year of demand data limits how well yearly seasons are learned.
- **Hidden demand.** On stockout days, true demand is unknown because nothing could be issued.
- **Few suppliers.** Only 12 suppliers were scored, which limits the clustering results.
- **Time.** The project had to fit within one academic year.
- **Resources.** Training used the free tier of Google Colab, which limits computing power.
- **User availability.** Busy pharmacy staff had limited time for interviews and testing.
- **Power and network.** Load shedding and slow internet affected some development work.

1.12 Project Timeline

The project runs for 34 weeks, from 3 August 2026 to 26 March 2027. Table 1.5 lists each task, its length and the tasks it depends on. Slack is the number of weeks a task can slip without delaying the project.

ID	Task	Weeks	Depends on	Start	End	Slack (wks)
A	Proposal and topic approval	3	-	03 Aug 2026	21 Aug 2026	0
B	Requirements gathering at PGH	2	A	24 Aug 2026	04 Sep 2026	4
C	Literature review	6	A	24 Aug 2026	02 Oct 2026	0
D	Data preparation and audit	2	B	07 Sep 2026	18 Sep 2026	8
E	ML model training and evaluation	5	D	21 Sep 2026	23 Oct 2026	8
F	System analysis and design	4	B,C	05 Oct 2026	30 Oct 2026	0
G	Backend development (Django)	7	F	02 Nov 2026	18 Dec 2026	0

ID	Task	Weeks	Depends on	Start	End	Slack (wks)
H	Frontend development (React)	7	F	02 Nov 2026	18 Dec 2026	3
I	Model integration	3	E,G	21 Dec 2026	08 Jan 2027	0
J	Testing and user acceptance	4	I,H	11 Jan 2027	05 Feb 2027	0
K	Final documentation	4	J	08 Feb 2027	05 Mar 2027	0
L	Submission and viva preparation	3	K	08 Mar 2027	26 Mar 2027	0

Table 1.5: Project tasks, durations, dependencies and slack

Figure 1.2 shows the same plan as a Gantt chart. Black bars mark critical tasks. Hatched bars mark tasks that have some slack.

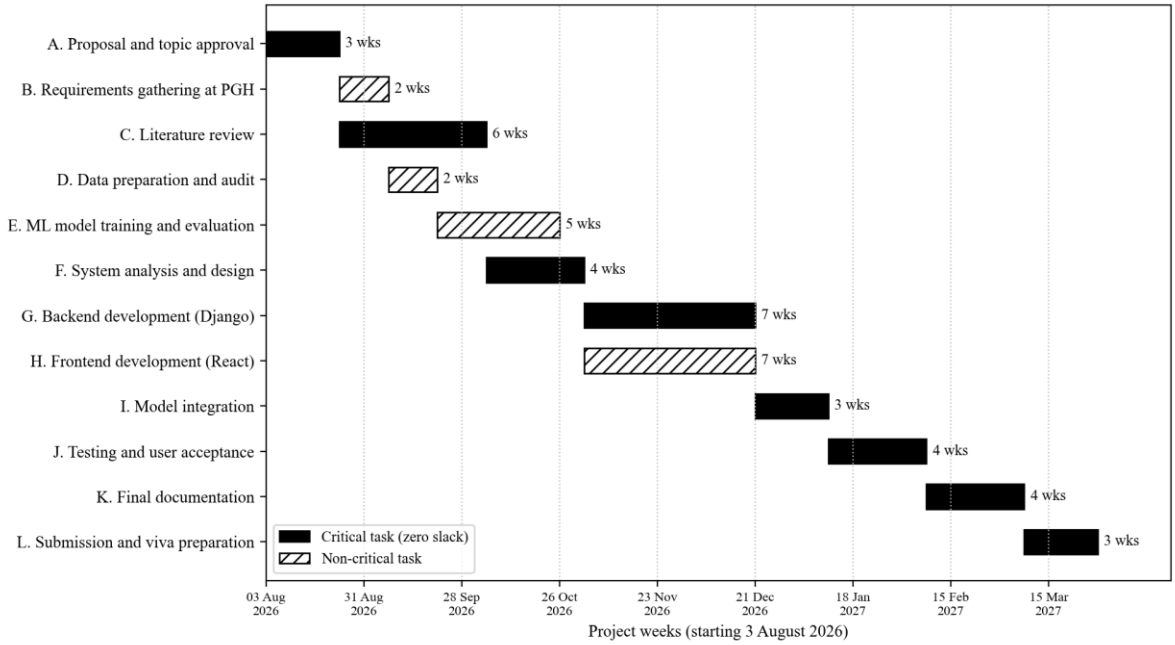


Figure 1.2: Gantt chart showing the critical path of the project

The critical path is A, C, F, G, I, J, K and L. These tasks have zero slack and add up to 34 weeks. A delay in any one of them delays the whole project. Requirements gathering has four weeks of slack. Data preparation and model training share eight weeks of slack. Frontend work has three weeks of slack, as it runs alongside the backend. Close attention was therefore given to the literature review, design and backend work.

1.13 Justification of the Project

Stockouts and expiries both drain the limited budget of a public hospital. A stockout forces emergency buying at higher prices. Worse still, it can interrupt life-saving treatment. An expired batch is money lost and must also be safely destroyed. Manual ordering cannot keep up with hundreds of medicines and shifting seasons. A data-driven system is therefore needed.

Early experiments show that the approach is promising. The Random Forest forecaster cut the average error by about 46 percent. This was compared with simply using the previous day's demand. The stockout classifier was tuned to catch at least 80 percent of real stockout cases. These results come from simulated data and still need confirming with live data. Even so, they show what the system could achieve.

The project is also affordable. It is built with free, open-source tools. It runs in a web browser on computers the hospital already owns. Pharmacists keep full control, since every recommendation needs their approval. For these reasons, the project is both needed and practical.

1.14 Summary

This chapter introduced Parirenyatwa Group of Hospitals and its Pharmacy Department. It showed that current stock systems record data but do not support decisions. This leads to stockouts, expired stock and slow manual ordering. The chapter set out the aim, seven objectives and five research questions. It also presented the theoretical framework and a conceptual model. Design Science Research, CRISP-DM and Agile Scrum were chosen as methods. The scope, limitations and a 34-week timeline were also given. The next chapter reviews related literature and assesses the feasibility of the system.

CHAPTER TWO: LITERATURE REVIEW AND FEASIBILITY STUDY

2.0 Introduction

Chapter One showed that the PGH Pharmacy Department records stock but lacks decision support. This chapter looks at what others have already done to solve similar problems. It starts with work done in Zimbabwe and the wider region. It then reviews global research and systems. From this review, the gap that this project fills becomes clear. The second half of the chapter tests whether the proposed system is feasible. Technical, operational, schedule, economic and security aspects are all assessed.

2.1 What Has Been Done Locally

This section reviews work done in Zimbabwe and in nearby African countries. It covers local systems, studies of how well they work, and government policy.

2.1.1 The State of Zimbabwe's Medicine Supply Chain

Zimbabwe's public health sector relies on NatPharm as its central medicine supplier. NatPharm buys medicines, stores them in regional warehouses and delivers them to facilities. Central hospitals such as PGH also buy directly from private suppliers when needed. This mixed system has struggled for many years. Foreign currency shortages make it hard to pay foreign suppliers on time. Local manufacturers also depend on imported raw materials. As a result, orders arrive late or only partly filled.

Recent audit work points to weaknesses inside hospitals themselves. Reports by the Auditor-General on public hospitals have noted weak stock control. They also cite inaccurate inventory records and poor forecasting (Office of the Auditor-General, 2025). Some hospitals were found holding expired medicines alongside usable stock. The Auditor-General called for better inventory systems through digitalisation. These findings match the problems described at PGH in Chapter One. They show that the gap is not only about funding. It is also about how stock data is used to make decisions.

2.1.2 From Push Systems to the Zimbabwe Assisted Pull System

Zimbabwe has tried several ways to distribute medicines to facilities. Early programmes used push systems, where central stores decided what each facility received. The Delivery Team Topping Up (DTTU) system was one such approach. Trucks visited clinics and topped up stock to set levels. Over time, separate systems grew for different disease programmes. This created duplication and extra cost.

In response, the MoHCC piloted the Zimbabwe Assisted Pull System (ZAPS) in Manicaland. ZAPS merged several distribution systems into one. Facilities reported their stock, and orders were calculated from that data. ZAPS kept supply performance steady at a lower cost. This was the finding of an official evaluation (USAID | DELIVER PROJECT, 2015). However, ZAPS still relied on simple reorder formulas based on past averages. It did not forecast seasonal changes or score suppliers. At hospital level, some older systems also continued to run alongside it.

2.1.3 Digital Health Systems in Zimbabwe

Zimbabwe has also invested in electronic systems for health data. The national electronic Logistics Management Information System (eLMIS) collects stock reports from facilities. It gives central managers better visibility of stock across the country. Its main role, however, is reporting rather than prediction.

Impilo is Zimbabwe's national electronic health record. It was developed locally under the MoHCC and piloted from 2016. It supports patient registration, tracking, stock monitoring and data analysis. Mugauri and Chimsimbe (2025) evaluated Impilo at ten facilities in Harare and Bulawayo. They interviewed 45 health workers about their experience. Poor connectivity, weak power backup and staff shortages limited its use. Interoperability problems and restricted editing rights were also reported. The study shows that local digital systems can work. Yet it also shows they must cope with power and network failures.

2.1.4 Regional Advances

Several African countries have tried technology to improve medicine availability. Four examples are most relevant to this study.

In Tanzania, the SMS for Life pilot used basic mobile phones to report stock levels. Health workers sent weekly antimalarial stock counts by text message. District managers could then move stock before facilities ran out. Over 21 weeks, facilities without antimalarials fell from 78% to 26% (Barrington et al., 2010). The project proved that simple, timely data can cut stockouts. However, it only showed current stock and did not predict future need.

South Africa introduced the Stock Visibility System (SVS) in primary health care clinics. Staff scan medicine barcodes with a smartphone to record stock. Managers can then see stock levels of ARVs, TB medicines and vaccines. Mbonane et al. (2023) studied 206 health workers using SVS in KwaZulu-Natal. Most had been trained and held positive views of the system. Yet only 17% showed good practice in using it. The authors called for continuous training. This lesson is important for any new system at PGH.

Zambia rolled out an eLMIS from 2014, starting with a six-month pilot. An endline evaluation found that facilities using the computerised edition had fewer stockouts. They experienced 15.5% fewer stockouts than similar facilities without it (AIDSFree Project, 2019). This shows the value of digital stock records in a setting like Zimbabwe's.

Rwanda has gone a step further by applying machine learning to medicine demand. Mbonyinshuti et al. (2022) used consumption data from Rwanda's eLMIS. They compared linear regression, artificial neural networks and Random Forest. Random Forest gave the most reliable forecasts for the medicines studied. This is the closest regional study to the present project. However, it focused on forecasting alone. It did not link forecasts to reorder decisions, supplier choice or expiry risk.

2.1.5 Successes, Failures and Limitations of Local Efforts

The local and regional work shows clear successes. Digital stock reporting improves visibility and reduces stockouts. Simple tools such as SMS can work even in rural areas. Machine learning has been shown to forecast medicine demand in an African setting.

Yet there are also clear limitations. Most local systems only record and report stock. Few turn data into a clear recommendation. Power cuts, poor networks and staff shortages often limit use. Training does not always lead to good daily practice. No local study was found that

combines forecasting, stockout risk, supplier scoring and expiry risk. This gap is the starting point for the proposed system.

2.1.6 Government Policies and Initiatives

Several Zimbabwean policies support the use of AI in health supply chains. Vision 2030 aims to turn Zimbabwe into an upper middle-income economy (Government of Zimbabwe, 2018). It names health and digital technology as key enablers. NDS1 calls for better health services and wider use of ICT (Government of Zimbabwe, 2020). The National Health Strategy 2021 to 2025 also aims to widen access to medicines. This is set out by the Ministry of Health and Child Care (2021). Education 5.0 also pushes universities to produce innovation and industrial solutions.

Some policies set limits that the system must respect. The Cyber and Data Protection Act governs personal data (Government of Zimbabwe, 2021). It names POTRAZ as the data protection authority. The Medicines and Allied Substances Control Act gives MCAZ control over medicine registration. The system must therefore only recommend suppliers and products that MCAZ has approved. Procurement by public hospitals must also follow public procurement law. For this reason, the system recommends orders but never places them by itself.

2.2 What Has Been Done Globally

This section reviews international research and systems. It focuses on supply chain optimisation, machine learning forecasting, stockout prediction and supplier evaluation.

2.2.1 Optimising the Pharmaceutical Supply Chain

Researchers have long used mathematical models to manage hospital medicine stock. Uthayakumar and Priyan (2013) built an inventory model for a pharmaceutical company and a hospital. Their model reduced total cost while keeping service levels high. Settanni et al. (2017) reviewed many operations research models for pharmaceutical supply chains. They found that most models treat demand as known or follow a simple pattern. In real hospitals, demand changes with disease seasons and outbreaks. This weakness opened the door for data-driven forecasting.

2.2.2 Machine Learning for Demand Forecasting

Carbonneau et al. (2008) were among the first to test machine learning for supply chain demand. They compared neural networks and support vector machines with traditional methods. The machine learning models gave better forecasts when demand was noisy. Since then, tree-based models have become popular for this task.

The M5 forecasting competition is one of the largest tests of forecasting methods. It used daily sales data for over 30,000 retail products (Makridakis et al., 2022). Gradient-boosted tree models such as LightGBM won most top places. Many winners trained one global model across all products. This lets slow-moving items learn from patterns in faster-moving ones. The present project uses the same global approach across 20 medicines.

Fourkiotis and Tsadiras (2024) applied similar methods to pharmacy sales data. XGBoost performed strongly among the machine learning models they tested. However, other studies warn that simple statistical models can still win on small datasets. For this reason, the present project compares several models fairly. A naive baseline, SARIMA and Linear Regression are included alongside tree-based models.

2.2.3 Predicting Stockouts and Backorders

Some studies predict stockouts directly instead of forecasting demand. de Santis et al. (2017) used machine learning to predict material backorders. Tree-based models performed well, but the data was highly imbalanced. Only a small share of items ever went on backorder. Saito and Rehmsmeier (2015) showed that accuracy is misleading on such data. They recommend the precision-recall curve instead. The present project follows this advice for its stockout model. It also sets the alert threshold to catch at least 80% of real stockouts.

2.2.4 Supplier Evaluation and Selection

Supplier evaluation has a long history in purchasing research. Dickson (1966) listed 23 criteria that buyers use to choose suppliers. Quality, on-time delivery and price were ranked among the most important. Ho et al. (2010) reviewed later multi-criteria methods for supplier selection. They found weighted scoring, the Analytic Hierarchy Process and data envelopment analysis widely used. Most of these methods depend on expert judgement for

weights. Few studies combine a weighted score with clustering to group suppliers into tiers. Fewer still check whether the ranking stays stable when weights change.

2.2.5 Commercial and Hospital Systems

In high-income countries, large hospitals use enterprise resource planning (ERP) systems. These systems link procurement, inventory and finance in one platform. Many include demand planning modules and automated reorder points. Automated dispensing cabinets also track each item as it leaves the shelf. These tools are powerful but costly. They need licences, reliable power and a strong network. They also need trained IT staff to maintain them. Such conditions are rare in Zimbabwe's public hospitals.

2.2.6 Challenges and Lessons Learned

Global experience also shows the limits of these tools. The COVID-19 pandemic disrupted supply chains around the world. Ivanov (2020) showed that sudden shocks break normal demand and supply patterns. Forecasting models trained on normal years then perform poorly. This means human judgement must stay in the loop. A second lesson concerns trust. Users are slow to act on predictions they cannot understand. SHAP helps by showing why a model made each prediction (Lundberg & Lee, 2017). A third lesson concerns data quality. Models are only as good as the stock records they learn from.

2.2.7 Summary of Related Studies

Table 2.1 summarises the most relevant studies reviewed in this chapter.

Study	Setting	Approach	Key finding	Gap for this project
USAID | DELIVER PROJECT (2015)	Zimbabwe	ZAPS pull system	Same performance at lower cost.	No forecasting or supplier scoring.
Mugauri & Chimsimbe (2025)	Zimbabwe	Impilo EHR evaluation	Power and network limit use.	Records data but gives no stock advice.
Barrington et al. (2010)	Tanzania	SMS stock reporting	Stockouts fell from 78% to 26%.	Shows current stock only.
Mbonane et al. (2023)	South Africa	SVS mobile app	Good attitudes, weak practice.	Visibility only, no prediction.
AIDSFree Project (2019)	Zambia	eLMIS	15.5% fewer stockouts.	Reporting tool, not decision support.
Mbonyinshuti et al.	Rwanda	ML demand	Random Forest	Forecast not linked to

Study	Setting	Approach	Key finding	Gap for this project
(2022)		forecasting	forecast well.	reorder decisions.
Makridakis et al. (2022)	Global retail	M5 competition	Global boosted trees did best.	Retail data, not medicines.
de Santis et al. (2017)	Global industry	ML backorder prediction	Trees handle rare events well.	Not applied to medicines.
Ho et al. (2010)	Global	Supplier selection review	Weighted scoring widely used.	Weights rarely stress-tested.

Table 2.1: Summary of related studies

2.3 What New Idea Is This Project Bringing?

2.3.1 Comparison with Existing Systems

Table 2.2 compares the proposed system with the main existing approaches.

Feature	Stock cards and spreadsheets	eLMIS / ZAPS	Impilo EHR	SVS (South Africa)	Commercial ERP	Proposed IDSS
Records stock movements	Yes	Yes	Partly	Yes	Yes	Yes
Forecasts demand with ML	No	No	No	No	Some	Yes
Predicts stockout risk	No	No	No	No	Some	Yes
Scores and tiers suppliers	No	No	No	No	Some	Yes
Flags expiry risk early	No	Partly	No	No	Yes	Yes
Explains its advice	No	No	No	No	Rarely	Yes
Pharmacist approval step	Manual	Manual	No	No	Optional	Built in
Low cost, open source	Yes	Donor funded	Yes	Donor funded	No	Yes

Table 2.2: Comparison of existing systems with the proposed IDSS

2.3.2 Originality and Novelty

The proposed system differs from existing work in six ways.

1. It combines four decision models in one system. These are demand forecasting, stockout risk, supplier scoring and expiry risk.

2. It feeds the forecast into the reorder point. Reorder levels therefore rise and fall with the seasons.
3. Its demand model includes Zimbabwe's rainy, cold and transition seasons, plus public holidays.
4. Its stockout alert is tuned to catch at least 80% of real stockouts. This puts patient safety ahead of raw accuracy.
5. Its supplier scores carry a confidence flag. Suppliers with few orders are marked as uncertain.
6. Every recommendation shows its reasons and needs a pharmacist's approval. The system supports people rather than replacing them.

2.3.3 Potential Contributions

The project adds value in several ways. For PGH, it offers a working tool that turns stock data into daily advice. For research, it gives local evidence on machine learning in a Zimbabwean hospital. It also shows how to test models honestly on time-based data. For practice, it offers a low-cost design that other public hospitals could adopt. Finally, its open design allows future links to eLMIS and Impilo.

2.4 Feasibility Study of the Proposed System

A feasibility study checks whether a system can and should be built. This section assesses the proposed IDSS from five angles.

2.4.1 Technical Feasibility

Technical feasibility asks whether the needed technology is available and suitable. Three questions guided this assessment.

Is the required technology available locally?

Yes. All the software used is free and open source. Python, Django, React and PostgreSQL can be downloaded anywhere. Google Colab is reachable from Zimbabwe through any internet connection. The hardware needed is ordinary office equipment sold in Harare. No special sensors or imported devices are required.

Can existing technologies be used effectively?

Yes. Early experiments confirm that the chosen methods work on the project data. The Random Forest forecaster beat the naive baseline by a wide margin. The stockout classifier reached a PR-AUC of about 0.81 on unseen data. The supplier rankings stayed stable when the scoring weights were changed by 30%. The trained models are small files that load quickly on a normal computer. The system runs in any modern web browser. It can also run on the hospital's local network if the internet goes down.

What resources are needed?

Table 2.3 lists the hardware, software and skills needed to build and run the system.

Resource	Requirement	Availability
Server computer	Core i5 or better, 16 GB RAM, 512 GB SSD	Can be bought locally
Client computers	Any PC with a modern web browser	Already in the pharmacy
Power backup	UPS and inverter battery	Can be bought locally
Network	Hospital local area network	Already in place
Operating system	Windows or Linux	Available
Software stack	Python, Django, React, PostgreSQL	Free and open source
Model training	Google Colab	Free tier available
Skills	Python, web development, machine learning	Held by the researcher
Support	IT officer for backups and updates	Hospital IT department

Table 2.3: Technical resources required

The technical assessment shows that the system is technically feasible.

2.4.2 Operational Feasibility

Operational feasibility asks whether the system will work in daily practice. Three questions guided this assessment.

Can the people involved handle the new technology?

Pharmacy staff already use computers for spreadsheets, eLMIS reports and email. The new system uses familiar web pages, tables and charts. Each user sees only the screens that match their role. Colour-coded alerts show at a glance which medicines need attention. This keeps the learning curve short.

Will the system fit with existing operations?

The system follows the pharmacy's current workflow rather than replacing it. Staff still review stock and approve orders as they do today. The difference is that the system prepares the evidence for them. Stock data can be imported from existing spreadsheets. The system does not replace eLMIS or Impilo. It sits alongside them and adds decision support.

What training is needed?

Lessons from South Africa show that training must continue after launch (Mbonane et al., 2023). Two short hands-on sessions will be held for pharmacists and technicians. A simple user guide will be left in the pharmacy. One IT officer will be trained to manage users, backups and model updates. Refresher sessions are planned every six months.

Pharmacy staff showed interest in a tool that reduces manual stock reviews. Management also supports any tool that cuts waste. The system is therefore operationally feasible.

2.4.3 Schedule Feasibility

Schedule feasibility asks whether the project can be finished on time. The project runs over 34 weeks in four stages.

1. Planning and design: proposal, requirements gathering, literature review and system design.
2. Development and implementation: data preparation, model training, backend and frontend coding, and integration.
3. Testing and evaluation: unit, integration, model and user acceptance testing.
4. Deployment and maintenance: installation, training, documentation and the viva.

Figure 2.1 shows the timeline as a Gantt chart. Some tasks run at the same time. For example, the literature review runs alongside requirements gathering. Model training also runs alongside system design. Backend and frontend development run in parallel too. Other tasks must follow a strict order. Integration cannot start until the models and backend are ready. Testing must wait for integration to finish.

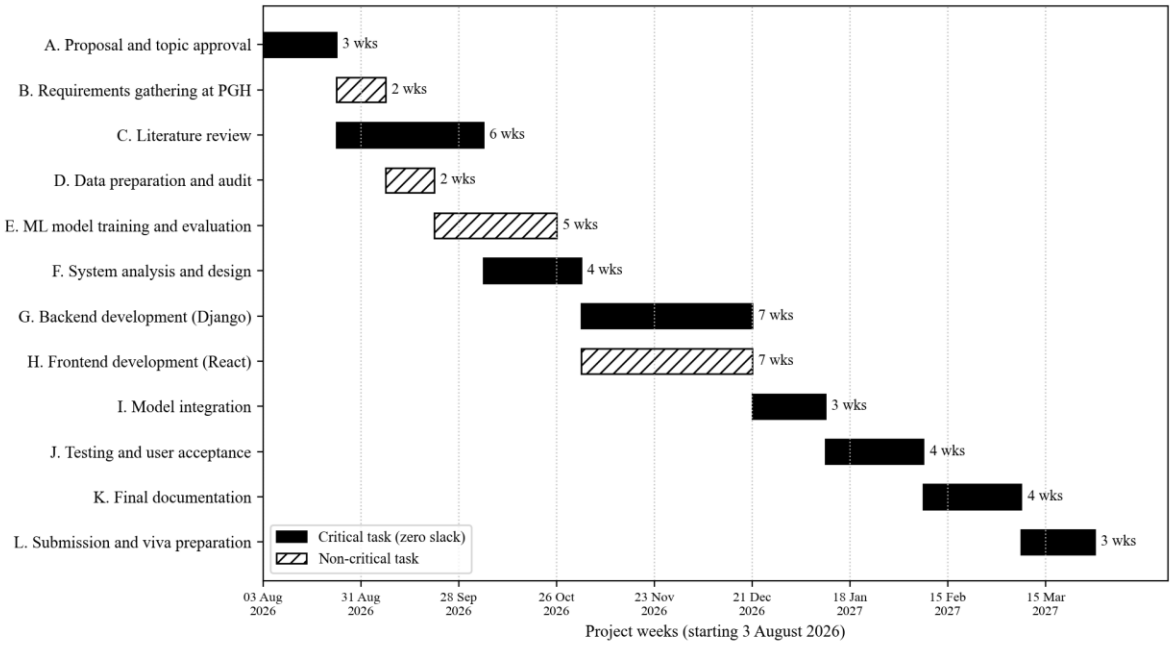


Figure 2.1: Project Gantt chart showing concurrency and sequencing of tasks

The critical path takes 34 weeks and ends on 26 March 2027. This is before the expected submission date. Non-critical tasks have between three and eight weeks of slack. The schedule is therefore feasible.

2.4.4 Economic Feasibility

Economic feasibility asks whether the benefits of the system outweigh its costs. All amounts are in United States dollars (USD), the main trading currency in Zimbabwe. Prices are estimates based on local market prices in 2026. Benefit figures are planning estimates. They should be updated with real PGH expiry and purchase records once available.

Costs

Table 2.4 shows the one-off development costs and yearly running costs.

Cost item	Type	Amount (USD)
Server computer (Core i5, 16 GB RAM, SSD)	Once-off	850
UPS and inverter battery for load shedding	Once-off	350
Network switch and cabling	Once-off	120
Internet data during development (6 months)	Once-off	240
Data collection, printing and stationery	Once-off	90

Cost item	Type	Amount (USD)
User training (two sessions)	Once-off	250
Developer labour (400 hours at USD 4 per hour, in kind)	Once-off	1,600
Total development cost		3,500
System maintenance and support	Yearly	600
Internet connectivity	Yearly	480
Electricity and battery upkeep	Yearly	150
Cloud notebook subscription for retraining	Yearly	120
Total yearly running cost		1,350

Table 2.4: Estimated costs of the proposed system

Benefits

Table 2.5 shows the expected yearly benefits once the system is fully in use. Only 60% of these benefits are assumed in the first year while staff adapt.

Benefit	How it arises	Amount (USD)
Less expired stock	Slow-moving batches are flagged early and used or moved.	2,600
Fewer emergency purchases	Stockouts are seen early, so costly urgent buying drops.	2,200
Staff time saved	About five hours a week saved on stock reviews.	1,300
Total yearly benefit		6,100

Table 2.5: Estimated yearly benefits of the proposed system

The system also brings benefits that are hard to price. Patients face fewer treatment breaks. Decisions become consistent and easy to audit. The hospital also gains better data for budgeting and tenders.

Net Present Value (NPV)

NPV converts future cash flows into today's value. A discount rate of 12% was used to reflect borrowing costs in Zimbabwe. The formula is shown below.

$$NPV = \Sigma \left[\text{Net benefit in year } t \div (1 + r)^{t} \right] - \text{Initial cost}$$

Year	Benefits (USD)	Running costs (USD)	Net benefit (USD)	Discount factor	Present value (USD)
0	-	3,500	-3,500	1.0000	-3,500
1	3,660	1,350	2,310	0.8929	2,062
2	6,100	1,350	4,750	0.7972	3,787

Year	Benefits (USD)	Running costs (USD)	Net benefit (USD)	Discount factor	Present value (USD)
3	6,100	1,350	4,750	0.7118	3,381
Total	15,860	7,550	8,310		NPV = 5,730

Table 2.6: Three-year cash flow and NPV at a 12% discount rate

The NPV is about USD 5,730. A positive NPV means the project creates more value than it costs.

Return on Investment (ROI)

ROI measures the gain from the project compared with its total cost.

$$ROI = (Total\ benefits - Total\ costs) \div Total\ costs \times 100$$

$$ROI = (15{,}860 - 7{,}550) \div 7{,}550 \times 100 = 110.1\%$$

Over three years, every dollar spent returns about two dollars and ten cents.

Payback Period

The payback period is the time needed to recover the initial cost. By the end of year one, USD 2,310 of the USD 3,500 has been recovered. The remaining USD 1,190 is recovered in about three months of year two. The payback period is therefore about 1.25 years, or 15 months.

Recommendation

The project has a positive NPV and an ROI above 100%. Its payback period is also under two years. It is therefore economically feasible. It is recommended that the project proceeds. To keep costs low, the system should run on the hospital's own network. Real expiry and purchase data should be used to confirm the benefits after six months.

2.4.5 Threats to Security and Controls

The system holds stock, supplier and user data. A breach could disrupt medicine supply or expose sensitive records. Table 2.7 lists the main threats and the controls built into the system.

Threat	Possible impact	Control in the system

Threat	Possible impact	Control in the system
Unauthorised access	Outsiders view or change stock data.	Login with JSON Web Tokens. Access tokens expire after 15 minutes.
Password guessing	Attackers break into user accounts.	Login limited to 10 attempts per minute. Passwords are hashed by Django.
Stolen session token	An attacker reuses a valid login.	Refresh tokens rotate and old ones are blacklisted.
Misuse by insiders	Staff approve orders beyond their role.	Role-based permissions for admin, manager, pharmacist and staff.
Data interception	Data is read while moving over the network.	HTTPS with HSTS and secure cookies in production.
Cross-site attacks	Malicious sites send requests as the user.	CSRF protection and an allow-list of trusted origins.
Tampering with models	A false model gives harmful advice.	Versioned model files with metadata. Only admins can replace them.
Data poisoning	Bad records skew predictions.	Data audit before retraining. Pharmacist approval for every order.
Undetected changes	Actions cannot be traced.	Audit trail records who did what and when.
Data loss	Power cuts or disk failure destroy records.	UPS power backup and daily database backups.

Table 2.7: Security threats and controls

Two further controls are recommended before full deployment. First, multi-factor authentication should be added for administrator accounts. Second, continuous monitoring of login failures should alert the IT officer. A clear incident response plan will also guide staff if a breach occurs. These controls follow the ISO/IEC 27001 standard for information security (International Organization for Standardization, 2022). With these controls in place, the security risks are manageable.

2.5 Summary

This chapter reviewed what has been done locally and globally. Zimbabwe has made progress with ZAPS, eLMIS and Impilo. Regional projects in Tanzania, South Africa, Zambia and Rwanda show that digital tools reduce stockouts. Global research shows that tree-based machine learning forecasts demand well. However, no reviewed system combines forecasting, stockout risk, supplier scoring and expiry risk. Few systems also explain their advice or keep the pharmacist in control. The proposed IDSS fills this gap. The feasibility study found the system technically, operationally and economically feasible. It can be

finished on schedule, and its security risks can be controlled. The next chapter describes the research methodology in detail.

CHAPTER THREE: METHODOLOGY

3.0 Introduction

Chapter Two found no local system that joins all four decision tasks. This chapter explains how the study set out to fill that gap. It describes the research approach and the steps taken to build and test the system. It also presents the test plan for the machine learning models and the software. The chapter ends with the ethical rules that guided the work.

3.1 Research Methodology

3.1.1 Research Philosophy and Design

This study takes a pragmatic view. Pragmatism judges research by whether it solves a real problem (Creswell & Creswell, 2018). It allows the researcher to use whatever methods best answer each question. For this reason, the study combines four approaches. Design Science Research is the main approach, since the goal is a working system. An experimental approach is used to test the machine learning models. Quantitative and qualitative methods are used to gather and analyse data. Table 3.1 shows how each approach links to the research questions from Chapter One.

Research question	Approach	Data source
RQ1: How does the pharmacy plan and order medicines?	Qualitative and quantitative	Questionnaire, interviews, observation
RQ2: How accurately can ML forecast daily demand?	Experimental	Demand transactions dataset
RQ3: Can a classifier detect stockout risk early?	Experimental	Reorder risk dataset
RQ4: How can suppliers be ranked fairly?	Experimental and quantitative	Supplier orders dataset
RQ5: Can the models form one usable system?	Design Science	System tests and user feedback

Table 3.1: Research approaches mapped to research questions

3.1.2 Design Science Approach

Design Science Research builds and tests a solution to a problem (Hevner et al., 2004). In this study, the artefact is the Intelligent Decision Support System. The five steps in the project guide were followed.

1. Problem identification. Stockouts, expiries and manual ordering at PGH were studied through questionnaires and interviews.
2. Designing the artefact. Four decision models and a web system were designed. Chapter Four presents the full design.
3. Implementation. Models were trained in Google Colab. The backend was built in Django and the frontend in React.
4. Evaluation. Each model was tested on unseen data. The full system is tested against the test plan in Section 3.2.
5. Reflection. Results were compared with the objectives. Weak areas were recorded for future work.

This approach suits the project because the main output is a working prototype. Each design choice is tested rather than assumed.

3.1.3 Experimental-Based Approach

An experimental approach was used to test the machine learning models. It follows five steps: hypothesis, experiment design, data collection, analysis and conclusion.

Hypotheses

Three hypotheses were set before training began.

- **H1.** A machine learning model forecasts daily medicine demand with lower error than a naive baseline.
- **H2.** A classifier detects stockout risk far better than a model that always predicts no stockout.
- **H3.** Supplier rankings stay stable when the scoring weights are changed by up to 30%.

Experiment Design

Each experiment compared several candidate models on the same data. A simple baseline was always included as the bar to beat. The independent variable was the type of model. The dependent variables were the error and detection scores in Section 3.2.4. Data was split by date, never at random. The first 70% of each series was used for training. The next 15% was

used for validation and the final 15% for testing. This mirrors real use, where a model learns from the past and predicts the future. Five-fold time-series cross-validation was then used to confirm each ranking (Hyndman & Athanasopoulos, 2021). The best model was tuned with Optuna over 40 trials (Akiba et al., 2019). A tuned model was kept only if it beat the untuned one on test data.

3.1.4 Quantitative Approach

The quantitative approach measures numerical variables and analyses them statistically. Two kinds of numbers were collected. The first came from the questionnaire, such as how often stockouts occur. The second came from the stock and supplier datasets. Model performance was measured with error metrics, detection metrics and clustering scores. Descriptive statistics, regression and classification were used to analyse these figures.

3.1.5 Qualitative Approach

The qualitative approach explores how staff work and what they need. Semi-structured interviews and observation were used. Interview notes were analysed with thematic analysis (Braun & Clarke, 2006). Notes were first read several times. Key ideas were then coded and grouped into themes. Expected themes included ordering practice, causes of stockouts and trust in computer advice. These themes shaped the system requirements in Chapter Four.

3.1.6 Population and Sampling

The target population is staff who handle medicine stock at the PGH Pharmacy Department. This includes pharmacists, pharmacy technicians, stores clerks and procurement officers. Purposive sampling was used to select respondents (Etikan et al., 2016). Only people with direct knowledge of stock control were chosen. A sample of 20 questionnaire respondents was targeted. Five key informants were chosen for in-depth interviews. These included senior pharmacists and a procurement officer.

3.1.7 Data Collection Instruments

Table 3.2 describes each data collection instrument.

Instrument	Content	Administration
Questionnaire	Section A: role and experience. Section B:	Google Forms link sent to

Instrument	Content	Administration
	current ordering practice. Section C: stockouts and expiries. Section D: views on a decision support tool.	staff. Paper copies for those without data.
Interview guide	Ordering steps, data used, supplier problems, and features wanted in a new system.	Face-to-face, about 30 minutes each. Notes taken with consent.
Observation checklist	Stock review steps, records used, time taken and problems seen.	Two observation visits to the pharmacy stores.
Document review form	Stock cards, delivery notes, requisitions and expiry records.	Reviewed on site. No copies of patient data taken.

Table 3.2: Data collection instruments

The questionnaire was pilot tested with three people before release. Unclear questions were reworded after the pilot. The consent form and questionnaire appear in Appendix A and Appendix B.

3.1.8 Datasets for Model Development

Real dispensing records could not be released because of confidentiality rules. Four structured datasets were therefore simulated for model development. They were shaped around 20 essential medicines used at PGH. They also reflect Zimbabwe's rainy, cold and transition seasons and public holidays. Supplier names reflect firms active in Zimbabwe, but their scores are simulated. Table 3.3 summarises the four datasets.

Dataset	Size	Period	Target	Used by
Demand transactions	7,320 rows, 11 columns	1 Jan to 31 Dec 2024	Quantity dispensed per day	Model 1
Reorder risk checks	6,000 rows, 13 columns	15 Feb to 12 Dec 2024	Stockout occurred (9.78% yes)	Model 2
Supplier orders	6,000 rows, 19 columns	2023 to early 2025	None (unsupervised)	Model 3
Batch expiry records	6,000 rows, 14 columns	2022 to 2027	Expired unused (30.43% yes)	Model 4

Table 3.3: Datasets used for model development

3.1.9 Data Preparation and Feature Engineering

Every dataset was audited before modelling. The audit checked for missing values, duplicate rows and impossible values. All four files had no missing cells and no exact duplicate rows.

For Model 1, calendar features were added for weekday, month and weekend. Lag features recorded demand 1, 7, 14 and 30 days earlier. Rolling means and standard deviations over 7 and 30 days were also added. Each lag was shifted before averaging. This stops a row from seeing its own answer. The first 30 days of each medicine were dropped because they lacked full history. This left 6,720 rows for modelling.

For Model 2, three features were built from inventory logic. Demand variability is the standard deviation of demand divided by its mean. Lead time margin is days of stock left minus the supplier lead time. A negative margin means stock will run out before the order arrives. Lead time variability was also scaled by the lead time. Medicine category was one-hot encoded, giving 21 features in total.

For Model 3, 6,000 orders were grouped into 12 supplier records. Four measures were computed for each supplier. These were on-time rate, fulfilment rate, price variance and defect rate. Each was scaled to a range of 0 to 1. Measures where lower is better were reversed before scoring.

For Model 4, days to depletion was computed from current stock and monthly use. The coverage gap is days to depletion minus days to expiry. A positive gap means some stock will expire before it is used.

3.1.10 Model Development

Table 3.4 lists the candidate models compared for each task.

Model	Task type	Candidates compared	Selection rule
M1: Demand forecast	Regression	Naive lag-1, SARIMA, Linear Regression, Random Forest, XGBoost, LightGBM	Lowest mean MAE in cross-validation
M2: Stockout risk	Binary classification	Majority-class baseline, Logistic Regression, Random Forest, XGBoost, LightGBM	Highest mean PR-AUC in cross-validation
M3: Supplier score	Unsupervised scoring	Weighted score with K-Means and hierarchical clustering	Stability and agreement checks
M4: Expiry risk	Rule and classification	Depletion rule, Logistic Regression, Random Forest	Must beat the transparent rule

Table 3.4: Candidate models for each decision task

Random Forest combines many decision trees and averages their votes (Breiman, 2001). It handles non-linear patterns and needs little tuning. Class weighting was used in Model 2 because stockouts were rare. In Model 3, the weights were 0.35 for on-time delivery and 0.30 for fulfilment. Price variance was weighted 0.20 and defect rate 0.15. Late and short deliveries were weighted highest because they cause stockouts. K-Means then grouped suppliers into Preferred, Acceptable and At-Risk tiers.

Each final model was saved with joblib alongside a JSON metadata file. The metadata records the version, feature list, metrics and decision threshold. The Django backend loads these files to make predictions.

3.1.11 Data Analysis

Questionnaire data was exported from Google Forms and analysed with Python and Excel. Frequencies, percentages and charts were used to summarise answers. Interview data was analysed with thematic analysis, as described in Section 3.1.5. Model results were analysed with the metrics in Section 3.2.4. SHAP values were used to explain which features drove each prediction (Lundberg & Lee, 2017).

3.2 Test Plan for the Proposed Model

The test plan sets out how the models and the full system are checked. It covers objectives, environment, data, metrics, test cases, results and issue reporting.

3.2.1 Test Objectives

- To confirm each model beats a simple baseline on data it has never seen.
- To confirm no future information leaks into model training.
- To find where each model is weak, by medicine category and season.
- To confirm every system feature works as the requirements state.
- To confirm the system blocks unauthorised users and actions.
- To confirm pharmacy staff find the system useful and easy to use.

3.2.2 Test Environment

Component	Specification

Component	Specification
Model training	Google Colab, Python 3, scikit-learn, XGBoost, LightGBM, Optuna, SHAP
Development machine	Laptop running Windows 11 with Python 3.10 and Node.js
Backend	Django 5.2, Django REST Framework, scikit-learn 1.6.1
Database	SQLite for development, PostgreSQL for deployment
Frontend	React 19 with TypeScript, built with Vite
Browsers	Google Chrome and Microsoft Edge
Unit testing	Django test runner
User testing	Pharmacy computers on the hospital network

Table 3.5: Test environment

3.2.3 Test Data

Model tests used the held-out final 15% of each dataset. For demand, this covered 11 November to 31 December 2024. For stockout risk, it covered checks from 18 October to 12 December 2024. For expiry risk, 560 batches were kept for testing. System tests used sample medicines, batches, suppliers and four user accounts. One account was created for each role. Edge cases included zero demand, zero stock, missing values and expired tokens.

3.2.4 Performance Metrics

Metric	What it measures	Target
MAE	Average forecast error in units per day	Lower than the naive baseline
RMSE	Forecast error that punishes large misses	Lower than the naive baseline
MAPE	Average forecast error as a percentage	Lower than the naive baseline
Precision	Share of stockout alerts that are real	At least 0.50
Recall	Share of real stockouts that are caught	At least 0.80
PR-AUC	Overall skill at finding rare stockouts	Well above the 0.09 baseline
Silhouette score	How well suppliers fit their tier (Rousseeuw, 1987)	Near the best value for any k
Adjusted Rand Index	Agreement between two clustering methods (Hubert & Arabie, 1985)	Above 0.80
Spearman correlation	Ranking stability when weights change	Above 0.90
Response time	Time for a page or prediction to load	Under 3 seconds
User acceptance	Share of staff rating the system useful	At least 70%

Table 3.6: Performance metrics

3.2.5 Test Cases

Table 3.7 lists the test cases for the machine learning models.

ID	Test	Expected result
MT-01	Audit each dataset for gaps, duplicates and impossible values	No missing cells or duplicates
MT-02	Check no feature is almost identical to the target	All correlations below 0.98
MT-03	Compare Model 1 with the naive baseline	Lower MAE than the baseline
MT-04	Confirm model ranking with time-series cross-validation	Same leading model across folds
MT-05	Measure Model 1 error by category and on stockout days	Weak areas identified
MT-06	Tune Model 2 threshold for recall	Recall of at least 0.80
MT-07	Keep a tuned model only if it beats the untuned one	Better model deployed
MT-08	Compare K-Means with hierarchical clustering	ARI above 0.80
MT-09	Change supplier weights by up to 30% over 500 runs	Spearman above 0.90
MT-10	Resample supplier orders 200 times	Most suppliers keep the same tier

Table 3.7: Machine learning test cases

Table 3.8 lists the functional and edge case tests for the system.

ID	Test scenario	Expected result
ST-01	Log in with a valid email and password	Dashboard opens for the user's role
ST-02	Log in with a wrong password	Clear error message, no access
ST-03	Try more than 10 logins in one minute	Further attempts are blocked
ST-04	Use the system after the token expires	Token refreshes or user is asked to log in
ST-05	Staff user tries to approve a reorder	Action refused for this role
ST-06	Open the demand forecast for a medicine	Forecast chart and model version shown
ST-07	Medicine's stock falls into the risk zone	Reorder recommendation appears with quantity and supplier
ST-08	Pharmacist approves a recommendation	Status changes to approved and an audit entry is saved
ST-09	Open the supplier page	Scores, tiers and low-confidence flags shown
ST-10	Open the expiry risk page	Batches at risk listed by expiry date
ST-11	Generate a stock report	PDF or CSV file downloads
ST-12	Medicine with no demand history	No crash. Forecast shown as unavailable
ST-13	Enter a negative stock quantity	Input rejected with a message
ST-14	Model file is missing or fails to load	System shows that predictions are unavailable

Table 3.8: System test cases

3.2.6 Test Results

The model tests have been run. Their results are reported here. The system test results are presented with screenshots in Chapter Five.

Model 1: Demand Forecasting

Model	MAE	RMSE	MAPE (%)	Mean CV MAE
Naive (lag-1)	39.36	66.01	67.20	-
Linear Regression	25.53	44.58	47.78	21.86
Random Forest	21.16	41.07	32.85	18.36
XGBoost	21.70	41.00	30.22	18.77
LightGBM	21.33	40.94	30.25	18.49
Random Forest (tuned, deployed)	21.30	40.71	33.66	-

Table 3.9: Model 1 results on the held-out test set

Random Forest had the lowest mean error in cross-validation, so it was tuned. The tuned version scored a test MAE of 21.30. This beat the untuned version retrained on the same data, which scored 21.67. The tuned model was therefore deployed. It cut the average error by about 46% compared with the naive baseline. Hypothesis H1 is supported. SARIMA was fitted to paracetamol alone as a reference and scored an MAE of 37.28.

Stress tests showed where the model is weaker. In the notebook test, antimalarials had the highest error (MAE 65.55 units). This reflects sharp malaria peaks in the rainy season. TB medicines had the lowest error, at 4.49 units. Error was also higher on stockout days, at 32.54 against 20.64. On those days, true demand was hidden because no stock was available.

Model 2: Stockout Risk

Model	Precision	Recall	F1	ROC-AUC	PR-AUC	Mean CV PR-AUC
Majority-class baseline	0.000	0.000	0.000	0.500	0.090	-
Logistic Regression	0.800	0.691	0.742	0.946	0.817	0.802
Random Forest	0.943	0.617	0.746	0.964	0.819	0.865
XGBoost	0.900	0.556	0.687	0.954	0.803	0.858
LightGBM	0.925	0.605	0.731	0.944	0.777	0.850
Random Forest (deployed, threshold 0.125)	0.570	0.802	0.667	0.963	0.810	-

Table 3.10: Model 2 results on the held-out test set

Random Forest had the highest mean PR-AUC in cross-validation. Tuning lowered its test PR-AUC from 0.810 to 0.783. The untuned version was therefore deployed. At the default threshold of 0.5, it caught only 61.7% of stockouts. The threshold was lowered to 0.125 to raise recall to 80.2%. Precision fell to 0.570 as a result. This means about four in ten alerts will be false. A pharmacist can quickly dismiss a false alert. A missed stockout, however, can harm a patient. A PR-AUC of 0.810 is far above the 0.090 baseline, so H2 is supported.

Model 3: Supplier Performance

Three tiers were chosen for the supplier clusters. Their silhouette score of 0.455 was close to the best value of 0.459. Three tiers also match how pharmacy managers think about suppliers. K-Means and hierarchical clustering agreed perfectly, with an ARI of 1.000. Changing the weights by up to 30% gave a mean Spearman correlation of 0.994. The lowest value over 500 runs was 0.972. Hypothesis H3 is supported. Six suppliers were rated Preferred, one Acceptable and five At-Risk. Three suppliers had fewer than 150 orders and were flagged as low confidence.

Model 4: Expiry Risk

A simple depletion rule was tested first as the benchmark. It agreed with 95.53% of the expiry labels, or 5,732 of 6,000 batches. Any learned model must beat this rule to be worth using. The batches were split by date into 1,254 training, 122 validation and 560 test records. Classifier training for Model 4 is still in progress. Its results will be reported in Chapter Five.

3.2.7 Bug and Issue Reporting

Every issue found during testing is logged with an ID, description, cause and action taken. Issues are ranked as high, medium or low severity. High severity issues must be fixed before deployment. Table 3.11 shows the issues recorded during model development.

ID	Issue	Severity	Action taken
I-01	Demand on stockout days is understated because stock ran out.	Medium	Rows kept but flagged. Reported as a limitation.
I-02	Reorder data has 1,789 repeated medicine and date checks.	Medium	Kept in the same data split to avoid leakage.
I-03	Default 0.5 threshold caught only 61.7% of stockouts.	High	Threshold lowered to 0.125 for 80.2% recall.

ID	Issue	Severity	Action taken
I-04	Tuned Model 2 did worse on the test set.	Medium	Untuned model deployed instead.
I-05	At the default threshold, Model 2 caught none of the 8 supplement stockouts.	High	Threshold lowered; 2 of 8 are now caught. Logged for retraining.
I-06	Three suppliers had very few orders.	Medium	Low-confidence flag shown with their scores.
I-07	SARIMA warned that no date frequency was set.	Low	Daily frequency inferred. Used as reference only.
I-08	Features could be built in a different order in Django.	High	Feature list stored in metadata. Library versions pinned.

Table 3.11: Issue log from model development

3.3 Ethical Considerations

Ethical rules protect participants, the hospital and the integrity of the research. This study follows the five areas set out in the project guide.

3.3.1 Originality and Referencing

This document is the researcher's own work. All ideas, data and figures from other sources are cited in APA style. The document has been checked with Turnitin to keep similarity below 15%.

3.3.2 Use of AI Tools

AI tools were used during this project, and their use is declared here. An AI assistant helped with drafting and editing parts of this document. AI coding assistants were also used during software development. All AI output was reviewed, tested and edited by the researcher. The researcher takes full responsibility for the final content and code.

3.3.3 Consent and Data Privacy

Written permission was sought from PGH management before any data collection. Each participant read a consent form before taking part. The form explained the purpose of the study and the right to withdraw. Signed consent forms are kept as proof and appear in Appendix A. The questionnaire did not ask for names or staff numbers. No patient data was

collected at any point. Research data is stored on a password-protected computer. The study follows the Cyber and Data Protection Act (Government of Zimbabwe, 2021).

3.3.4 Bias and Fairness in AI Models

A model can be accurate overall yet unfair to certain groups (Mehrabi et al., 2021). In this system, the groups are medicine categories and suppliers. Each model was therefore tested by category, not just overall. These tests revealed weaker forecasts for antimalarials. They also revealed that most supplement stockouts were missed. These weak spots are reported openly rather than hidden in an average. Suppliers with few orders carry a low-confidence flag. This stops them being judged unfairly on thin evidence. Supplier scores in this study are simulated. They are not real judgements of any named company.

3.3.5 Environmental and Social Responsibility

Large AI models can use a great deal of energy (Strubell et al., 2019). This project chose light models that train in minutes on free cloud notebooks. The final system runs on one ordinary server in the hospital. It also reduces paper by replacing printed stock reviews with on-screen dashboards. Socially, the system aims to reduce medicine shortages for patients. It keeps pharmacists in control rather than replacing their jobs.

3.4 Summary

This chapter explained how the study was carried out. Design Science Research guided the building of the system. An experimental approach tested the models against clear hypotheses. Questionnaires, interviews and observation gathered staff views and practices. Four datasets supported the development of four decision models. The test plan set out objectives, metrics and test cases. Model tests support all three hypotheses, though some weak spots remain. Ethical issues such as consent, bias and AI use were also addressed. The next chapter presents the analysis and design of the system.

CHAPTER FOUR: SYSTEM ANALYSIS AND DESIGN

4.0 Introduction

Chapter Three explained how the study was carried out and how the models were tested. This chapter shows how the system was planned before and during coding. It moves from what the system must do to how it is built. Functional models show the flow of data through the pharmacy. System models show how users and components interact over time. Data models show how records are stored. The chapter then presents the user interface and the security design. Every design choice is linked back to the objectives in Chapter One.

4.0.1 Requirements Summary

The requirements came from three sources. These were the study objectives and the literature in Chapter Two. The third was the planned consultation with PGH pharmacy staff. Table 4.1 lists the functional requirements.

ID	The system shall	Objective
FR1	Let users log in with an email and password and give each user a role.	6
FR2	Show stock, reorder points and expiry dates for every medicine.	1, 6
FR3	Record dispensing, taking stock from the earliest-expiring batch first.	6
FR4	Forecast daily demand for each medicine for 1 to 90 days ahead.	2
FR5	Estimate the chance of a stockout before the next delivery.	3
FR6	Score and tier suppliers from their delivery records.	4
FR7	Flag batches likely to expire before they are used.	5
FR8	Combine the four models into reorder recommendations with reasons.	6
FR9	Let a pharmacist approve or reject each recommendation.	6
FR10	Create a purchase order when a recommendation is approved.	6
FR11	Let users test each model with their own inputs.	7
FR12	Produce reports in CSV and PDF format.	6
FR13	Let the administrator manage users and import datasets.	6

Table 4.1: Functional requirements

Table 4.2 lists the non-functional requirements. These describe how well the system must work.

Quality	Requirement
Performance	A full decision run for all medicines finishes in under five seconds. Pages load in under three seconds.

Quality	Requirement
Usability	A pharmacist can run and approve a reorder without training beyond one short session.
Security	Only logged-in users can see data. Each action is checked against the user's role.
Reliability	The system runs on the hospital network and keeps working when the internet is down.
Auditability	Every decision, approval and dispense is recorded with the user and the time.
Transparency	Every model result shows the main reasons behind it.
Maintainability	Models are stored as versioned files and can be replaced without changing code.
Portability	Users need only a modern web browser.

Table 4.2: Non-functional requirements

4.1 Functional Modeling

Functional modeling shows what the system does with data. It uses data flow diagrams in the style of Gane and Sarson (1979). Rounded boxes are processes, square boxes are external entities and open boxes are data stores.

4.1.1 Context Diagram

Figure 4.1 shows the system as a single process. It also shows the six external entities that exchange data with it. Pharmacists send receipts, dispensing records and approvals. In return, they receive forecasts, risk alerts and reorder recommendations. The pharmacy manager sets policy and reviews dashboards and reports. Stores staff look up stock and receive stock and expiry lists. The administrator manages users and roles. Approved orders go to suppliers as purchase orders. Finally, trained model files come from Google Colab, where the models were built.

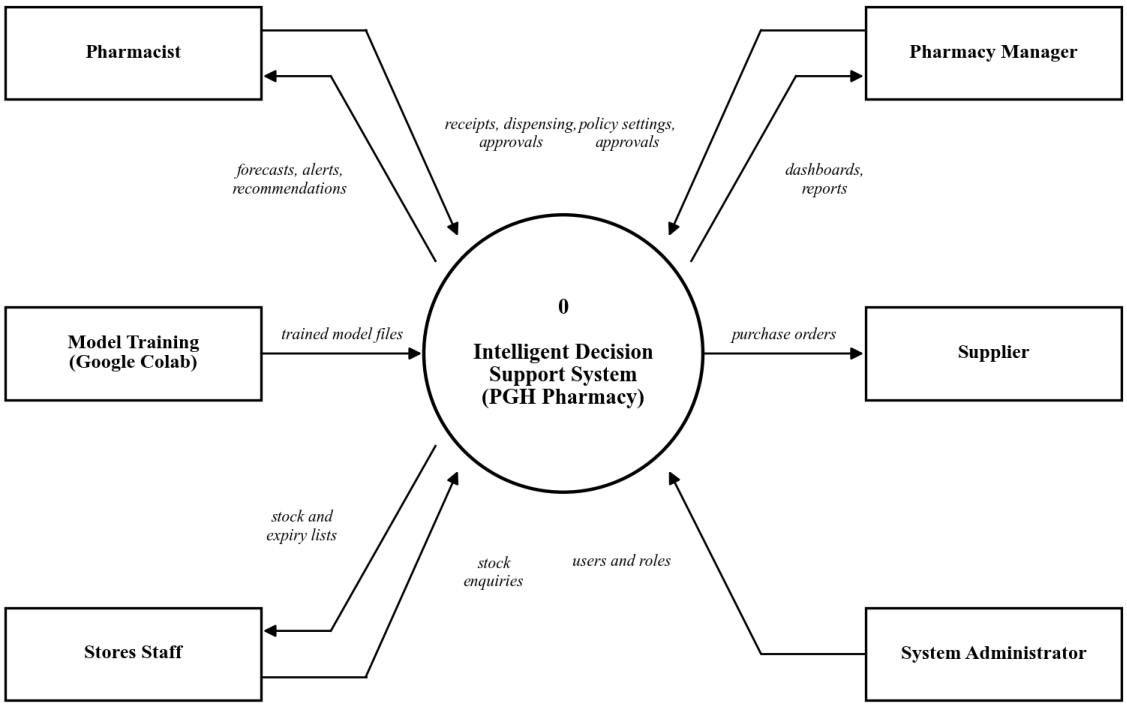


Figure 4.1: Context diagram of the proposed system

4.1.2 Data Flow Diagram (Level 1)

Figure 4.2 breaks the system into seven main processes and seven data stores. Process 1.0 checks each user against the users store (D1). Process 2.0 records receipts and dispensing and updates medicines and batches (D2). Process 3.0 reads the demand history (D3) and forecasts demand with Model 1. Process 4.0 reads delivery records (D4) and scores suppliers with Model 3. Process 5.0 uses stock, the forecast and the model files (D6) to assess risk. It applies Model 2 for stockouts and Model 4 for expiry. Process 6.0 combines all results into recommendations and saves them in D5. It writes each action to the audit log (D7). It also sends purchase orders to suppliers. Process 7.0 turns stored decisions into reports and dashboards for pharmacy staff.

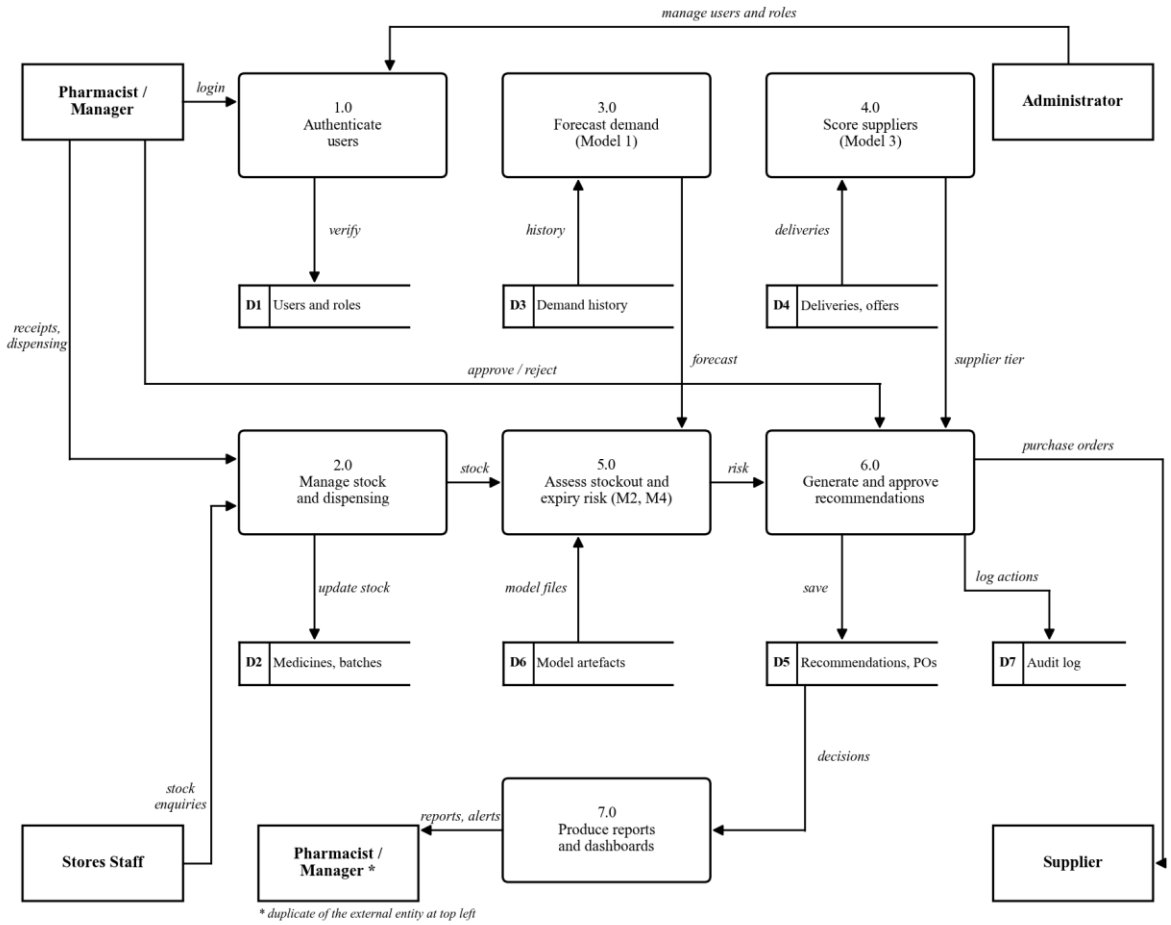


Figure 4.2: Level 1 data flow diagram

4.2 System Modeling

System modeling describes the behaviour of the system. The Unified Modeling Language (UML) was used for these diagrams (Fowler, 2004).

4.2.1 Use Case Diagram

Figure 4.3 shows the four user roles and the services each one uses. The roles form a chain. Each role can do everything the role above it can do, plus more. Stores staff can log in and view stock and expiry lists. Pharmacists add forecasting, the Model Lab, the decision engine and approvals. Pharmacy managers can also update supplier details. The administrator can also manage users and import datasets.

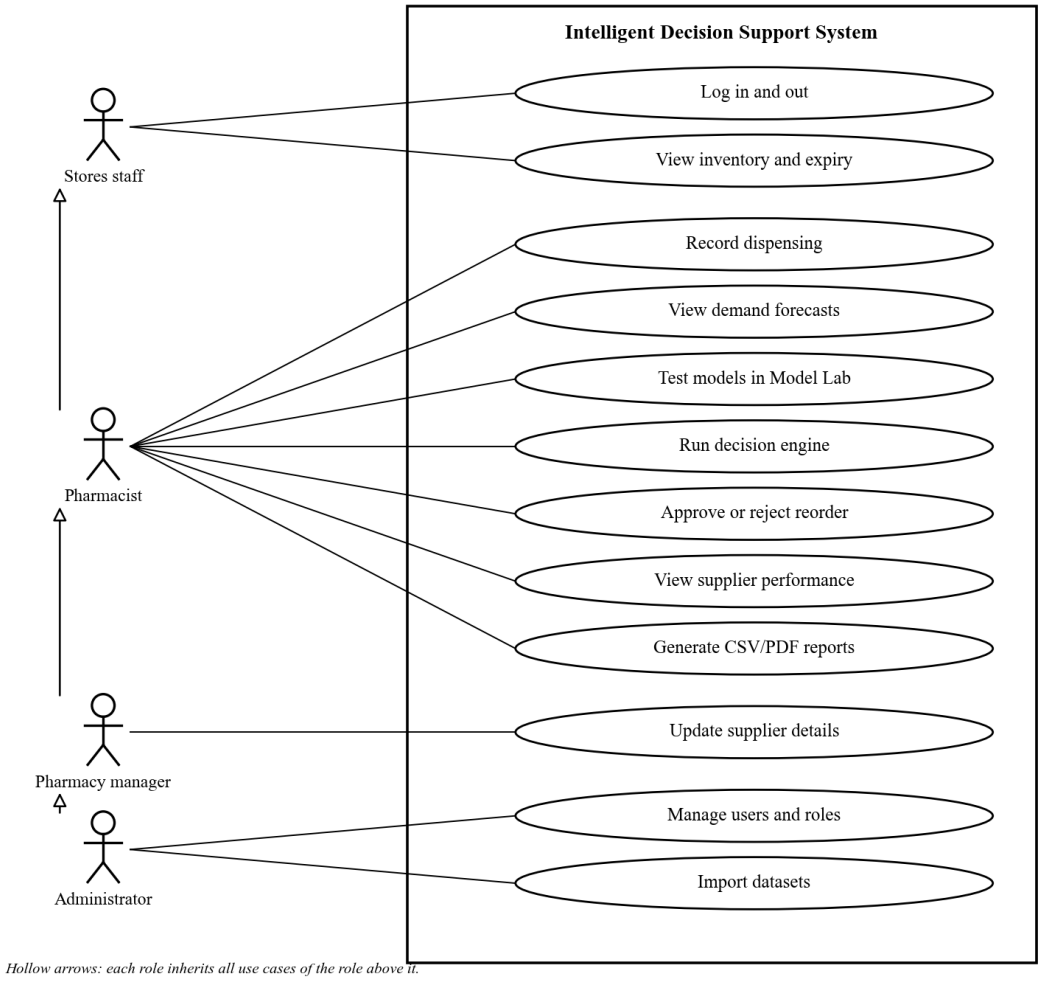


Figure 4.3: Use case diagram

Table 4.3 describes the most important use case in detail.

Item	Description
Use case	Run decision engine and approve a reorder
Actor	Pharmacist (also manager and administrator)
Precondition	The user is logged in. Stock, demand and supplier data are loaded.
Main flow	1. The user clicks Run decision engine. 2. The system checks every medicine with the four models. 3. The system lists orders, expedite advice and medicines needing no action. 4. The user opens a recommendation and reads its reasons. 5. The user approves it, with or without changing the quantity. 6. The system creates a purchase order and records the approval.
Alternative flow	At step 5 the user rejects the recommendation. The system marks it rejected and records who did so.
Postcondition	A purchase order exists, or the recommendation is rejected. The audit log holds the decision.

Table 4.3: Use case description for running the decision engine

4.2.2 Sequence Diagrams

Figure 4.4 shows the login sequence. The React interface sends the email and password to the Django authentication API. The API first checks that the user has not made too many attempts. It then compares the password with the stored hash. If they match, it returns a short-lived access token and a refresh token. The interface uses the access token to fetch the user's name and role. It then shows only the menus that the role allows.

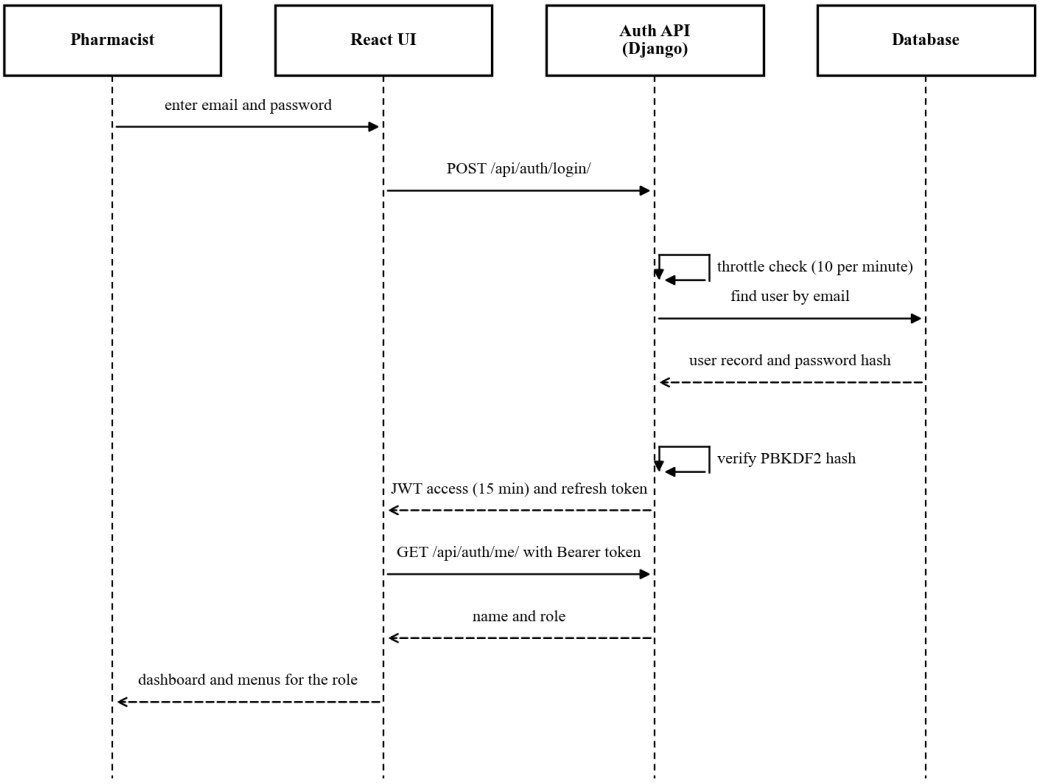


Figure 4.4: Sequence diagram for user login

Figure 4.5 shows the core sequence of the system. The pharmacist starts a decision run from the Reorder Decisions page. The decision engine reads stock, demand and delivery data from the database. It then calls the four models in turn. The results are saved as recommendations, and an audit event is written. When the pharmacist approves one, the API creates a purchase order. The order's due date is set from the supplier's lead time.

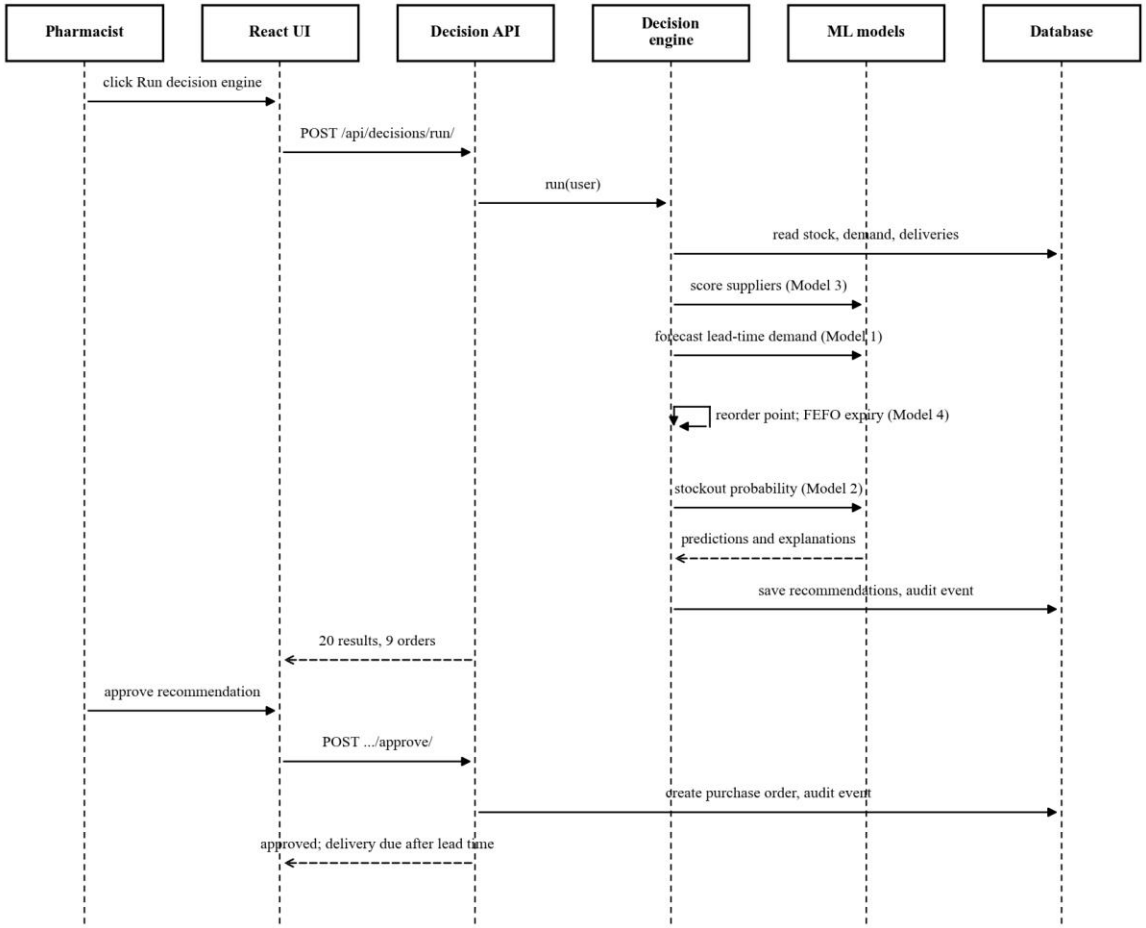


Figure 4.5: Sequence diagram for running the decision engine and approving a reorder

4.2.3 State Diagrams

Figure 4.6 shows the life of a recommendation and of a purchase order. A recommendation starts as pending when the engine creates it. It then becomes approved, rejected or superseded. It is superseded when a newer engine run replaces it. This prevents two open recommendations for the same medicine. An approved recommendation creates a purchase order. The order starts as pending and becomes partial, delivered or cancelled.

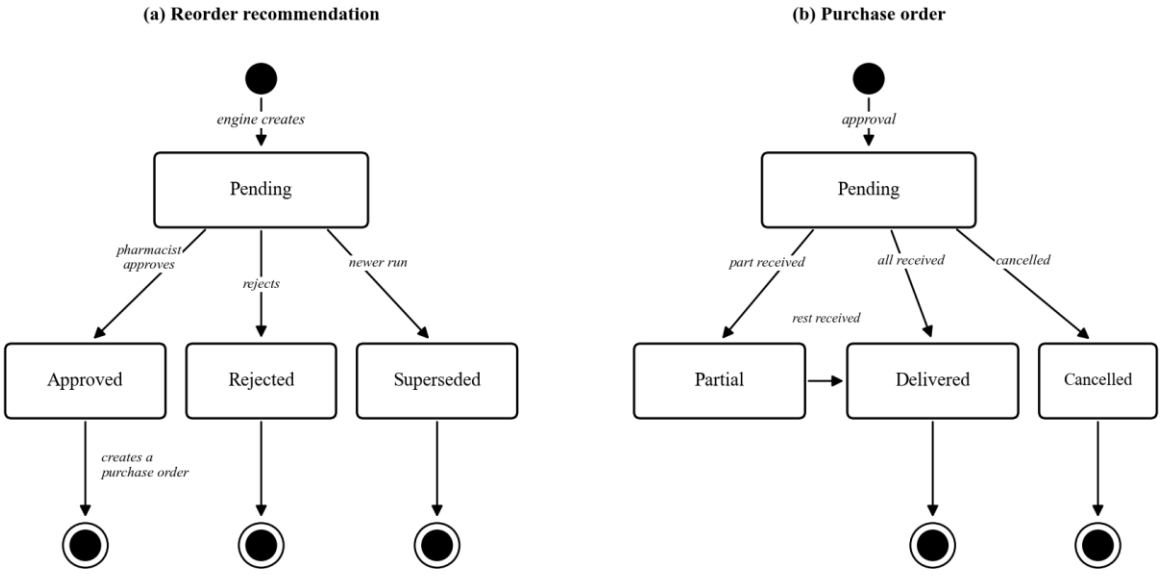


Figure 4.6: State diagrams for (a) a recommendation and (b) a purchase order

4.2.4 Activity Diagram

Figure 4.7 shows the reorder decision logic for one medicine. The engine first gathers data and chooses a supplier. It forecasts demand and sets the reorder point. It then removes stock that will expire unused and estimates the stockout risk. If usable stock is at or below the reorder point, an order is proposed. Stock may sit above that point while the risk is still high. In that case, the engine checks for open orders. With no open order, it proposes a new one. With an open order, it advises the pharmacist to expedite that order instead. Ordering again would not help, because a new order would arrive just as late.

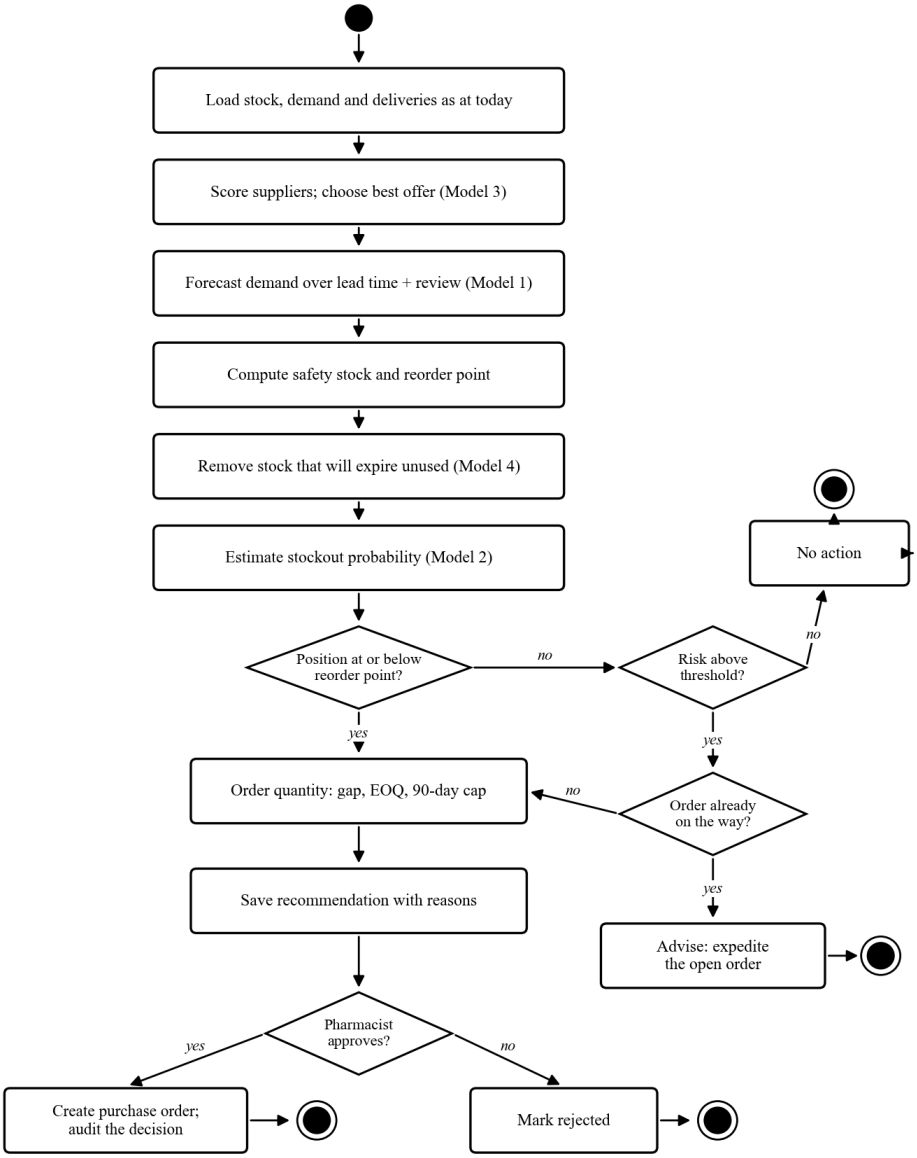


Figure 4.7: Activity diagram for the reorder decision

4.2.5 Architectural Design

Figure 4.8 shows the architecture of the system. It follows a three-tier design with a separate service layer (Sommerville, 2016). The presentation tier is a React web application. It runs in any modern browser on the pharmacy's computers. The application tier is a Django REST API. It checks every request for a valid token and the right role. The service layer holds the

decision engine, the Model Lab and the reporting logic. The data tier is a PostgreSQL database. SQLite is used during development.

The machine learning models are trained offline in Google Colab. Only the finished model files are copied to the server. The server loads them read-only and never accepts model files from users. This keeps heavy training work away from the hospital server. It also means the whole system can run on the hospital network without internet access. This matters in Zimbabwe, where internet outages and load shedding are common.

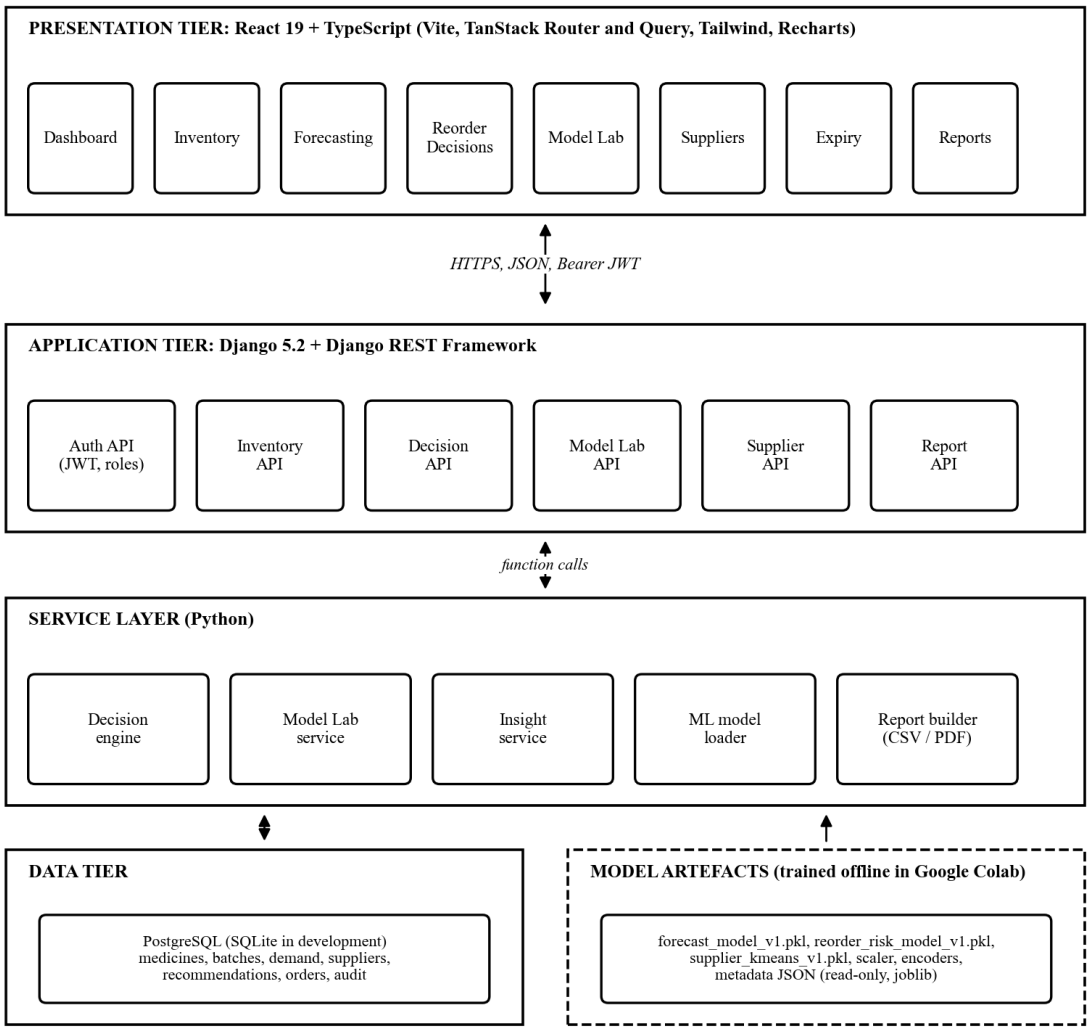


Figure 4.8: System architecture

4.2.6 Component and Module Design

Figure 4.9 shows the main software components and how they depend on each other. Table 4.4 describes the role of each module.

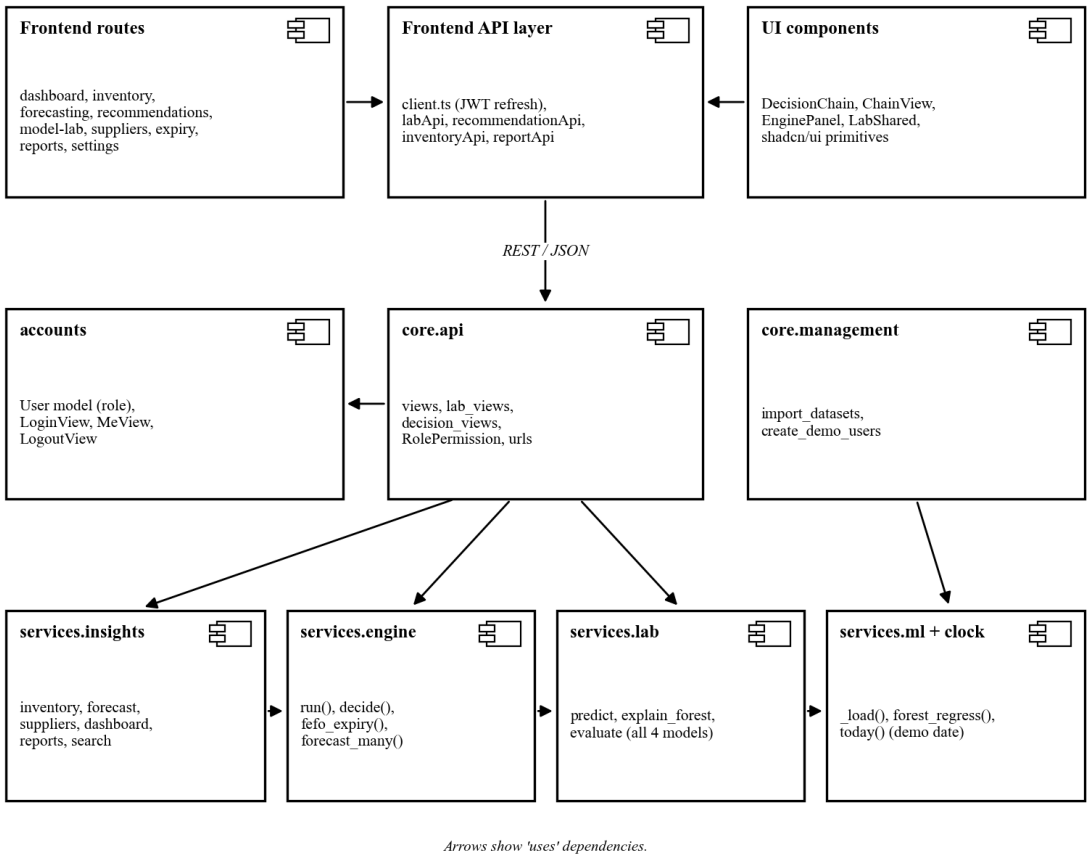


Figure 4.9: Component diagram

Module	Responsibility
Frontend routes	One page per task. Pages cover the dashboard, inventory, forecasts, reorders, Model Lab, suppliers, expiry, reports and settings.
Frontend API layer	Sends all requests to the backend. Adds the access token and renews it when it expires.
accounts	Stores users and roles. Handles login, logout and the current user's profile.
core.api	Exposes the REST endpoints. Checks the role permission for every request.
services.engine	Runs the four models together and produces reorder decisions.
services.lab	Runs any single model on custom inputs, explains the result and re-tests the model.
services.insights	Builds the figures shown on every page and produces CSV and PDF

Module	Responsibility
	reports.
services.ml and clock	Loads model files once and keeps them in memory. Supplies today's date, or the demo date.
core.management	Command-line tools to import the datasets and create demo users.

Table 4.4: Module responsibilities

4.3 Data Modeling

4.3.1 Entity Relationship Diagram

Figure 4.10 shows the entities stored by the system and how they relate. MEDICINE sits at the centre. Each medicine has many daily demand records, batches, supplier offers and recommendations. Each supplier has many offers and many delivery records. A supplier offer links one medicine to one supplier with a price and lead time. A recommendation points to the chosen offer. An approved recommendation leads to at most one purchase order. Users record stock movements, decide on recommendations and create purchase orders. Every important action is written to the audit log.

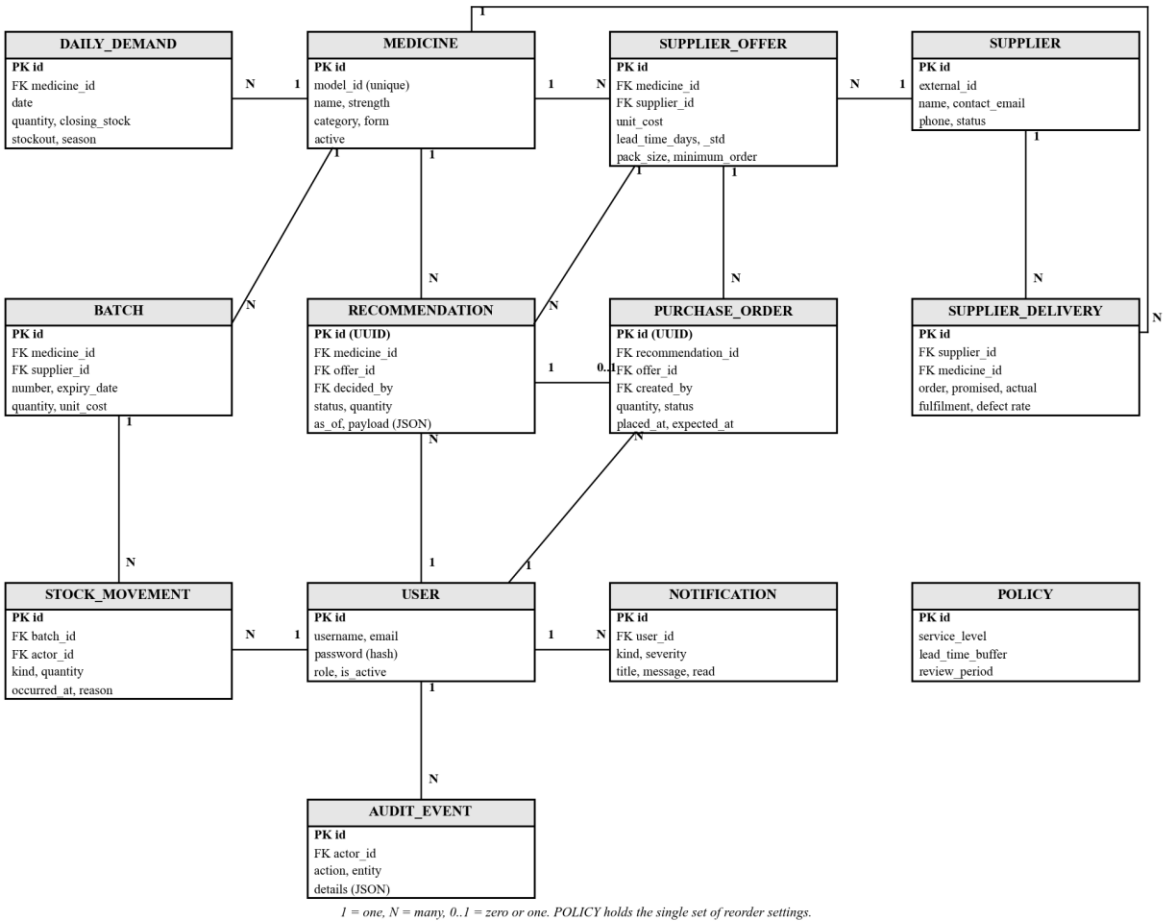


Figure 4.10: Entity relationship diagram

4.3.2 Class Diagram

Figure 4.11 shows the main classes. The upper classes are data classes that map to database tables. The lower classes are service classes that hold the decision logic. The DecisionEngine reads batches, uses the ModelLab's explanation method and creates recommendations. The InsightService calls the engine without saving, to build the figures on each page. It caches the results and recomputes them only when stock or orders change.

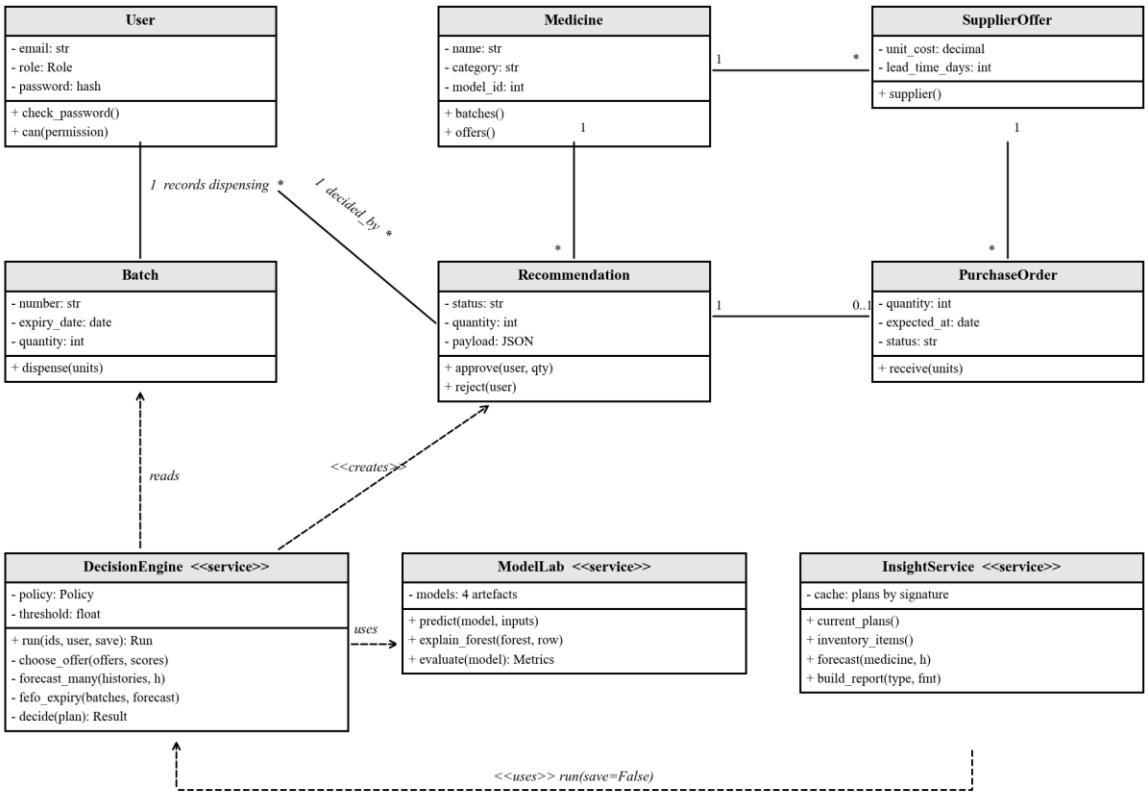


Figure 4.11: Class diagram

4.3.3 Database Design

The database has 18 application tables. Tables 4.5 to 4.9 show the five most important ones. PK marks a primary key and FK marks a foreign key.

Field	Type	Key or rule
id	Integer	PK
model_id	Integer	Unique; links the medicine to the model's encoding
name	Text (200)	Required
strength	Text (80)	Optional
category	Text (100)	Required
form	Text (80)	Optional
active	Boolean	Default true

Table 4.5: MEDICINE table

Field	Type	Key or rule
id	Integer	PK
medicine_id	Integer	FK to MEDICINE

Field	Type	Key or rule
supplier_id	Integer	FK to SUPPLIER; optional
number	Text (100)	Unique for each medicine
received_at	Date	Required
expiry_date	Date	Must not be before received_at
unit_cost	Decimal (12,4)	Zero or more
quantity	Integer	Zero or more
quarantined	Boolean	Excluded from stock when true

Table 4.6: BATCH table

Field	Type	Key or rule
id	Integer	PK
medicine_id	Integer	FK to MEDICINE
date	Date	Unique with medicine_id
quantity	Integer	Units dispensed that day
closing_stock	Integer	Stock at the end of the day
stockout	Boolean	True when stock ran out that day
holiday	Boolean	Public holiday flag
season	Text (50)	Rainy, dry/cold or transition

Table 4.7: DAILY_DEMAND table

Field	Type	Key or rule
id	UUID	PK
medicine_id	Integer	FK to MEDICINE
offer_id	Integer	FK to SUPPLIER_OFFER
status	Text (15)	Pending, approved, rejected or superseded
quantity	Integer	Suggested units
unit_cost	Decimal (12,4)	Price at the time of the decision
as_of	Date	Date of the decision
payload	JSON	Full reasoning from all four models
decided_by	Integer	FK to USER; set on approval or rejection
decided_at	Date and time	Set on approval or rejection

Table 4.8: RECOMMENDATION table

Field	Type	Key or rule
id	UUID	PK
recommendation_id	UUID	FK to RECOMMENDATION; one order at most
offer_id	Integer	FK to SUPPLIER_OFFER
quantity	Integer	More than zero
received_units	Integer	Cannot exceed quantity
placed_at	Date	Date of approval
expected_at	Date	placed_at plus supplier lead time

Field	Type	Key or rule
status	Text (20)	Pending, partial, delivered or cancelled
created_by	Integer	FK to USER

Table 4.9: PURCHASE_ORDER table

The tables were normalised to the third normal form (Codd, 1970; Elmasri & Navathe, 2016). Each field holds a single value, which satisfies the first normal form. Every non-key field depends on the whole primary key, which satisfies the second normal form. No non-key field depends on another non-key field, which satisfies the third normal form. For example, supplier names live only in the SUPPLIER table. Offers refer to suppliers by key instead of repeating their details.

Two fields break this rule on purpose. First, the recommendation keeps its full reasoning in a JSON payload. This freezes the evidence exactly as it was when the decision was made. An auditor can later see why an order was placed, even after stock has changed. Second, purchase orders copy the unit cost at the time of approval. This keeps the order value correct even if the supplier's price changes later.

4.4 Interface Design

4.4.1 Design Principles

The interface was designed for busy pharmacy staff. It follows recognised usability principles (Nielsen, 1993). Pages use plain words, consistent layouts and clear colour-coded status labels. Every model result shows the reasons behind it. A decision chain at the top of each page shows where the page sits. The chain runs from raw data to analytics, forecast, decision and action. Figures 4.12 to 4.16 show the main screens of the working system. They are shown in greyscale to follow the project guide.

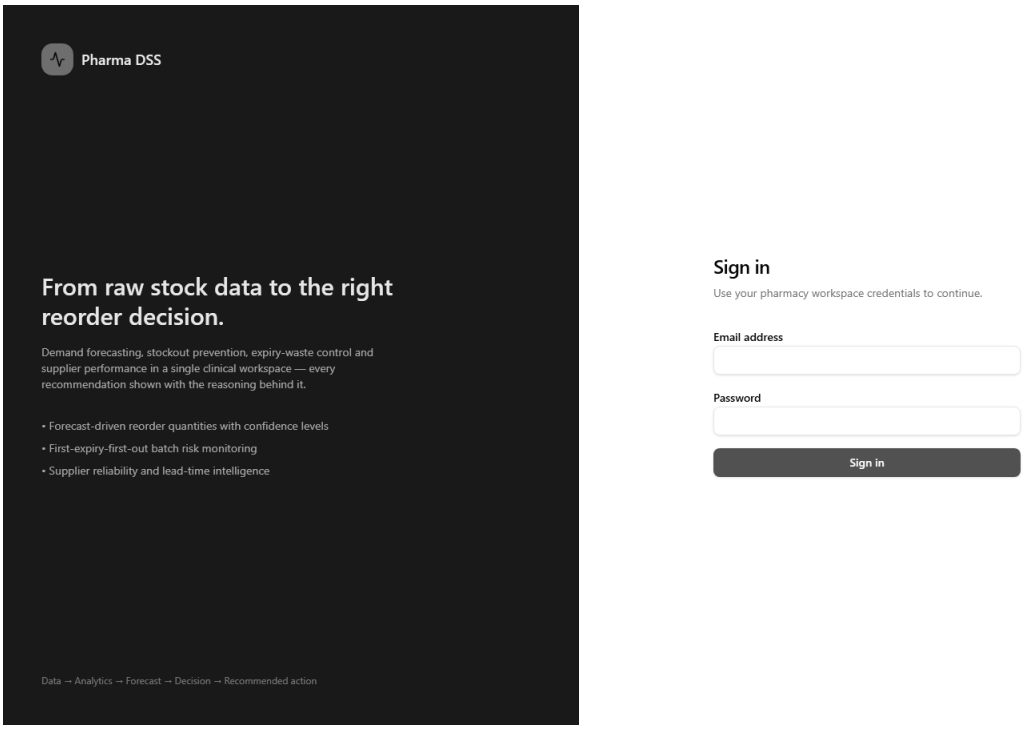


Figure 4.12: Login screen

Figure 4.12 shows the login screen. The left panel explains what the system does. The right panel accepts the user's email and password.

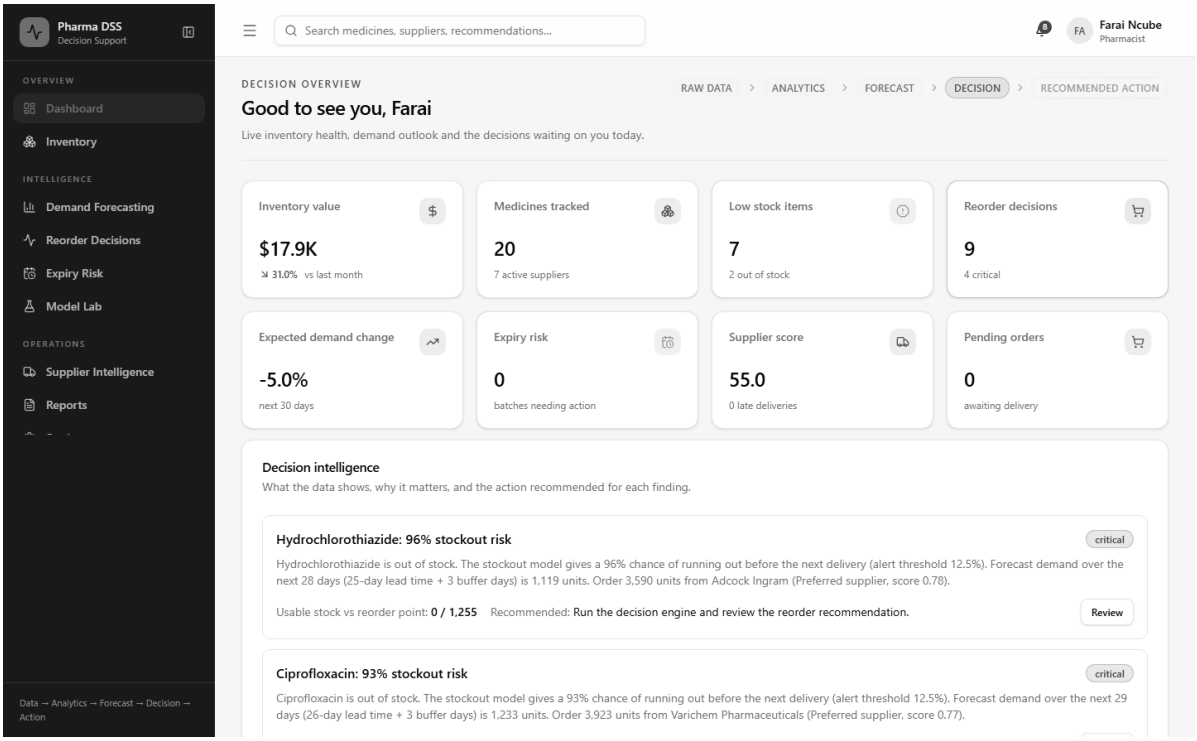


Figure 4.13: Dashboard showing key indicators and decision intelligence

Figure 4.13 shows the dashboard as at 31 December 2024. It shows stock value, low stock items, pending reorder decisions and expected demand change. Below these, the decision intelligence panel lists the most urgent findings in plain language. For example, it warns that Hydrochlorothiazide is out of stock with a 96% stockout risk.

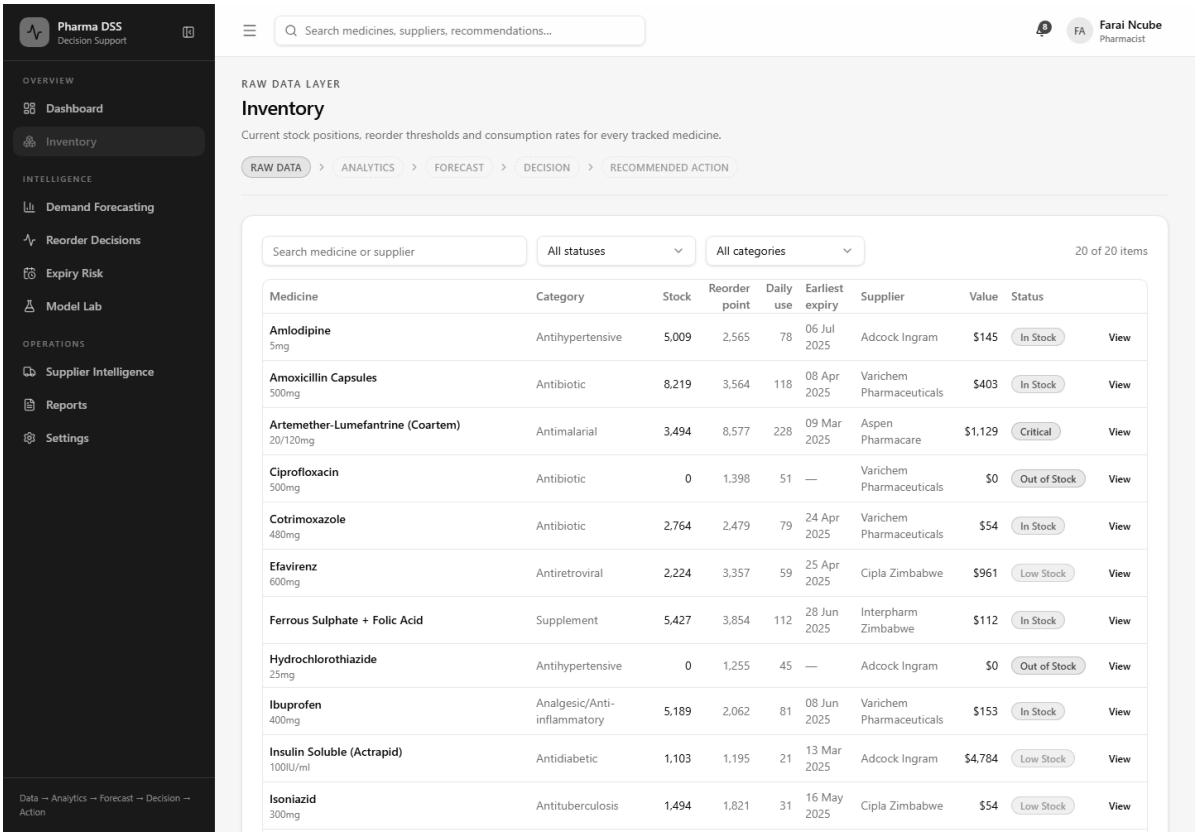


Figure 4.14: Inventory screen with model-based reorder points

Figure 4.14 shows the inventory screen. Each medicine has its own reorder point and daily use, both computed by the system. Status labels show which medicines are critical, low or out of stock.

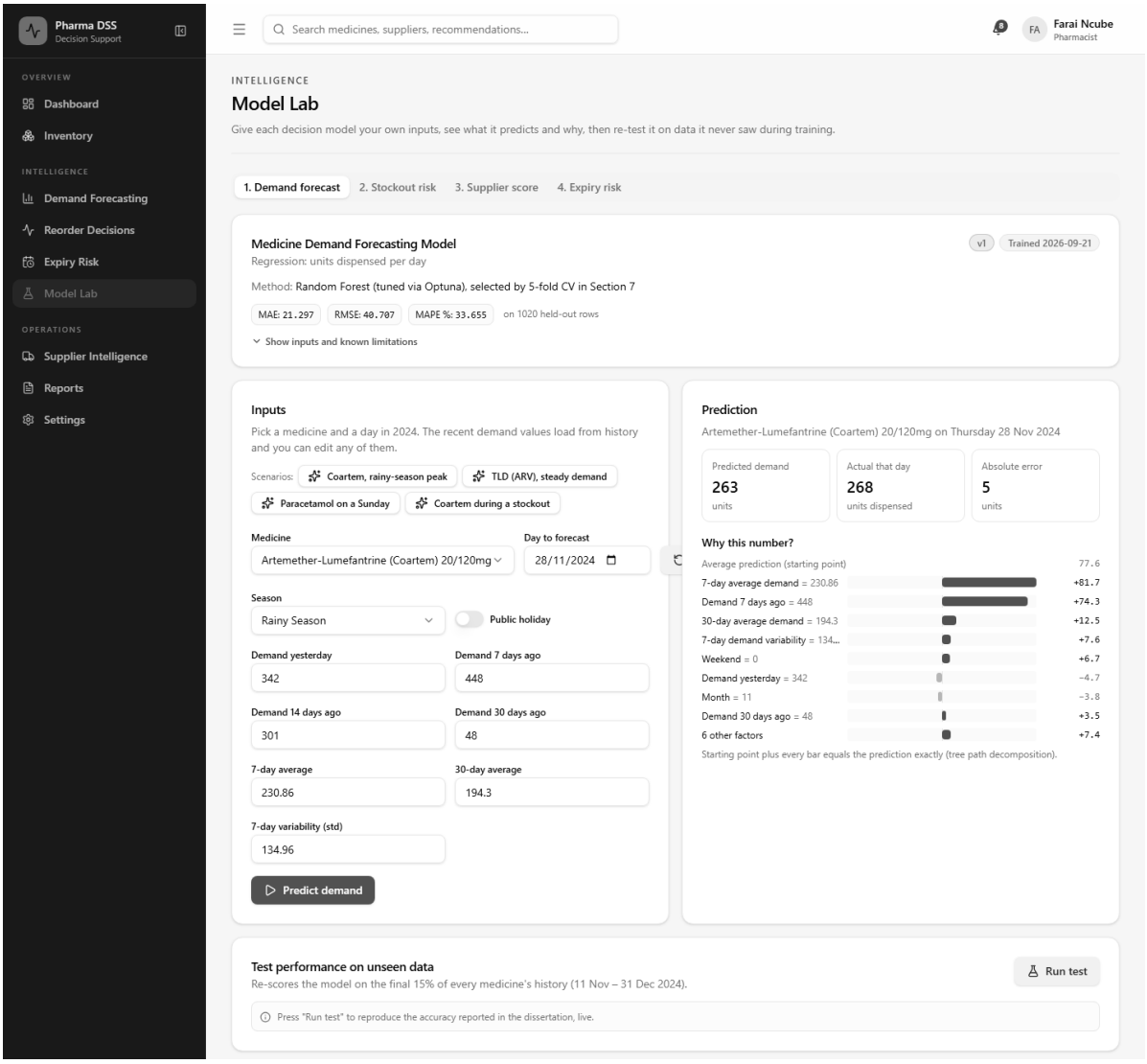


Figure 4.15: Model Lab screen for the demand forecasting model

Figure 4.15 shows the Model Lab. Here a user can test any of the four models with their own inputs. In this example, the demand model predicts 263 units of Coartem for 28 November 2024. The actual figure that day was 268. The bars on the right explain how each input moved the prediction up or down. A test button re-scores the model on data it never saw during training.

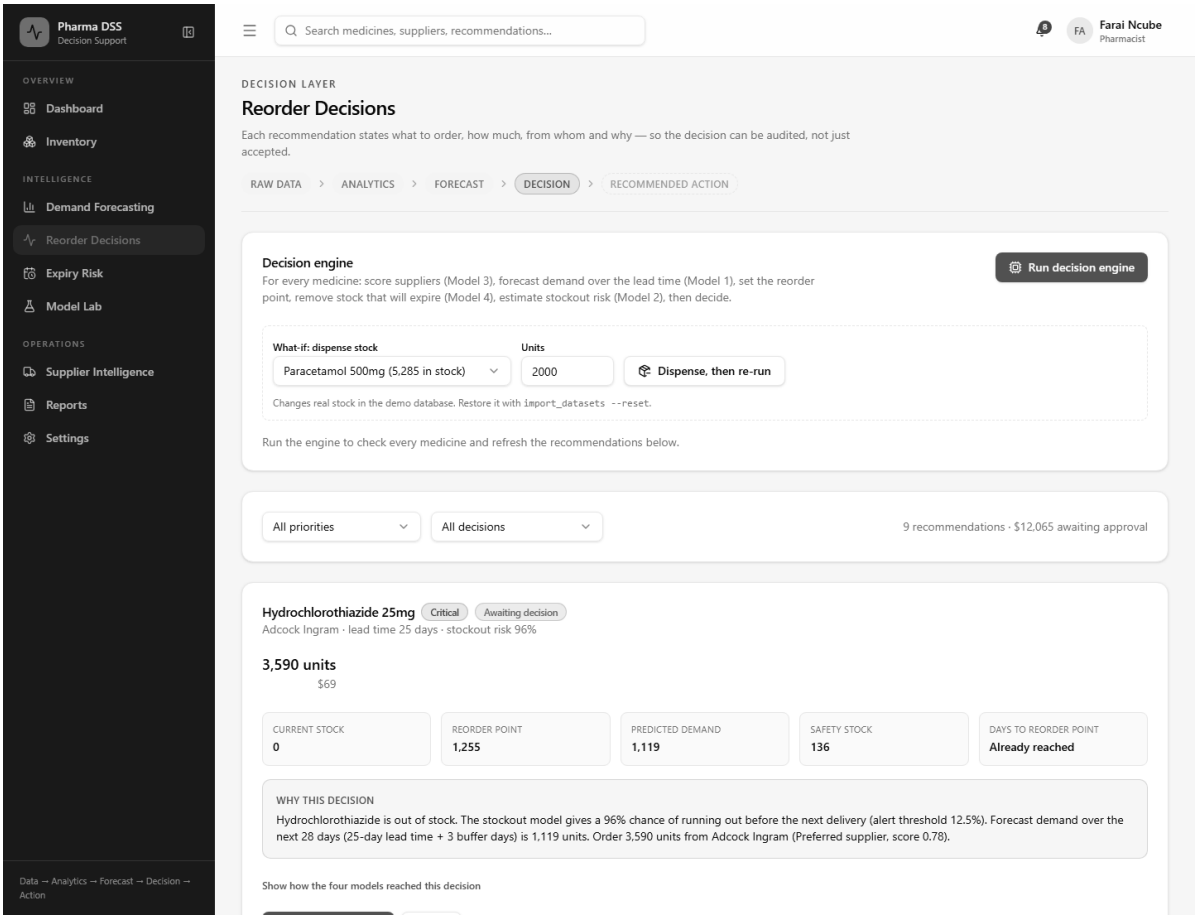


Figure 4.16: Reorder Decisions screen with the decision engine

Figure 4.16 shows the Reorder Decisions screen. The pharmacist can run the decision engine and try a what-if dispense. Each recommendation shows the quantity, cost, reorder point and the reason in plain words. A link expands the full chain of reasoning from all four models.

4.4.2 Pseudocode for Core Components

The following pseudocode describes the core logic without showing raw source code.

```
FUNCTION Login(email, password)
    IF more than 10 attempts in the last minute THEN
        RETURN "Too many attempts. Try again shortly."
    user <- find user WHERE email matches
    IF user not found THEN RETURN error "Email is not valid."
    IF hash(password) does not match user.password_hash THEN
        RETURN error "Wrong password."
    IF user is disabled THEN RETURN error "Account is disabled."
    access  <- signed token valid for 15 minutes
    refresh <- signed token valid for 1 day
    RETURN access, refresh, user.name, user.role
```

```
END FUNCTION
```

Pseudocode 4.1: User login

```
FUNCTION Decide(medicine, today)
    offers   <- supplier offers for medicine
    supplier <- offer with best tier, then highest score, then shortest lead time   // Model 3
    L        <- supplier.lead_time + buffer days
    forecast <- predict daily demand for next (L + review period) days           // Model 1
    mean, sd <- average and spread of recent demand, ignoring stockout days
    safety   <- z * sd * SQRT(L)
    reorder_point <- SUM(forecast over L days) + safety
    expiring <- units that will expire before use (FEFO check)                   // Model 4
    usable   <- stock on hand - expiring + units already on order
    risk     <- probability of stockout before next delivery                     // Model 2
    IF usable <= reorder_point OR (risk >= threshold AND nothing on order) THEN
        quantity <- MAX(order-up-to gap, MIN(EOQ, 90 days of demand))
        RETURN "Order", quantity, supplier, reasons
    ELSE IF risk >= threshold AND an order is already on the way THEN
        RETURN "Expedite the open order", reasons
    ELSE
        RETURN "No action", reasons
END FUNCTION
```

Pseudocode 4.2: Decision engine for one medicine

```
FUNCTION ExpiringUnits(batches, forecast)
    sort batches by expiry date, earliest first
    day <- 0 ; expiring <- 0
    FOR EACH batch IN batches
        demand_before_expiry <- forecast demand from day until batch expires
        used    <- MIN(batch.quantity, demand_before_expiry)
        expiring <- expiring + (batch.quantity - used)
        day     <- day + days needed to use 'used' units
    END FOR
    RETURN expiring
END FUNCTION
```

Pseudocode 4.3: First-expiry-first-out expiry check

```
FUNCTION PredictAndExplain(model, inputs)
    VALIDATE every input is present, numeric and in range
    row <- build feature row exactly as in the training notebook
    prediction <- model.predict(row)
    baseline <- 0 ; contributions <- zeros
    FOR EACH tree IN model
        follow the path of row from root to leaf
        add each change in node value to the feature used at that split
    END FOR
    average baseline and contributions over all trees   // baseline + SUM = prediction
    RETURN prediction, baseline, top contributions
END FUNCTION
```

Pseudocode 4.4: Model Lab prediction with explanation

The explanation method in Pseudocode 4.4 follows the tree path decomposition of Saabas (2014). It splits each prediction exactly into a starting value plus one contribution per input. This lets a pharmacist see why the model gave a particular answer.

```
FUNCTION Approve(recommendation, user, quantity)
    IF user role lacks "recommendations.approve" THEN RETURN "Access denied"
    LOCK recommendation
    IF recommendation.status is not "pending" THEN RETURN "Already decided"
    IF quantity < 1 THEN RETURN error
    order <- new purchase order(quantity, unit cost now,
                                placed today, due today + supplier lead time)
    recommendation.status <- "approved" ; record user and time
    write audit event "approve_recommendation"
    RETURN updated recommendation
END FUNCTION
```

Pseudocode 4.5: Approve a recommendation

4.5 Security Design

The system holds stock, supplier and user data for a public hospital. Its security design follows the threats listed in Chapter Two. It also respects the Cyber and Data Protection Act (Government of Zimbabwe, 2021).

4.5.1 Password Hashing

Passwords are never stored as plain text. Django stores them as salted PBKDF2-SHA256 hashes. Each password is run through the hash function many thousands of times. This makes stolen hashes very slow to crack. Passwords must also pass Django's built-in strength checks.

4.5.2 Data Encryption

In production, all traffic between the browser and the server uses HTTPS. The server also sends HSTS headers, so browsers refuse plain HTTP. Session and CSRF cookies are marked secure. Secret keys and database passwords are kept in an environment file outside the code. For data at rest, the server disk should be encrypted, for example with BitLocker.

4.5.3 Authentication and Authorisation Logic

Users log in with an email and password. The server then issues JSON Web Tokens (Jones et al., 2015). The access token lasts 15 minutes and the refresh token lasts one day. Refresh

tokens are rotated, and old ones are blacklisted. A stolen token therefore has a short useful life. Login attempts are limited to ten per minute to slow down password guessing.

Authorisation is checked on the server for every request. Each endpoint names the permission it needs. The server compares this with the permissions of the user's role. The frontend also hides menus a role cannot use. However, hiding a menu is only a convenience. The server check is what actually protects the data.

4.5.4 User Access Roles

Table 4.10 shows the permissions of each role.

Permission	Staff	Pharmacist	Manager	Admin
View inventory and expiry	Yes	Yes	Yes	Yes
Record dispensing	No	Yes	Yes	Yes
View forecasts and use the Model Lab	No	Yes	Yes	Yes
Run the decision engine	No	Yes	Yes	Yes
Approve or reject reorders	No	Yes	Yes	Yes
View supplier performance	No	Yes	Yes	Yes
Edit supplier details	No	No	Yes	Yes
Generate reports	No	Yes	Yes	Yes
Manage users and settings	No	No	No	Yes

Table 4.10: Role permission matrix

4.5.5 Secure Storage and Integrity

Several further controls protect the stored data. Model files are loaded read-only from a fixed folder. They are never uploaded through the web. Stock changes run inside database transactions, so a failed update leaves no half-written records. Each stock movement carries a unique key, so a repeated request is rejected. The audit log records who did what and when. Imported datasets are stored with a SHA-256 fingerprint to detect changes. Finally, the database should be backed up daily. A UPS should protect the server from power cuts.

4.6 Hardware Design

This project is a software system, so no custom hardware was designed. No circuits, sensors or printed circuit boards are needed. The system runs on one ordinary server on the hospital network. Staff use the pharmacy's existing computers and a web browser. The hardware needed was listed in the technical feasibility study in Chapter Two.

4.7 Summary

This chapter presented the analysis and design of the system. It began with 13 functional and eight non-functional requirements. The context and data flow diagrams showed how data moves through the pharmacy. UML diagrams showed the users, the main sequences, the states and the decision logic. The architecture separates the interface, the API, the services and the data. The ERD, class diagram and tables showed how records are stored in third normal form. The interface design showed the main screens and the pseudocode for the core logic. The security design covered hashing, encryption, tokens and role-based access. The next chapter presents the coding and testing of the system.

CHAPTER FIVE: CODING AND TESTING

5.0 Introduction

Chapter Four presented the design of the system. This chapter shows how that design was built and tested. Section 5.1 describes the coding of the models, the backend and the frontend. In line with the project guide, pseudocode is used instead of raw source code. Section 5.2 presents the testing evidence. It covers automated tests, model tests, system tests, performance tests and the defects found. Every result reported here was produced by running the finished system. Screenshots were captured during the tests and annotated with numbered markers.

5.1 Coding

5.1.1 Development Environment

The system was built with the tools listed in Chapter One. The models were trained in Google Colab using Python and scikit-learn. The backend was written in Python with Django 5.2 and Django REST Framework. The frontend was written in TypeScript with React 19. Development took place on a Windows 11 laptop using Visual Studio Code. Git was used for version control, with each completed stage saved as a commit. SQLite was used during development, and the same code runs on PostgreSQL.

5.1.2 Building the Machine Learning Models

Each model was built in its own Colab notebook. Every notebook followed the same ten stages. These were setup, data audit, exploration, charts and feature building. They were followed by training, validation, stress testing, tuning and export. The data was always split by date, never at random. This stopped future data from leaking into training. Pseudocode 5.1 summarises the training of the demand model.

```
LOAD daily demand records for 20 medicines (2024)
CHECK for missing values, duplicates and negative quantities
FOR EACH medicine
    ADD calendar features: weekday, month, weekend, public holiday, season
    ADD lag features: demand 1, 7, 14 and 30 days earlier (shifted, no leakage)
    ADD 7-day and 30-day averages and the 7-day spread
END FOR
```

```
SPLIT each medicine's history by date: first 70% train, next 15% validate, last 15% test
TRAIN naive, Linear Regression, Random Forest, XGBoost and LightGBM
RANK models by mean error in 5-fold time-series cross-validation
TUNE the best model with Optuna (40 trials)
KEEP the tuned model only if it beats the untuned one on the test set
SAVE model, encoders and metadata (features, metrics, limitations) as version v1
```

Pseudocode 5.1: Training the demand forecasting model

The stockout model followed the same pattern. Stockouts made up only about 10% of the records. Class weighting was therefore used during training. The alert threshold was then lowered to 0.125. This catches at least 80% of stockouts. The supplier model is unsupervised. It combines a weighted score with K-Means clustering into three tiers.

5.1.3 Training and Testing the Expiry Model

Chapter Three reported that the expiry classifier was still in training. It was completed during this phase. Only batches whose outcome was already known on each split date were used for training. This left 1,254 training batches, 122 validation batches and 560 test batches. Logistic Regression and Random Forest were compared on the validation set. Random Forest scored higher and was tuned to catch at least 85% of at-risk batches. Table 5.1 compares it with the simple depletion rule on the 560 unseen test batches.

Method	Precision	Recall	F1	At-risk batches missed
Random Forest classifier	96.5%	92.7%	0.946	15 of 206
Depletion rule (stock exceeds use before expiry)	95.6%	94.7%	0.951	11 of 206

Table 5.1: Expiry classifier compared with the depletion rule on 560 test batches

The trained classifier did not beat the rule. It missed four more at-risk batches. The rule is also simpler to explain to pharmacists. The system therefore keeps the depletion rule as its expiry model. This is an honest result rather than a failure. A learned model should replace a rule only when it clearly performs better.

5.1.4 Backend Implementation

The backend was built as two Django apps. The accounts app holds users and roles. The core app holds medicines, stock, suppliers, recommendations and orders. Business logic lives in

four service modules rather than in the web views. This keeps the API layer thin and easy to test.

A management command loads the project datasets into the database. It turns the four CSV files into a working pharmacy as at 31 December 2024. Pseudocode 5.2 shows the main steps.

```
READ demand, supplier and batch CSV files
VALIDATE required columns, empty cells and negative quantities
CREATE 20 medicines and 7,320 daily demand records (with closing stock and stockout flags)
CREATE 12 suppliers and their delivery records
    SKIP any delivery that arrives after the snapshot date (no future data)
CREATE one offer per medicine and supplier: average price and lead time from past orders
FOR EACH medicine
    total <- closing stock on the snapshot date
    SPLIT total across three real batches (older batches hold less stock)
END FOR
RECORD a fingerprint of each file and an audit event
```

Pseudocode 5.2: Importing the datasets as a demonstration pharmacy

Testing showed that recent demand was understated after stockouts. On a stockout day nothing can be dispensed, so the record shows zero. This hides the true demand. The engine was therefore changed to ignore stockout days when it measures demand. Pseudocode 5.3 shows this step.

```
INPUT last 60 days of (quantity, stockout flag)
clean <- quantities in the last 30 days where stockout flag is false
IF fewer than 10 clean days THEN
    clean <- quantities in the last 60 days where stockout flag is false
IF fewer than 5 clean days THEN
    clean <- all quantities in the last 30 days
RETURN mean(clean), standard deviation(clean), number of stockout days
```

Pseudocode 5.3: Measuring demand without stockout days

For Ibuprofen, this changed the recent daily average from 27 to 81 units. The medicine had been out of stock for 20 days in December. The corrected figure now matches the forecast and the medicine's yearly average.

5.1.5 Integrating the Models

The trained models were copied from Colab into the project as read-only files. The server loads each model once and keeps it in memory. Models are preloaded when the server starts, so the first request is fast. Forecasts beyond one day feed each prediction back in as the next

day's input. For speed, the engine forecasts one day for all 20 medicines in a single step. Pseudocode 5.4 shows this step.

```
INPUT for each medicine: category and last 30 days of demand
FOR day = 1 TO horizon
    rows <- one feature row per medicine for this day
            (season from the month, holiday from the calendar, lags from history)
    predictions <- model.predict(rows)        // one call for all medicines
    APPEND each prediction to that medicine's history and forecast
END FOR
RETURN forecasts
```

Pseudocode 5.4: Forecasting many medicines in one pass

Early runs were slow. A full decision run for 20 medicines took about 21 seconds. Profiling found the cause. The library re-checked the model before each of its 750 trees. The trees were then called directly. This gave identical predictions about twelve times faster. Through the API, a full run now takes about 1.5 seconds.

5.1.6 Frontend Implementation

The frontend has one page for each task in the use case diagram. All server calls go through a single API layer. This layer adds the user's access token to every request. When the access token expires, it renews it silently, as shown in Pseudocode 5.5.

```
FUNCTION Request(path, options)
    response <- send request with the current access token
    IF response is 401 (expired token) THEN
        new_token <- send refresh token to /token/refresh/
        IF new_token received THEN
            save new_token ; response <- send request again with new_token
        ELSE
            clear tokens ; RETURN "Session expired" (user goes to login)
    IF response is an error THEN
        RETURN a plain-language message for its status (400, 403, 409, 422, 429, 503 ...)
    RETURN response data
END FUNCTION
```

Pseudocode 5.5: Sending a request with automatic token renewal

The Model Lab and the decision engine panel were added for the viva demonstration. In the Model Lab, examiners can test each model with their own inputs. They can also see why it answered. The decision panel runs all four models together and supports a what-if dispense.

5.2 Testing

5.2.1 Testing Approach

Testing followed the plan in Chapter Three. It was carried out at five levels. Automated tests checked the backend code and the reproduced model results. Model tests checked accuracy on data the models had never seen. System tests checked each feature through the real interface. Performance tests measured response times. User acceptance testing with PGH pharmacy staff is the final level. System tests were run with an automated browser driven by Playwright. This made each test repeatable and produced a screenshot at every step.

5.2.2 Automated Unit and Integration Tests

The backend has 35 automated tests written with the Django test framework. Table 5.2 groups them by purpose. All 35 tests passed in the final run.

Group	Tests	What they check	Result
Authentication	4	Clear login messages, and blocking after ten attempts in a minute	All passed
Administration	2	All data tables are registered in the admin site	All passed
Model reproduction	4	Live re-tests match the Colab results for all four models	All passed
Explanations	3	Explanation bars add up exactly to each prediction	All passed
Model Lab access	6	Role checks, input validation and safe handling of missing model files	All passed
Decision engine	8	Orders for out-of-stock items, no duplicates, approval, expedite and FEFO dispensing	All passed
Page data	8	No placeholder values, real reports, search, demo date and stockout-day handling	All passed

Table 5.2: Automated backend tests

5.2.3 Model Testing

Table 5.3 shows the results of the model test cases planned in Table 3.7. Each model was re-tested live inside the finished system. The results matched the Colab notebooks to two decimal places.

ID	Test	Expected	Actual	Result
MT-01	Audit each dataset	No gaps or duplicates	All four files: no missing cells, no duplicate rows	Pass
MT-02	Leakage check	All correlations	Highest 0.76 (demand) and 0.42	Pass

ID	Test	Expected	Actual	Result
		below 0.98	(stockout)	
MT-03	Demand model vs naive baseline	Lower MAE	MAE 21.30 vs 39.36, an error cut of 45.9%	Pass
MT-04	Ranking under cross-validation	Same leader across folds	Random Forest led both supervised tasks	Pass
MT-05	Errors by category	Weak areas found	Antimalarials weakest (MAE 64.97); stockout days harder	Pass
MT-06	Stockout alert threshold	Recall of at least 0.80	Recall 0.802 at threshold 0.125	Pass
MT-07	Keep tuned model only if better	Better model deployed	Demand: tuned kept. Stockout: untuned kept	Pass
MT-08	K-Means vs hierarchical	ARI above 0.80	ARI 1.000	Pass
MT-09	Weights changed by 30% (500 runs)	Spearman above 0.90	Mean 0.994, lowest 0.972	Pass
MT-10	Resample orders 200 times	Most suppliers keep their tier	Each kept its main tier in at least 73% of runs	Pass

Table 5.3: Machine learning test results

Figure 5.1 shows one of these re-tests running in the Model Lab. The stockout model's live test reproduced a recall of 80.2% and a PR-AUC of 0.81. It caught 65 of 81 real stockouts and raised 49 false alarms.

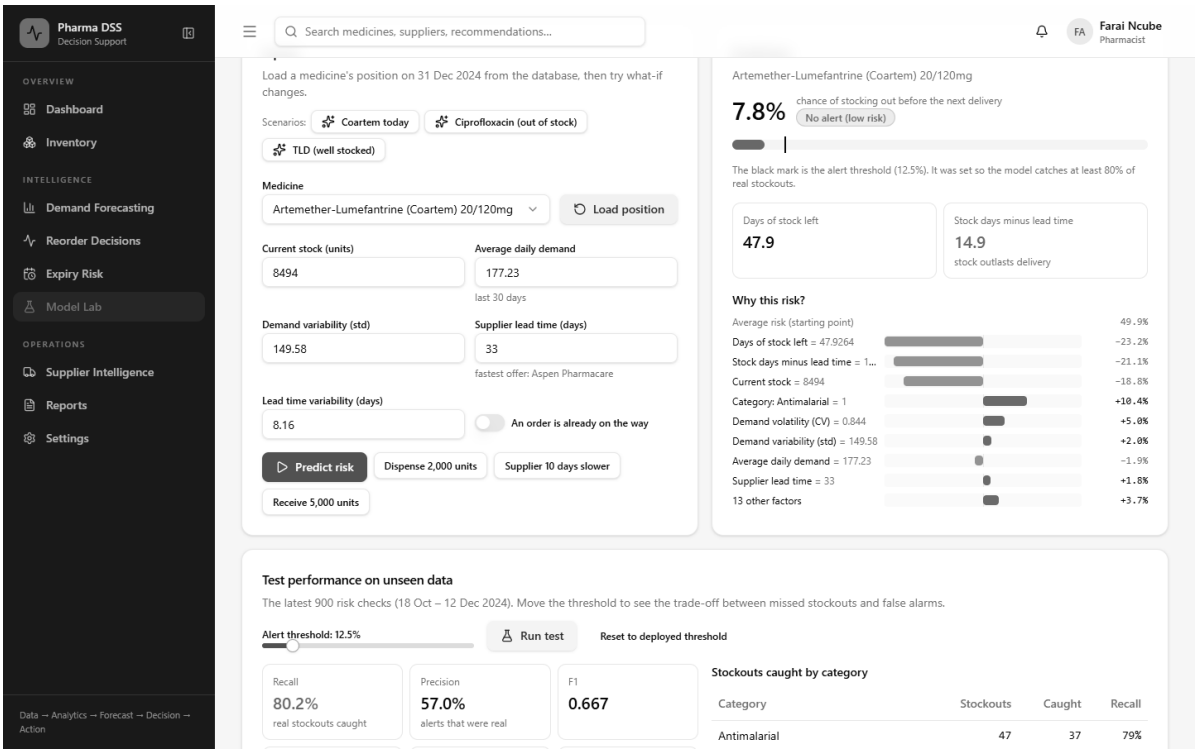


Figure 5.1: Live re-test of the stockout model in the Model Lab

5.2.4 System Functional Testing

Table 5.4 summarises the system test cases planned in Table 3.8. The actual results were recorded automatically during the test run.

ID	Test scenario	Expected	Actual	Result
ST-01	Log in with a valid email and password	Dashboard opens for the user's role	Dashboard opened for Farai Ncube (Pharmacist) with live indicators.	Pass
ST-02	Log in with a wrong password	Clear error message, no access	The page showed 'Wrong password'. The user stayed on the login page.	Pass
ST-03	Try more than 10 logins in one minute	Further attempts are blocked	Of eleven quick attempts, attempts 9 to 11 got HTTP 429. Earlier test logins used part of the limit. The page then showed 'Too many attempts'.	Pass
ST-04	Use the system after the access token expires	Token refreshes or the user is asked to log in	The invalid access token was rejected and silently replaced using the refresh token. The inventory page loaded without asking the user to log in again.	Pass

ID	Test scenario	Expected	Actual	Result
ST-05	Staff user tries to approve or change stock	Action refused for this role	The staff menu had no reorder pages. Server calls for recommendations, engine runs and dispensing all got HTTP 403.	Pass
ST-06	Open the demand forecast for a medicine	Forecast chart and model version shown	The chart showed an 80% prediction interval and total demand. Model version v1 and MAE 21.30 were listed.	Pass
ST-07	Medicine's stock falls into the risk zone	Reorder recommendation appears with quantity and supplier	Paracetamol had 5,285 units and needed no action. After 4,000 units were dispensed, it showed an order of 13,633 units at 64% risk.	Pass
ST-08	Pharmacist approves a recommendation	Status changes to approved and an audit entry is saved	Status changed to Approved with 'decided by Farai Ncube'. A purchase order was created.	Pass
ST-09	Open the supplier page	Scores, tiers and low-confidence flags shown	All 12 suppliers listed with score, on-time rate, lead time, defect rate and review status.	Pass
ST-10	Open the expiry risk page	Batches at risk listed by expiry date	54 batches listed by expiry date with daily use, projected unused units and advice. No batch was projected to expire unused in the demo data.	Pass
ST-11	Generate a stock report	PDF or CSV file downloads	File inventory-2024-11-30-to-2024-12-31.csv downloaded with 20 data rows. Header: medicineId,medicineName,strength,category,currentStock,reorderPoint,sa	Pass
ST-12	Medicine with no demand history	No crash. Forecast shown as unavailable	No crash. The page showed 'Not enough data'. It said at least 30 days of history are needed.	Pass
ST-13	Enter a negative stock quantity	Input rejected with a message	Rejected with the message 'Invalid request Quantity must be at least 1.'. Stock was not changed.	Pass
ST-14	Model file is missing or fails to load	System shows that predictions are unavailable	Forecast, Model Lab and engine requests returned HTTP 503 'Model artifact is missing'. No crash.	Pass

Table 5.4: System test results

All fourteen system tests passed. The following figures give annotated evidence for the main tests. Numbered markers point to the part of the screen being tested.

ST-01 and ST-02: Login

Figure 5.2 shows a login attempt with a wrong password. Marker 1 shows the clear error message. The user stays on the login page and gains no access.

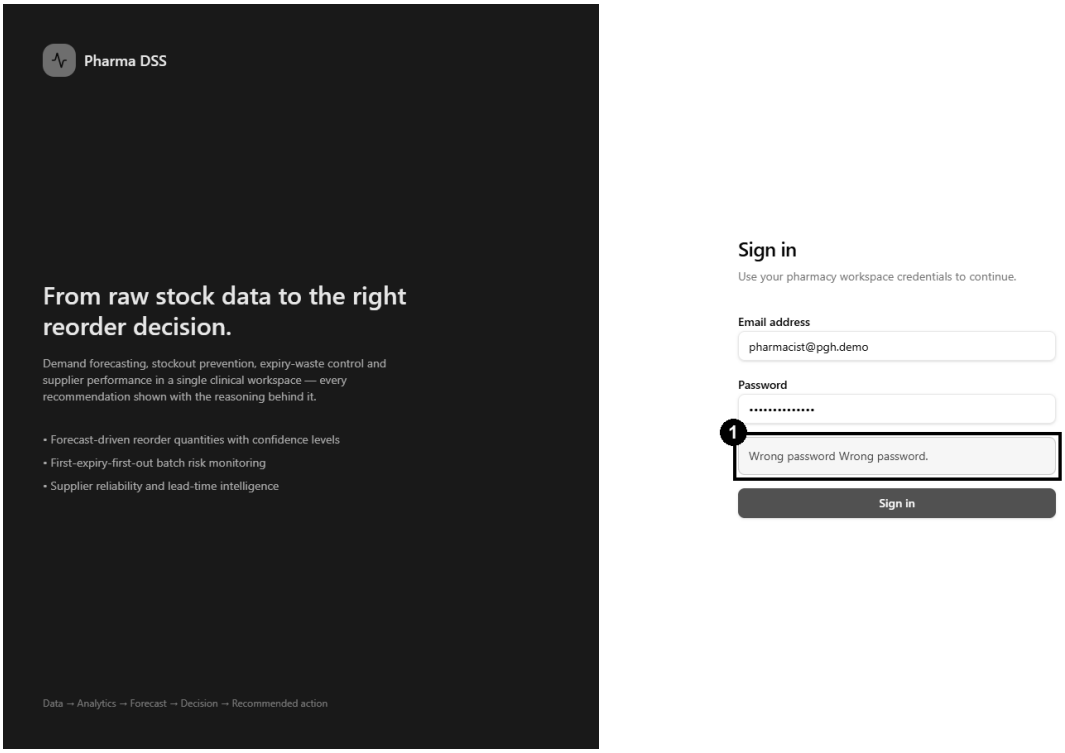


Figure 5.2: ST-02, login rejected with a wrong password

Figure 5.3 shows a successful login. Marker 1 shows the logged-in pharmacist, Farai Ncube. Marker 2 shows the pending reorder decisions waiting for this user.

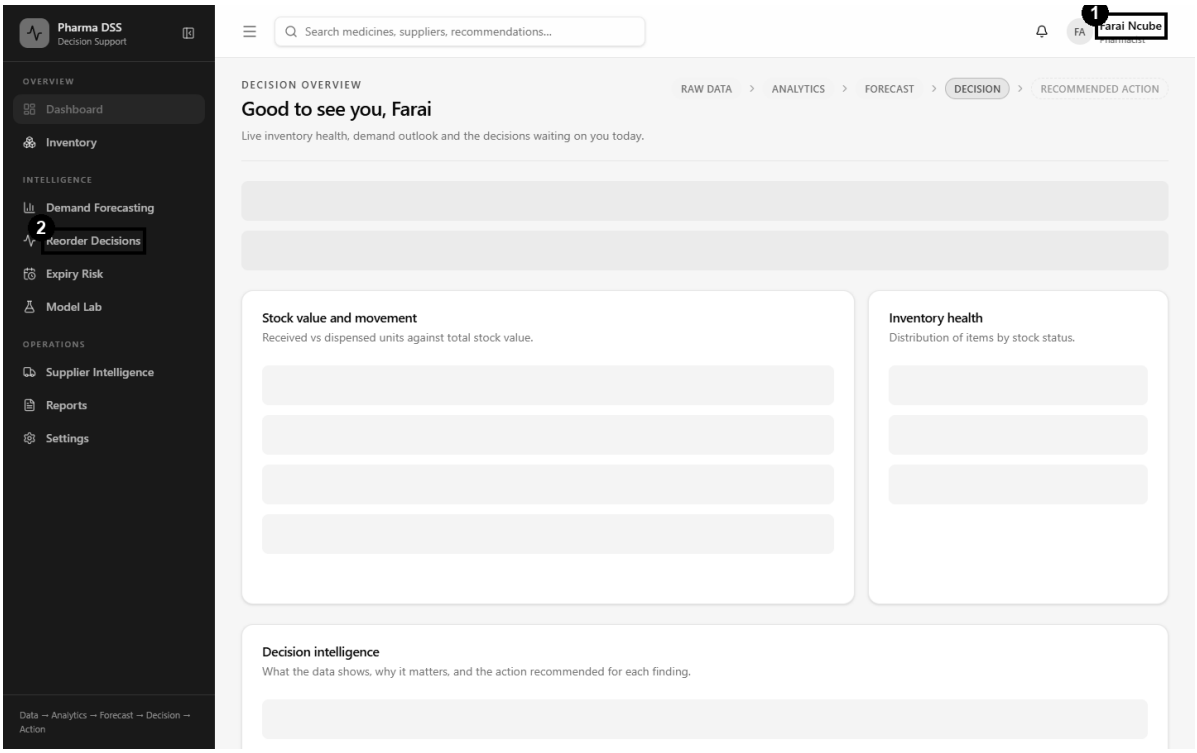


Figure 5.3: ST-01, dashboard after a successful login

ST-03 and ST-05: Security Controls

Eleven wrong passwords were sent in quick succession. The first ten were rejected as wrong passwords. The eleventh was blocked with HTTP 429. Figure 5.4 shows the message that the login page then displayed.

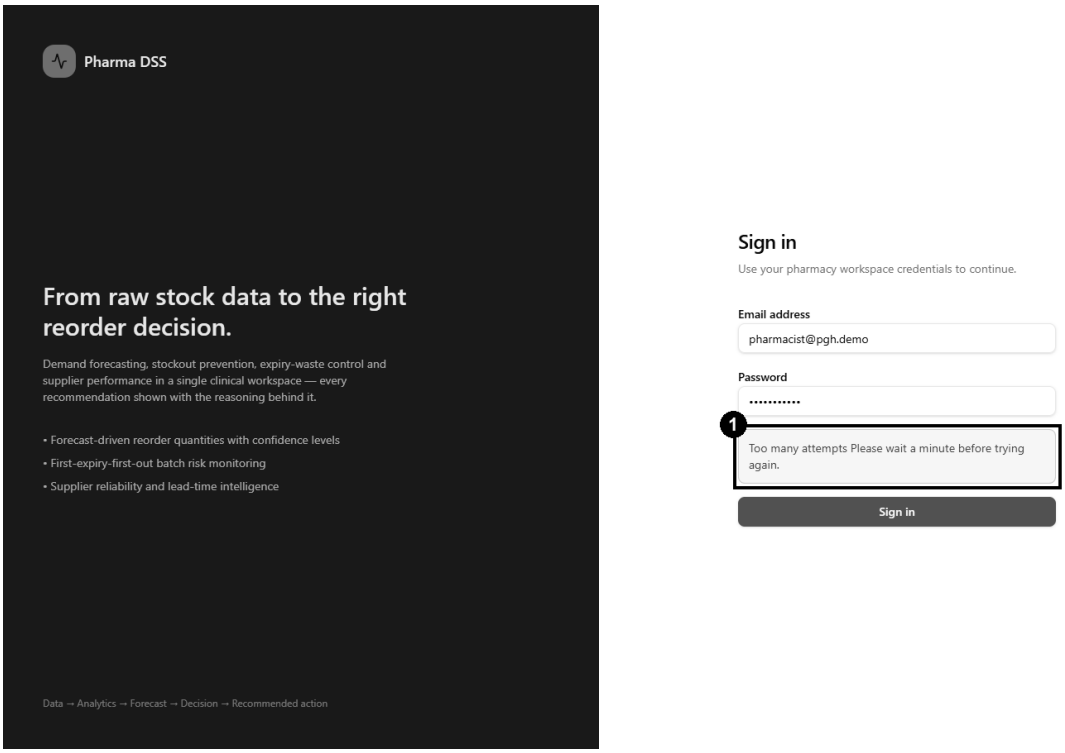


Figure 5.4: ST-03, repeated login attempts blocked

Figure 5.5 shows the system for a stores staff account. Marker 1 shows that the menu offers no forecasting, reorder or supplier pages. Direct requests to those services were also refused by the server.

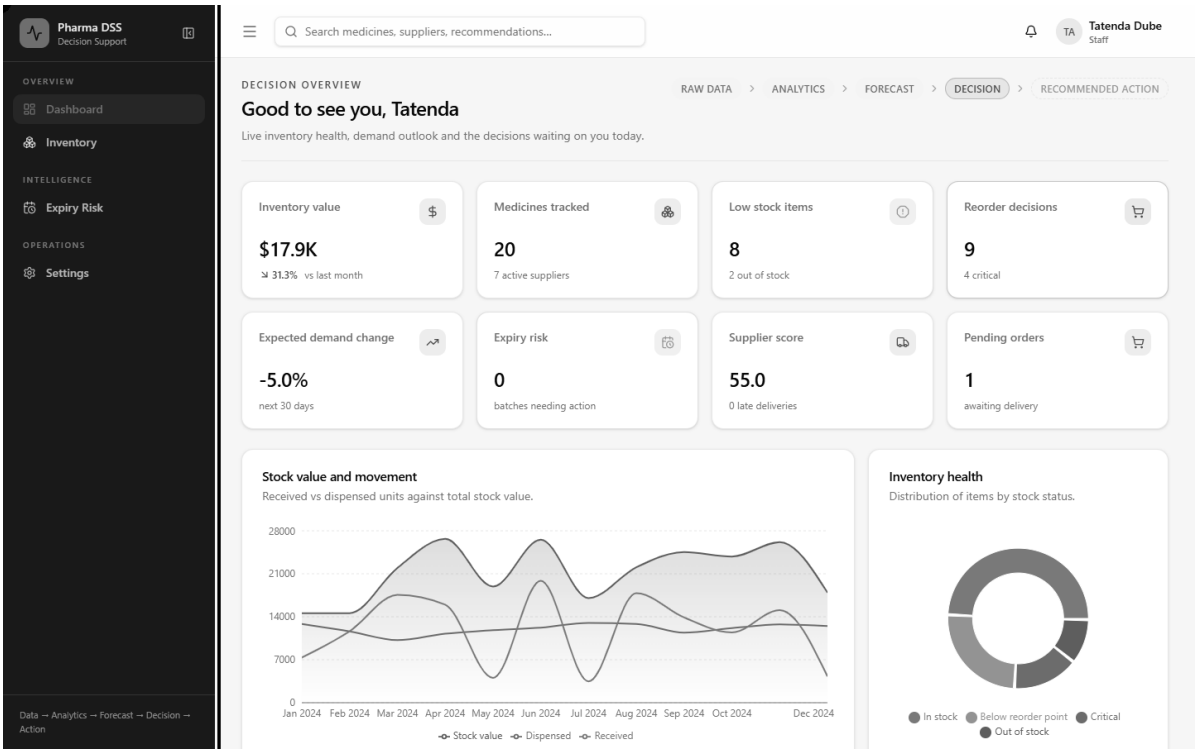


Figure 5.5: ST-05, reduced menu for the stores staff role

ST-06 and ST-12: Demand Forecasting

Figure 5.6 shows the forecast page. Marker 1 shows the model version. Marker 2 shows the 80% prediction interval around the forecast.

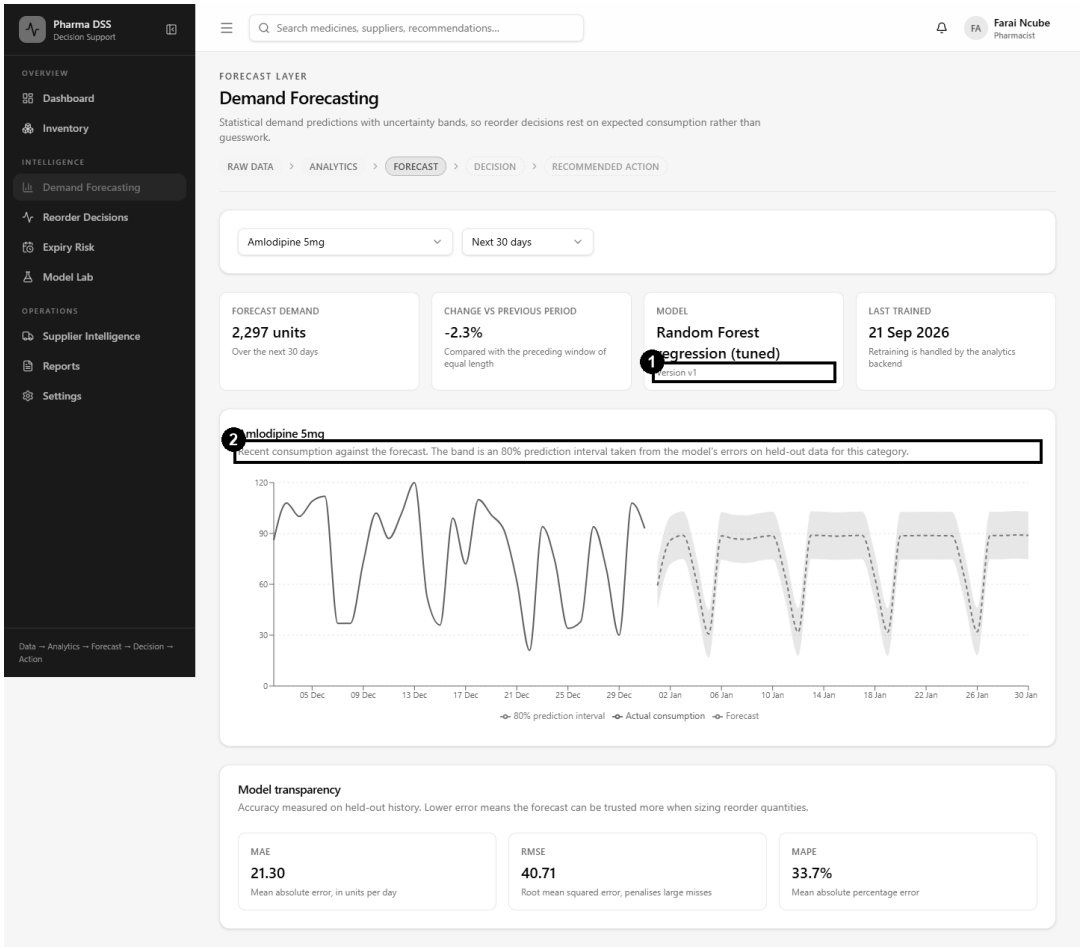


Figure 5.6: ST-06, demand forecast with model details

Figure 5.7 shows a medicine with no demand history. Marker 1 shows that the page explains the problem instead of crashing.

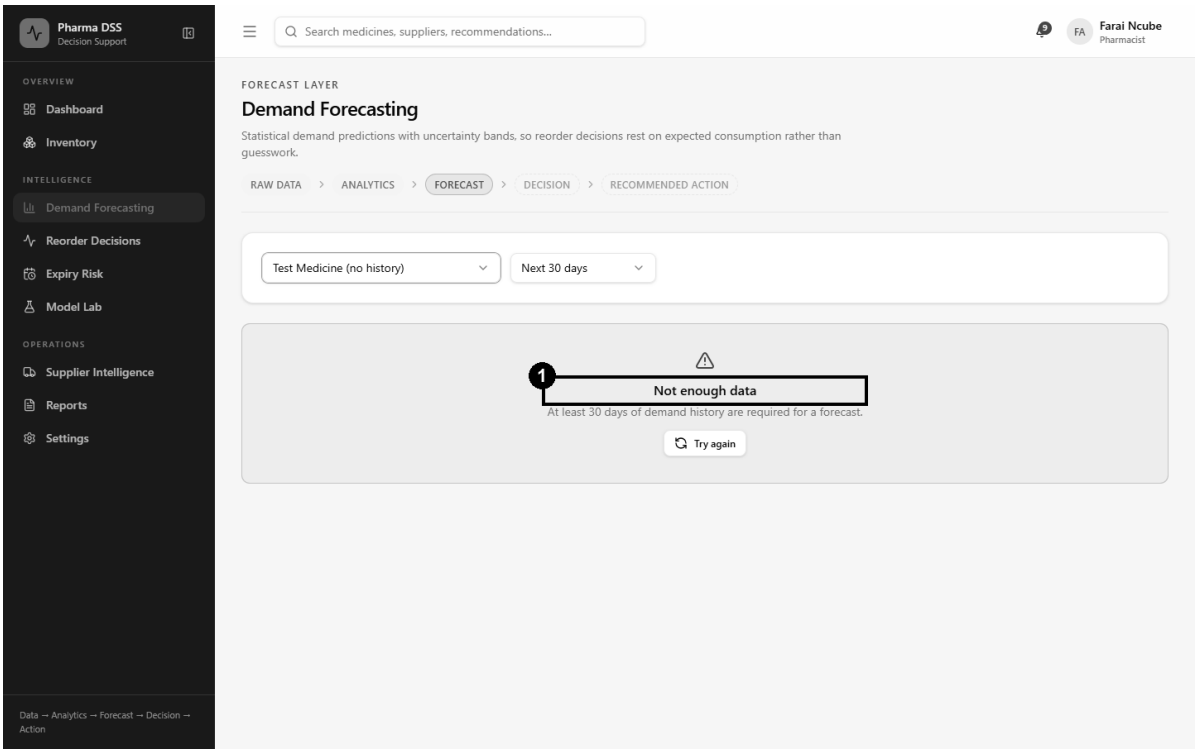


Figure 5.7: ST-12, a medicine with no history

ST-07, ST-08 and ST-13: The Decision Engine

Figure 5.8 shows the decision engine before the what-if test. Paracetamol had 5,285 units in stock and needed no action.

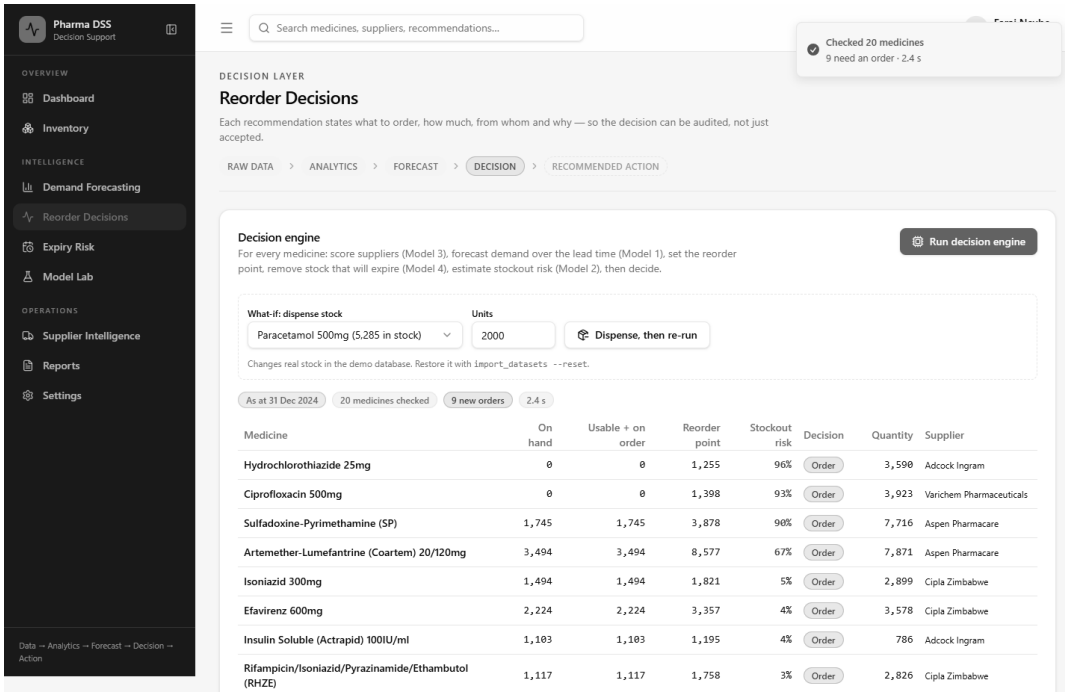

Pharma DSS
Decision Support
OVERVIEW
Dashboard
Inventory
INTELLIGENCE
Demand Forecasting
Reorder Decisions
Expiry Risk
Model Lab
OPERATIONS
Supplier Intelligence
Reports
Settings
Data → Analytics → Forecast → Decision → Action
Search medicines, suppliers, recommendations...
Checked 20 medicines
9 need an order · 2.4 s
DECISION LAYER
Reorder Decisions
Each recommendation states what to order, how much, from whom and why — so the decision can be audited, not just accepted.
RAW DATA
ANALYTICS
FORECAST
DECISION
RECOMMENDED ACTION
Decision engine
For every medicine: score suppliers (Model 3), forecast demand over the lead time (Model 1), set the reorder point, remove stock that will expire (Model 4), estimate stockout risk (Model 2), then decide.
Run decision engine
What-if: dispense stock
Paracetamol 500mg (5,285 in stock)
Units
2000
Dispense, then re-run
Changes real stock in the demo database. Restore it with import_datasets --reset.
As at 31 Dec 2024
20 medicines checked
9 new orders
2.4 s
Medicine
On hand
Usable + on order
Reorder point
Stockout risk
Decision
Quantity
Supplier
Hydrochlorothiazide 25mg 0 0 1,255 96% Order 3,590 Adcock Ingram
Ciprofloxacin 500mg 0 0 1,398 93% Order 3,923 Varichem Pharmaceuticals
Sulfadoxine-Pyrimethamine (SP) 1,745 1,745 3,878 90% Order 7,716 Aspen Pharmacare
Artemether-Lumefantrine (Coartem) 20/120mg 3,494 3,494 8,577 67% Order 7,871 Aspen Pharmacare
Isoniazid 300mg 1,494 1,494 1,821 5% Order 2,899 Cipla Zimbabwe
Efavirenz 600mg 2,224 2,224 3,357 4% Order 3,578 Cipla Zimbabwe
Insulin Soluble (Actrapid) 100IU/ml 1,103 1,103 1,195 4% Order 786 Adcock Ingram
Rifampicin/Isoniazid/Pyrazinamide/Ethambutol (RHZE) 1,117 1,117 1,758 3% Order 2,826 Cipla Zimbabwe
Tenofovir/Lamivudine/Dolutegravir (TLD) 7,073 7,073 8,878 1% Order 5,424 Cipla Zimbabwe
Cotrimoxazole 480mg 2,764 2,764 2,479 3% No action – Varichem Pharmaceuticals
Amlodipine 5mg 5,009 5,009 2,565 2% No action – Adcock Ingram
Paracetamol 500mg 5,285 5,285 3,935 2% No action – Varichem Pharmaceuticals
Metformin 500mg 4,222 4,222 2,342 2% No action – Adcock Ingram
Salbutamol Inhaler 100mcg 1,458 1,458 1,317 2% No action – Aspen Pharmacare
Amoxicillin Capsules 500mg 8,219 8,219 3,564 1% No action – Varichem Pharmaceuticals
Ferrous Sulphate + Folic Acid 5,427 5,427 3,854 1% No action – Interpharm Zimbabwe
Oral Rehydration Salts (ORS) Sachets 7,861 7,861 5,007 1% No action – Plus Health Pharmaceuticals
Omeprazole 20mg 2,932 2,932 2,048 1% No action – Plus Health Pharmaceuticals
Ibuprofen 400mg 5,189 5,189 2,062 1% No action – Varichem Pharmaceuticals
Zinc Sulphate 20mg 4,066 4,066 3,159 1% No action – Interpharm Zimbabwe
Assumptions: 95% service level (z = 1.6449), lead time + 3 buffer days, 14-day review period, ordering cost USD 25, holding cost 20% a year, maximum 90 days of cover per order.
All priorities
All decisions
9 recommendations · $12,065 awaiting approval
Hydrochlorothiazide 25mg
Critical
Awaiting decision
Adcock Ingram · lead time 25 days · stockout risk 96%
3,590 units
$69
CURRENT STOCK
0
REORDER POINT
1,255
PREDICTED DEMAND
1,119
SAFETY STOCK
136
DAYS TO REORDER POINT
Already reached
WHY THIS DECISION
Hydrochlorothiazide is out of stock. The stockout model gives a 96% chance of running out before the next delivery (alert threshold 12.5%). Forecast demand over the next 28 days (25-day lead time + 3 buffer days) is 1,119 units. Order 3,590 units from Adcock Ingram (Preferred supplier, score 0.78).
Show how the four models reached this decision
Review and approve
Reject
Sulfadoxine-Pyrimethamine (SP)
Critical
Awaiting decision
Aspen Pharmacare · lead time 35 days · stockout risk 90%
7,716 units
$711
CURRENT STOCK
1,745
REORDER POINT
3,878
PREDICTED DEMAND
3,384
SAFETY STOCK
494
DAYS TO REORDER POINT
Already reached
WHY THIS DECISION
Usable stock of 1,745 units is at or below the reorder point of 3,878. The stockout model gives a 90% chance of running out before the next delivery (alert threshold 12.5%). Forecast demand over the next 38 days (35-day lead time + 3 buffer days) is 3,384 units. Order 7,716 units from Aspen Pharmaca
Show how the four models reached this decision
Review and approve
Reject

Figure 5.8: Decision engine results before the what-if dispense

Next, 4,000 units of Paracetamol were dispensed, and the engine ran again. Figure 5.9 shows the result. Marker 1 shows that Paracetamol now needs an order with a 64% stockout risk.

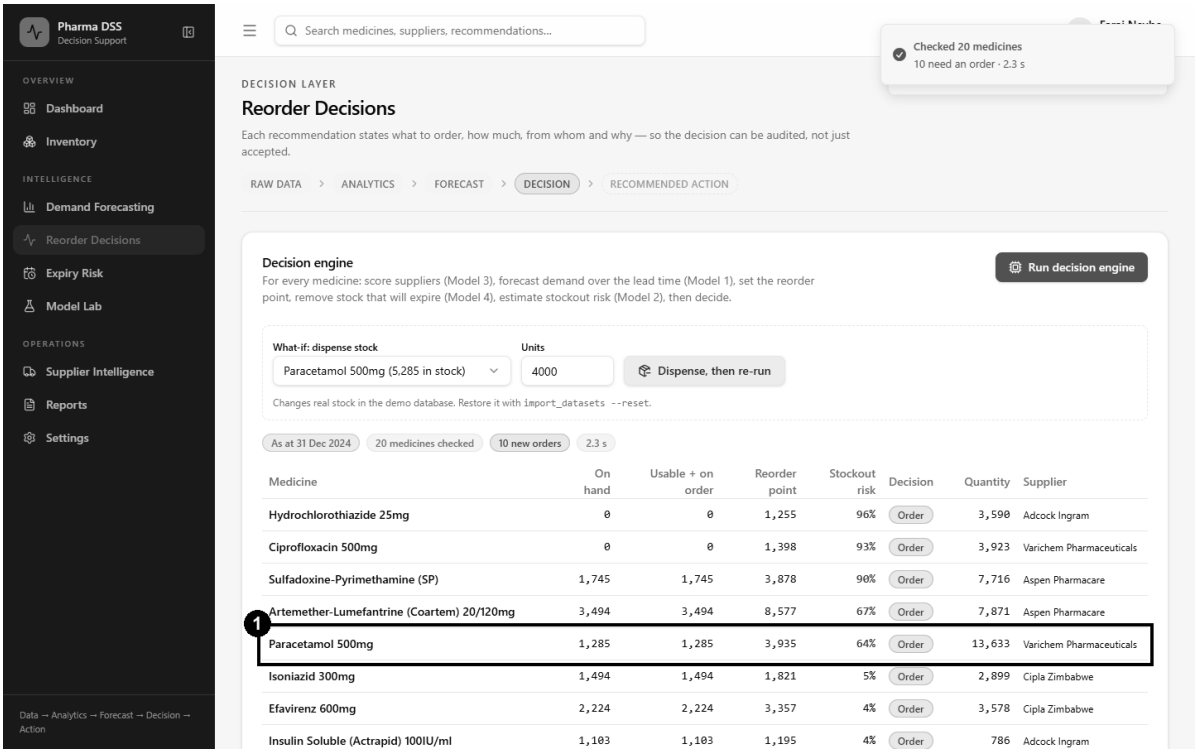


Figure 5.9: ST-07, Paracetamol moves into the risk zone after dispensing

Figure 5.10 shows a dispense of minus five units. Marker 1 shows that the system rejected it with a clear message.

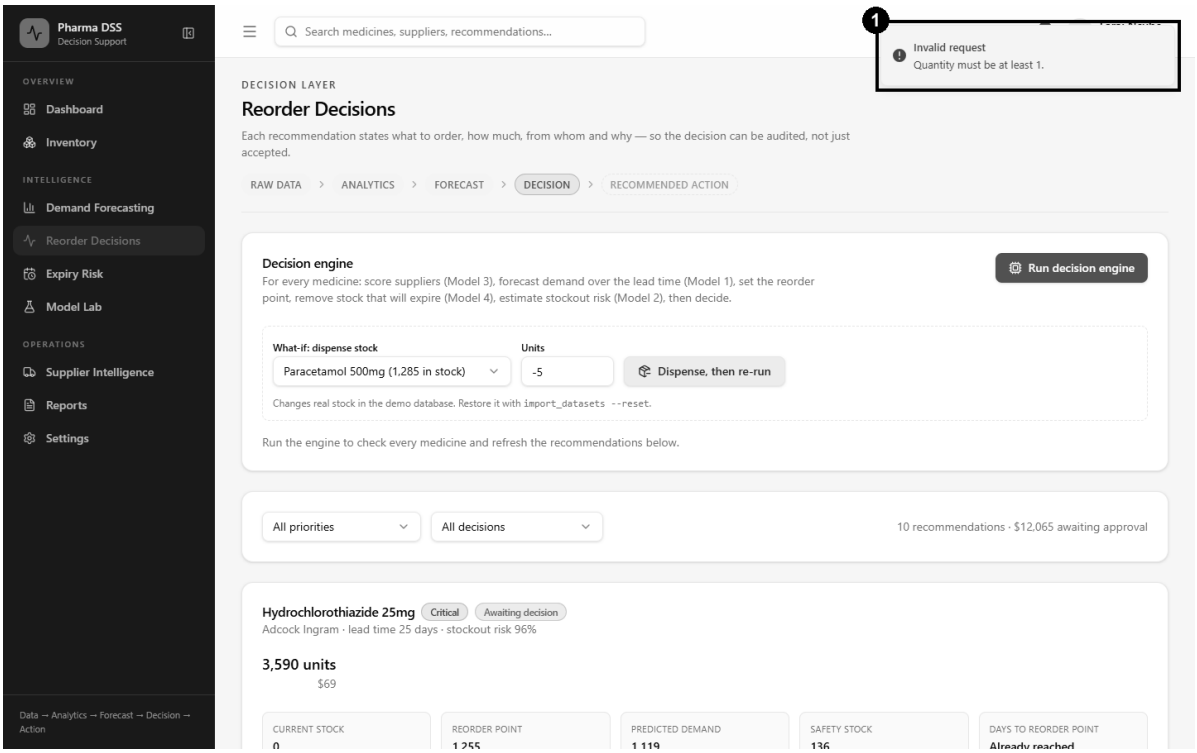


Figure 5.10: ST-13, a negative quantity is rejected

Figure 5.11 shows the full chain of reasoning for the Paracetamol recommendation. It lists the supplier choice, forecast, reorder point, expiry check, stockout risk and quantity.

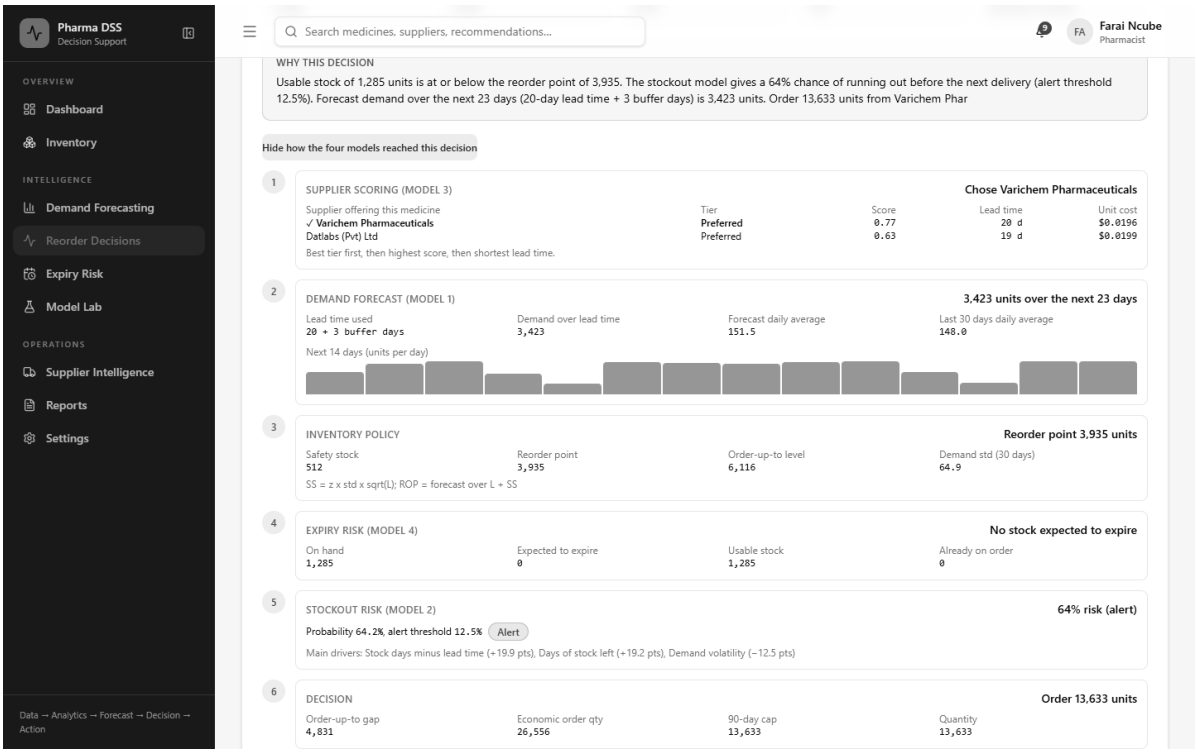


Figure 5.11: The four-model decision chain for a recommendation

The pharmacist then approved the recommendation. Figure 5.12 shows the result. Marker 1 shows the approved status. Marker 2 records who made the decision.

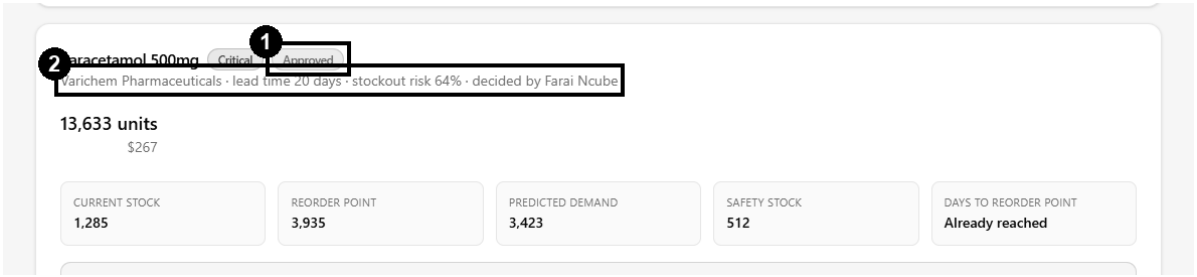


Figure 5.12: ST-08, recommendation approved and recorded

ST-09, ST-10 and ST-11: Suppliers, Expiry and Reports

Figure 5.13 shows the supplier page with scores and tiers. Marker 1 shows a supplier flagged for review because of its low tier.

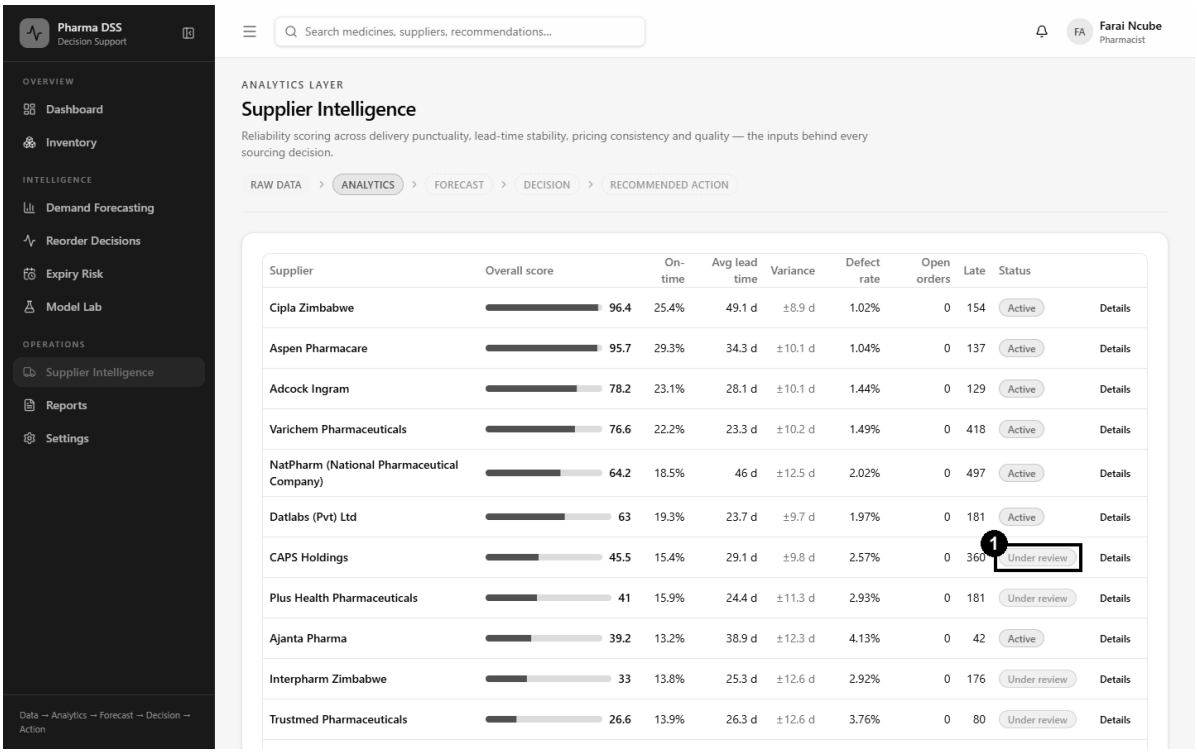


Figure 5.13: ST-09, supplier performance page

Figure 5.14 shows the expiry page, with batches in first-expiry-first-out order.

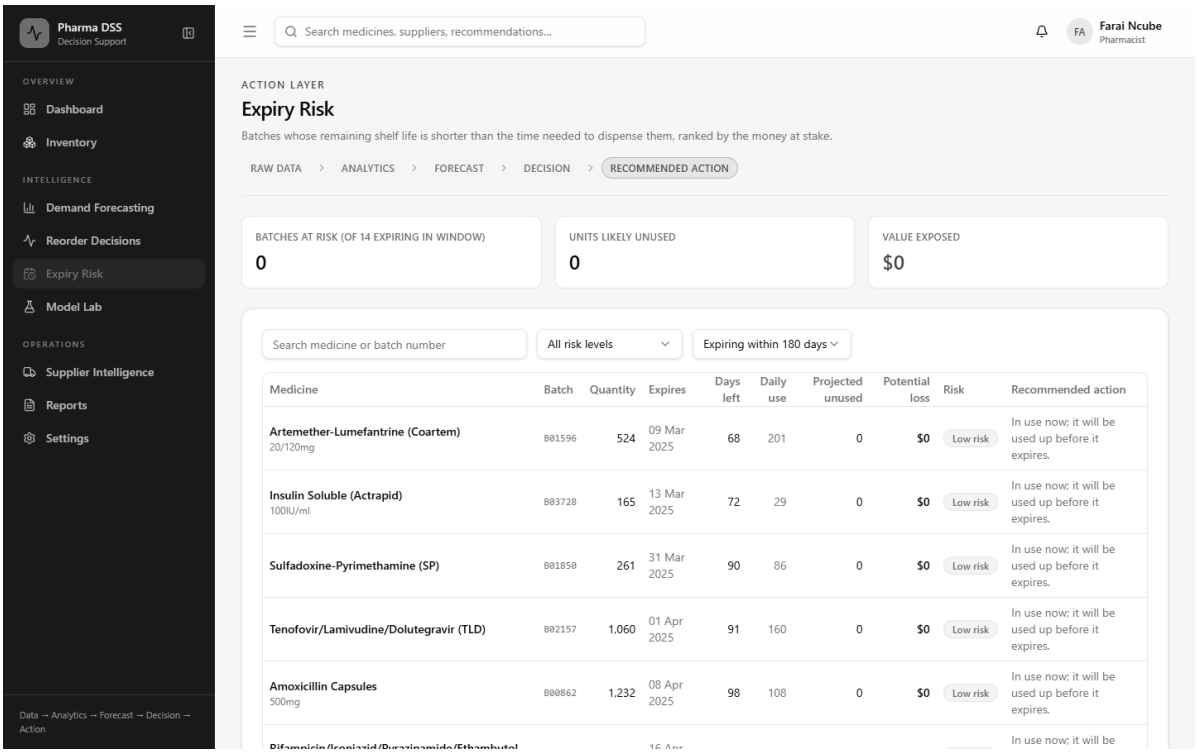


Figure 5.14: ST-10, expiry risk page

Figure 5.15 shows a generated inventory report. Marker 1 shows the row count. The downloaded CSV file was opened and checked. It held 20 data rows, one for each medicine.

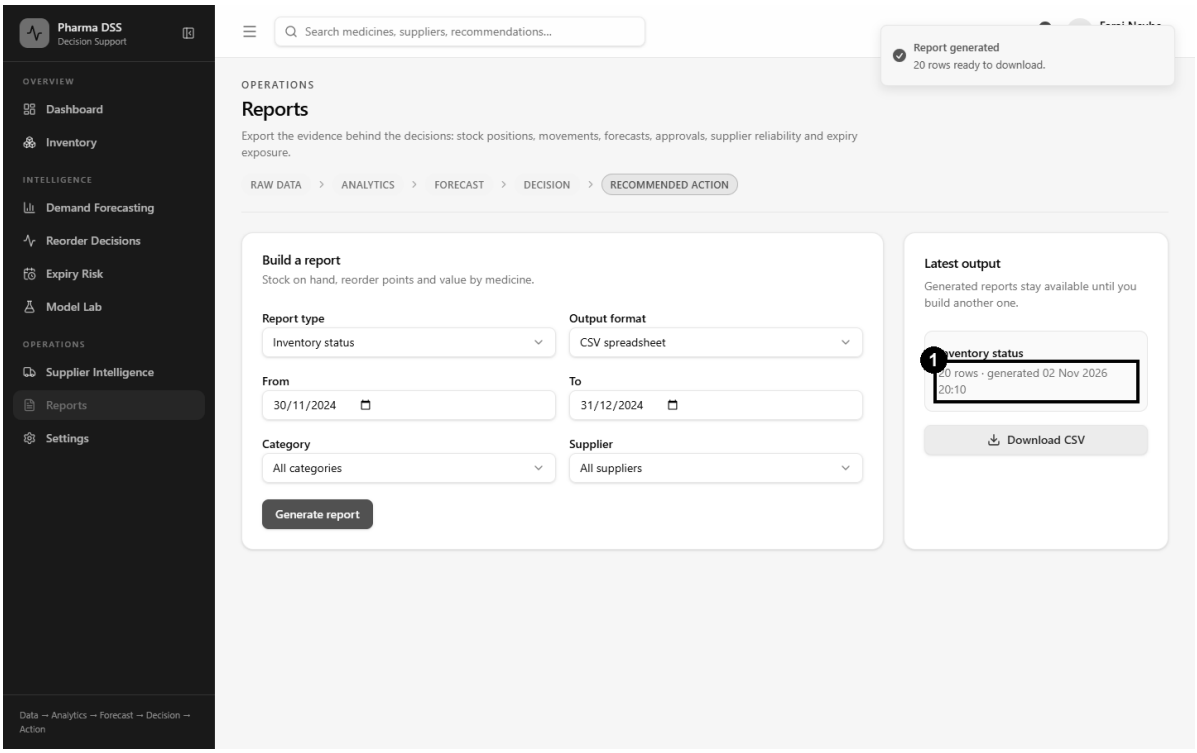


Figure 5.15: ST-11, inventory report generated and downloaded

The Model Lab What-If Test

Figures 5.16 and 5.17 show a what-if test in the Model Lab. Coartem first showed a stockout risk of 67.5%, which raised an alert. After 5,000 units were added, the risk fell to 7.8% and the alert cleared.

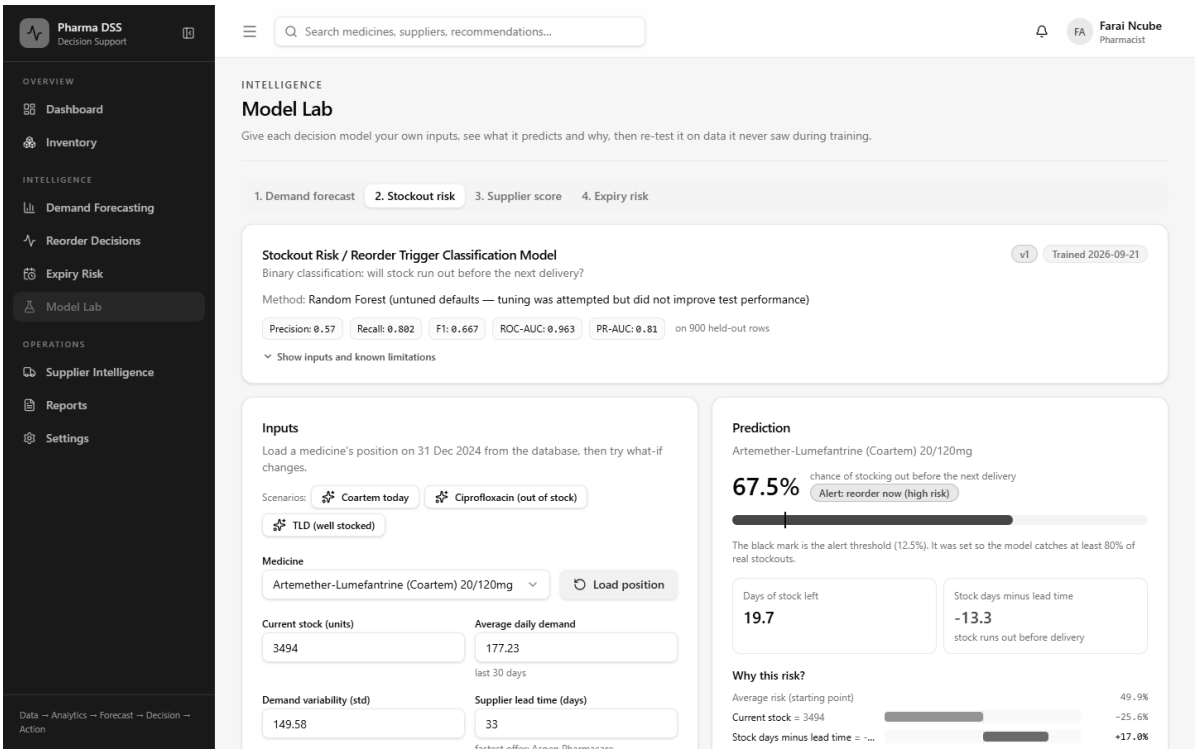


Figure 5.16: Stockout risk for Coartem before the what-if change

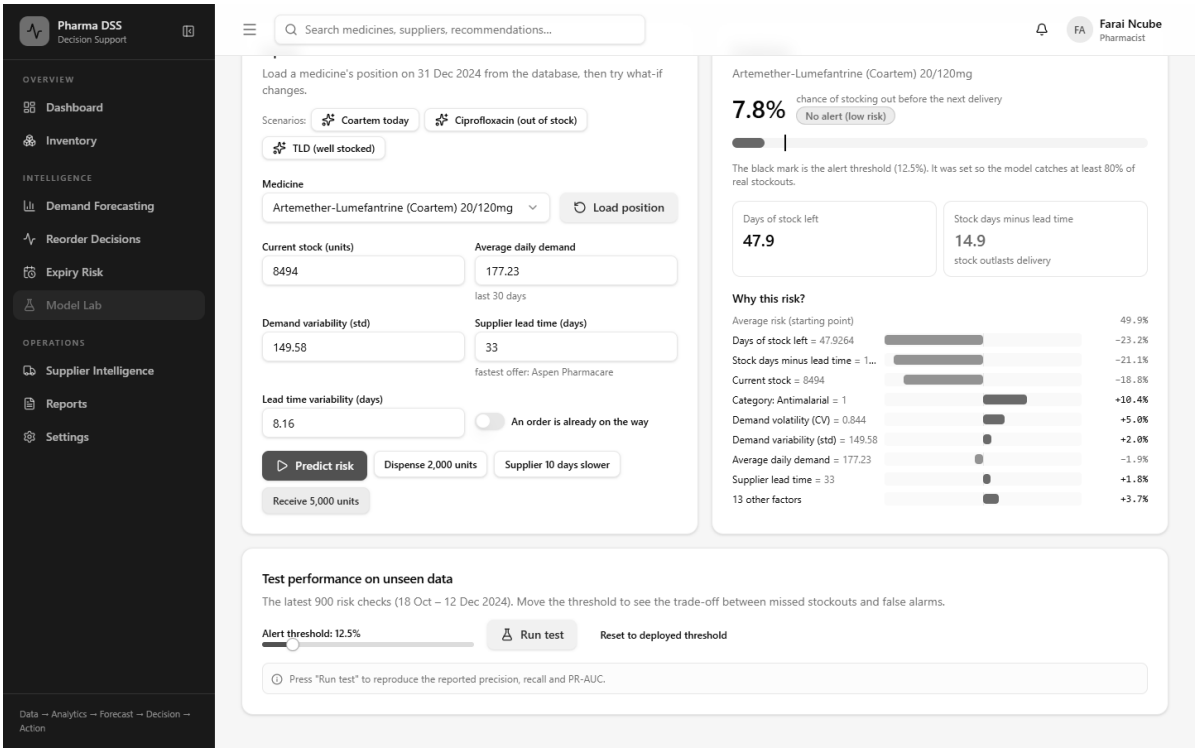


Figure 5.17: Stockout risk for Coartem after adding 5,000 units

5.2.5 Performance Testing

Response times were measured on the development laptop with the server running locally. Table 5.5 shows the median of five runs for each operation.

Operation	Target	Measured (median)	Result
Run decision engine for 20 medicines	under 5 s	1.51 s	Pass
30-day demand forecast for one medicine	under 3 s	0.18 s	Pass
90-day demand forecast for one medicine	under 3 s	0.50 s	Pass
Model Lab: one demand prediction with explanation	under 3 s	0.10 s	Pass
Model Lab: re-test demand model (repeat run, cached)	under 3 s	0.02 s	Pass
Inventory list (20 medicines)	under 3 s	0.01 s	Pass
Dashboard indicators	under 3 s	0.17 s	Pass
Generate PDF inventory report	under 3 s	0.11 s	Pass
Dashboard page fully loaded in the browser	under 3 s	1.05 s	Pass

Table 5.5: Performance test results

All operations met their targets. The slowest step is the very first page load after the development server starts. This happens because the development server compiles each page on first use. A production build is compiled in advance, so users do not see this delay.

5.2.6 Defects Found and Fixed

Testing found several defects. Each one was fixed and then re-tested. Table 5.6 lists the most important ones.

ID	Defect found	Severity	Fix
D-01	Approving a recommendation crashed because the order date was never set.	High	Order date set; delivery date now uses the supplier lead time.
D-02	Several screens showed fixed sample numbers instead of calculated ones.	High	Every figure now comes from the data and the decision engine.
D-03	The interface and the API used different data formats on four pages.	High	Formats aligned and checked by automated tests.
D-04	Stockout days lowered demand averages, e.g. a false 202% rise for Ibuprofen.	High	Stockout days are excluded from demand statistics.
D-05	After approval, the engine proposed a second, duplicate order.	Medium	It now advises expediting the open order instead.

ID	Defect found	Severity	Fix
D-06	A full decision run took about 21 seconds.	Medium	Trees called directly; a run now takes about 1.5 seconds.
D-07	Pharmacists could not approve orders, unlike the design.	Medium	Role permissions updated to match Chapter Four.
D-08	Reports were saved as JSON files with the wrong extension.	Medium	Real CSV and PDF files are now produced.
D-09	A missing model file revealed the server's folder path.	Medium	Messages now name the file only.
D-10	The model list crashed (HTTP 500) when a model file was missing.	Medium	It now returns a clear 'model unavailable' message.
D-11	Some errors showed only 'Something went wrong'.	Low	Plain messages added for too many attempts, conflicts, missing data and unavailable models.
D-12	Every medicine looked close to expiry because of how demo batches were chosen.	Low	Batches now reflect a realistic mix of expiry dates.
D-13	The 10-per-minute login limit was configured but never enforced.	High	Rate limiting switched on and covered by a new automated test.

Table 5.6: Defect log

5.2.7 User Acceptance Testing

User acceptance testing checks whether real users find the system useful. It will be carried out with pharmacists and technicians at the PGH Pharmacy Department. Each participant will complete five tasks without help. They will log in, check a medicine's stock and forecast, and run the decision engine. They will then approve one recommendation and download a report. They will then rate it on a five-point scale using the form in Appendix C. The system passes if at least 70% rate it useful or very useful. Table 5.7 shows the tasks and acceptance criteria.

Task	Acceptance criterion
Log in with your own account	Completed without help
Find the stock, reorder point and forecast for Coartem	Completed in under two minutes
Run the decision engine and explain one recommendation	Participant can state the main reason
Approve or reject a recommendation	Completed without help
Download the inventory report	File opens correctly
Overall rating	At least 70% rate the system useful or very useful

Table 5.7: User acceptance test plan

The results of this testing will be reported in Chapter Six.

5.3 Summary

All planned parts of the system were implemented. These include the four models, the decision engine, the Model Lab and every designed page. All 35 automated tests, all 10 model tests and all 14 system tests passed. Performance targets were met, with a full decision run taking about 1.5 seconds.

Several challenges arose during coding and testing. The datasets were simulated, so results must still be confirmed with real PGH records. Stockout days hid true demand until the engine was changed to ignore them. The expiry classifier did not beat a simple rule, so the rule was kept. The development server was slow to start on the test laptop. Testing also found thirteen defects, and all were fixed and re-tested.

Some areas need further work. User acceptance testing with PGH staff must be completed. The models should be retrained on real hospital data. Multi-factor login should be added for administrators. Links to eLMIS and Impilo would remove the need for manual data loading. The next chapter discusses the results against the objectives of the study.

CHAPTER SIX: RESULTS AND DISCUSSIONS

6.0 Introduction

Chapter Five showed that the system was built and that it passed its tests. This chapter asks a different question. Did the system achieve what the study set out to do? Each objective from Chapter One is linked to the features that meet it. Screen dumps of the working system show each objective in action. The results are then compared with the studies reviewed in Chapter Two. The chapter ends with the limitations, recommendations for further research and a conclusion.

6.1 Achievement of Objectives

Table 6.1 gives a summary before each objective is discussed.

Objective	Feature that meets it	Key evidence	Status
1. Analyse current practices at PGH	Background study and gap analysis	Five gaps identified (Section 1.2.2)	Partly achieved
2. Forecast daily demand	Model 1: Random Forest regression	MAE 21.30 vs 39.36 naive	Achieved
3. Predict stockout risk	Model 2: Random Forest classifier	Recall 80.2%, PR-AUC 0.81	Achieved
4. Score suppliers	Model 3: weighted score and K-Means	ARI 1.000; rank stability 0.994	Achieved
5. Flag expiry risk	Model 4: depletion rule with FEFO	95.53% agreement; F1 0.951	Achieved
6. Integrate into a secure web system	Decision engine, Model Lab, role-based pages	All four models run in 1.5 seconds	Achieved
7. Evaluate the system	Automated, model, system and performance tests	59 of 59 tests passed	Partly achieved

Table 6.1: Summary of objectives and their achievement

6.1.1 Objective 1: Analyse Current Inventory and Procurement Practices

The first objective was to understand how the PGH pharmacy plans and orders medicines. Chapter One described the current tools: stock cards, ledgers, spreadsheets, eLMIS reports and Impilo. It found five gaps. There was no demand forecasting, no early stockout warning and no objective supplier rating. Expiry tracking was weak, and reports did not lead to clear actions. Each gap shaped one part of the system, as Table 6.2 shows.

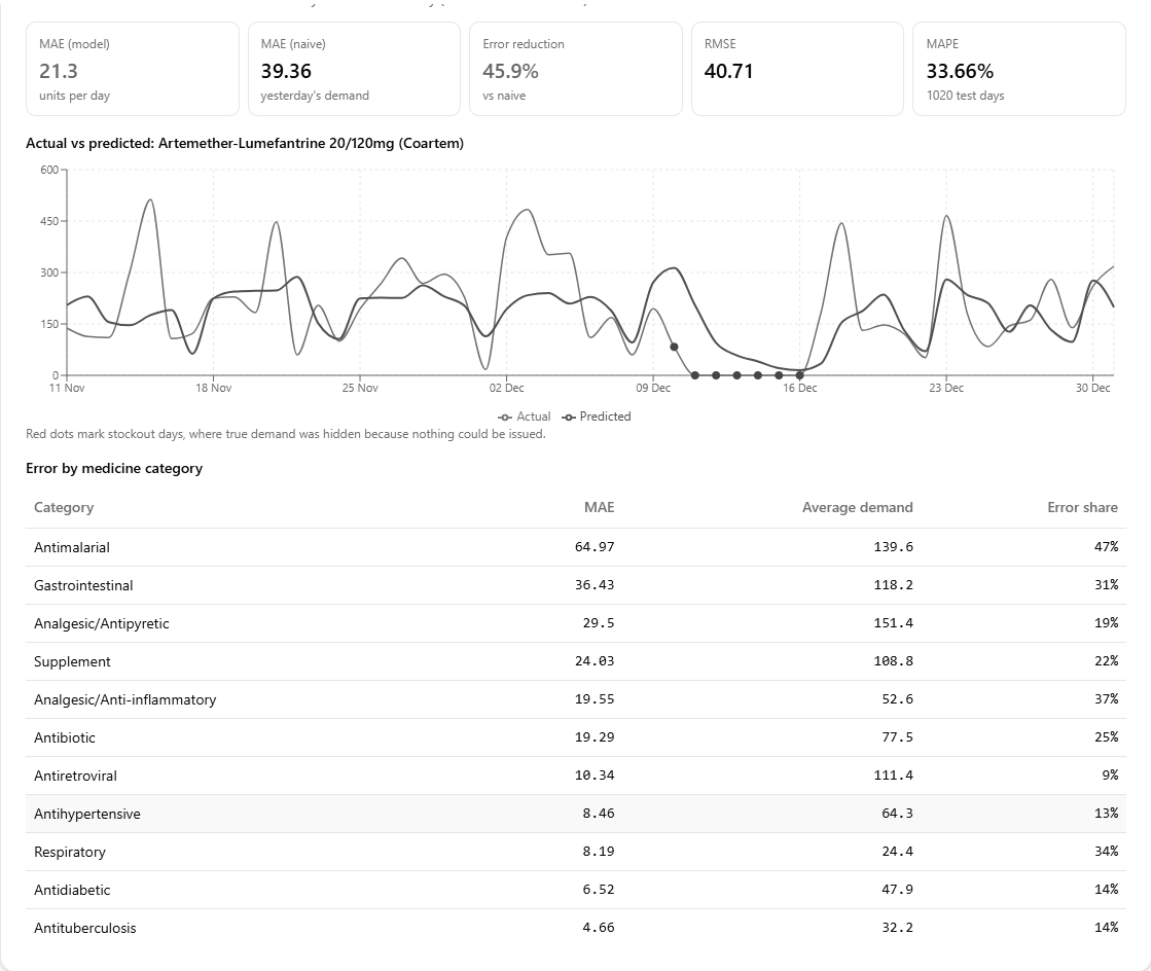
Gap found	Feature that addresses it
No demand forecasting	Daily forecasts for every medicine, with an 80% prediction interval
No early stockout warning	Stockout probability before the next delivery, with alerts
No objective supplier rating	Supplier scores and tiers from delivery records
Weak expiry tracking	First-expiry-first-out check of every batch against the forecast
No link from data to action	Decision engine that recommends what, how much and from whom

Table 6.2: Gaps in the current system and the features that address them

This objective is marked as partly achieved. The gaps came from the literature, national reports and the design of the current tools. They still need to be confirmed with PGH staff through the planned questionnaire and interviews.

6.1.2 Objective 2: Forecast Daily Medicine Demand

The second objective was a machine learning model that forecasts daily demand for each medicine. A single Random Forest model was trained across all 20 medicines. On 1,020 unseen test days, its mean absolute error was 21.30 units a day. A naive forecast, which repeats yesterday's demand, had an error of 39.36. The model therefore cut the error by 45.9%. Figure 6.1 shows this result being reproduced live in the Model Lab.



Category	MAE	Average demand	Error share
Antimalarial	64.97	139.6	47%
Gastrointestinal	36.43	118.2	31%
Analgesic/Antipyretic	29.5	151.4	19%
Supplement	24.03	108.8	22%
Analgesic/Anti-inflammatory	19.55	52.6	37%
Antibiotic	19.29	77.5	25%
Antiretroviral	10.34	111.4	9%
Antihypertensive	8.46	64.3	13%
Respiratory	8.19	24.4	34%
Antidiabetic	6.52	47.9	14%
Antituberculosis	4.66	32.2	14%

Figure 6.1: Demand model re-tested on unseen data in the Model Lab

The model forecasts steady medicines best. Errors were lowest for TB medicines and antiretrovirals. They were 9% to 14% of average demand. Errors were highest for antimalarials, at about 47% of average demand. Malaria demand rises and falls sharply in the rainy season. This makes it hard to predict. The black dots in Figure 6.1 mark stockout days, when true demand was hidden.

These results agree with Mbonyinshuti et al. (2022). They also found Random Forest a reliable forecaster of essential medicines in Rwanda. One shared model for all medicines also won the M5 competition (Makridakis et al., 2022). One finding was unexpected. The season feature contributed only 0.1% to the model's decisions. The 7-day average and the demand a week earlier contributed 65%. Recent demand already reflects the season, so the model learns the season from it indirectly.

6.1.3 Objective 3: Predict Stockout Risk

The third objective was a classifier that predicts stockout risk before the next delivery. Only about one check in ten was a stockout, so accuracy would have been misleading. The model was judged instead by recall and PR-AUC (Saito & Rehmsmeier, 2015). At an alert threshold of 0.125, it caught 80.2% of real stockouts. Its PR-AUC was 0.81, against 0.09 for a model that never raises an alert. Figure 6.2 shows the model scoring Coartem as at 31 December 2024.

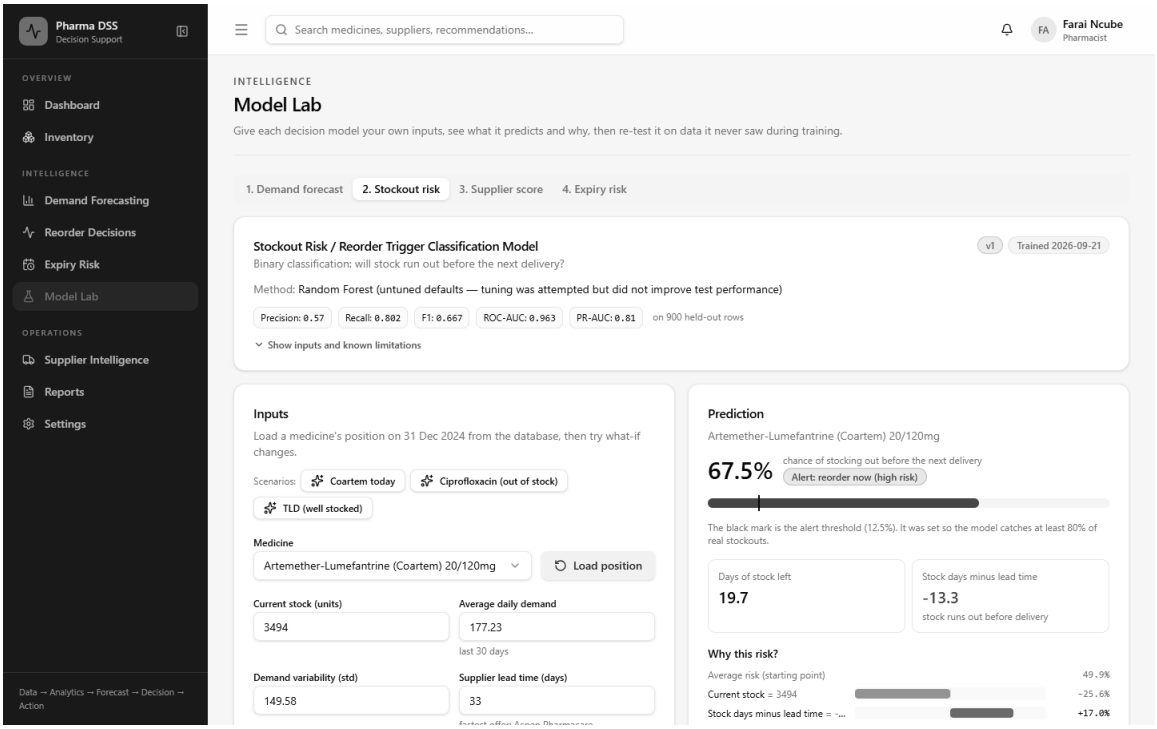


Figure 6.2: Stockout risk for Coartem, with the reasons behind it

The model gave Coartem a 67.5% risk. Its stock would last about 20 days. The supplier needs about 33 days to deliver. The explanation bars show that this gap drove the alert. Three inputs mattered most. They were days of stock left, stock days minus lead time and current stock. Together they made up 75% of its decisions. This matches inventory theory, where stock must outlast the lead time (Silver et al., 2017).

The threshold choice has a cost. About four in ten alerts are false. This was accepted on purpose. A false alarm costs a pharmacist a minute to check. A missed stockout can interrupt treatment for patients on ARVs or TB medicines.

6.1.4 Objective 4: Score and Tier Suppliers

The fourth objective was a supplier scoring model. Each supplier was scored on on-time delivery, order fulfilment, price and defects. K-Means then grouped suppliers into Preferred, Acceptable and At-Risk tiers. Figure 6.3 shows a new supplier being scored and ranked against the existing twelve.

Result

Weighted score (0 = worst observed, 1 = best observed) and K-Means tier.

Score	Tier	Rank
0.727	Preferred	3 of 13

Score breakdown (weights 35 / 30 / 20 / 15)

Starting point	0.000
On-time delivery = 0.2	+0.241
Order fulfilment = 0.85	+0.251
Price vs market = -3	+0.102
Defect rate = 0.015	+0.132

#	Supplier	Score	Tier
1	Aspen Pharmacare	0.956	Preferred
2	Cipla Zimbabwe	0.893	Preferred
3	**Your candidate**	0.727	Preferred
4	Varichem Pharmaceuticals	0.653	Preferred
5	Adcock Ingram	0.652	Preferred
6	NatPharm (National Pharmaceutical Company)	0.575	Preferred
7	Datlabs (Pvt) Ltd	0.561	Preferred
8	CAPS Holdings	0.396	At-Risk
9	Plus Health Pharmaceuticals	0.350	At-Risk
10	Ajanta Pharma *	0.308	Acceptable
11	Interpharm Zimbabwe	0.274	At-Risk
12	Trustmed Pharmaceuticals	0.177	At-Risk
13	ZimMed Distributors	0.000	At-Risk

* fewer than 150 orders. Tiers come from clustering all four measures, so a tier can differ from score order.

Figure 6.3: Scoring a candidate supplier in the Model Lab

There is no correct ranking to test a supplier score against. The model was therefore tested for consistency instead. An independent clustering method produced identical tiers, with an adjusted Rand index of 1.000. Changing the weights by up to 30% hardly moved the ranking. The mean correlation was 0.994. Figure 6.4 shows that the live system reproduces all twelve exported scores exactly.

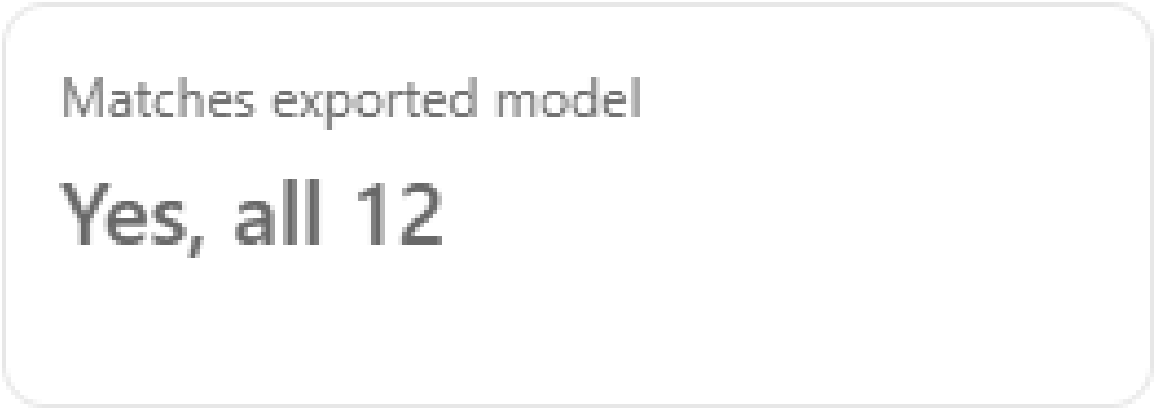


Figure 6.4: Supplier scores recomputed live from 6,000 orders

Ho et al. (2010) noted that most supplier models rely on expert weights that are rarely tested. This study tested its weights and showed that the ranking does not depend on them. The results also raise a practical concern for PGH. Four medicines, including oral rehydration salts, are offered only by At-Risk suppliers. The pharmacy may need to qualify new suppliers for these items.

6.1.5 Objective 5: Flag Batches at Risk of Expiry

The fifth objective was an expiry risk module. The module compares how long a batch will last with its time to expiry. Batches are used earliest-expiry-first, in line with good pharmacy practice. Figure 6.5 shows a slow-moving batch being checked.

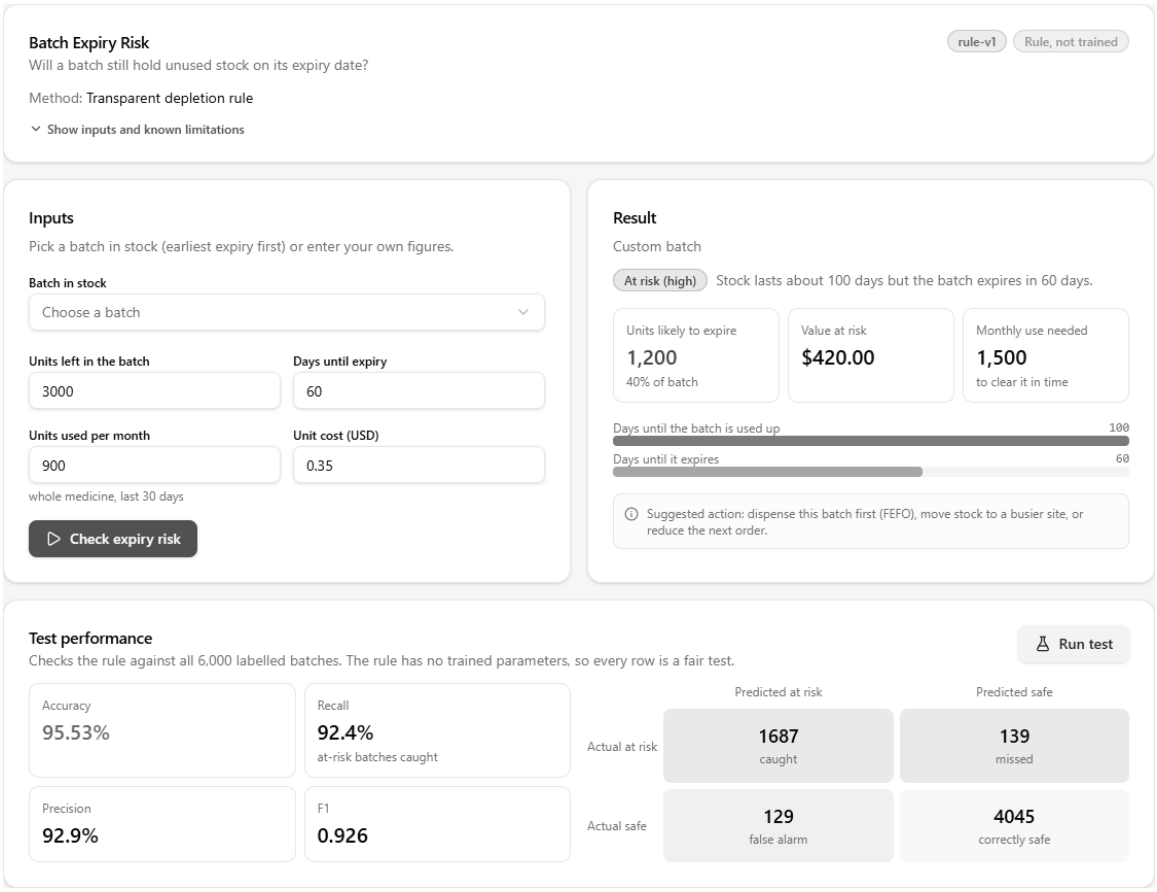


Figure 6.5: Expiry risk check and its test results

The batch in Figure 6.5 has 3,000 units and 60 days to expiry. At 900 units a month, it would take 100 days to use. About 1,200 units, worth USD 420, would therefore expire. Across 6,000 labelled batches, the rule agreed with 95.53% of the outcomes.

A Random Forest classifier was also trained for this task. On 560 unseen batches it scored an F1 of 0.946, against 0.951 for the rule. It also missed more at-risk batches. The rule was therefore kept. Machine learning should not replace a clear rule that works as well.

6.1.6 Objective 6: Integrate the Models into a Secure Web System

The sixth objective was to combine the models into one secure web system. The decision engine runs all four models for every medicine in about 1.5 seconds. It chooses a supplier, forecasts demand and sets the reorder point. It also removes stock that will expire. It then estimates the stockout risk and recommends an action. Figure 6.6 shows the full chain of reasoning for one recommendation.

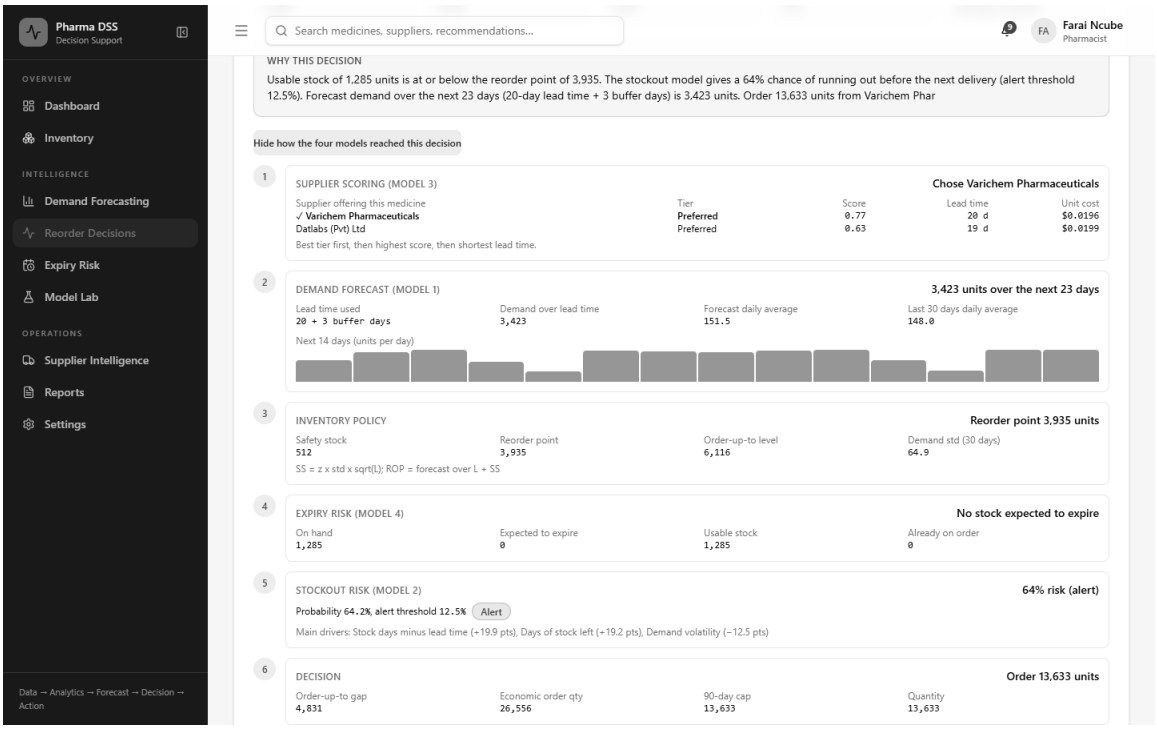


Figure 6.6: How the four models reached one reorder decision

This design follows Simon's (1960) model of decision making. The models carry out the intelligence and design phases. The pharmacist keeps the choice phase by approving or rejecting each order. Security is enforced on the server for every request. Figure 6.7 shows the reduced menu that a stores staff account receives.

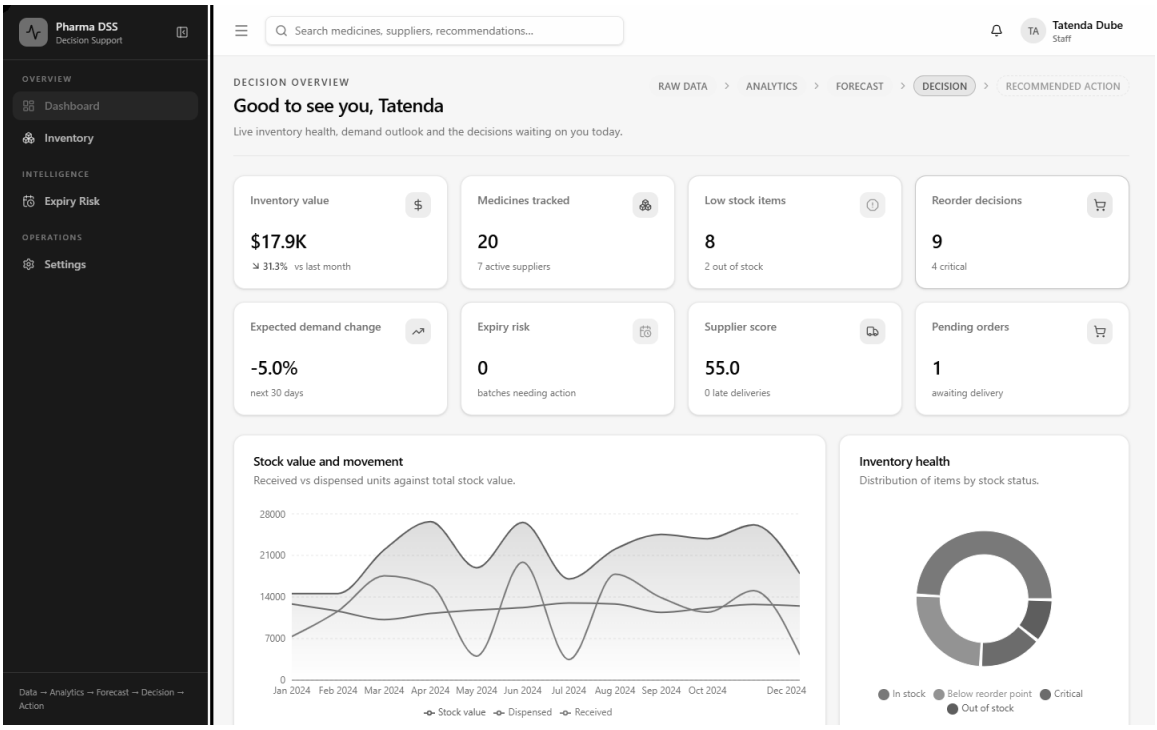


Figure 6.7: Role-based access for a stores staff account

The system also runs on the hospital network without internet access. Mugauri and Chimsimbe (2025) found that poor connectivity and power cuts limited Impilo in Zimbabwe. Designing for offline use addresses this local constraint directly.

6.1.7 Objective 7: Evaluate the System

The final objective was to evaluate the system with accuracy metrics and user acceptance testing. Table 6.3 summarises the technical evaluation reported in Chapter Five.

Test level	Tests	Passed
Automated backend tests	35	35
Machine learning tests	10	10
System functional tests	14	14
Performance tests	9	9

Table 6.3: Summary of the technical evaluation

Testing also found and fixed thirteen defects. The most serious was that login throttling had been configured but never switched on. Finding this before deployment shows the value of testing each claim against the running system.

User acceptance testing with PGH pharmacy staff is the remaining step. Its plan is set out in Section 5.2.7. Table 6.4 will record its results.

Measure	Result
Number of participants	[to be completed]
Tasks completed without help	[to be completed]
Participants rating the system useful or very useful	[to be completed]
Main suggestions from participants	[to be completed]

Table 6.4: User acceptance test results (to be completed after testing at PGH)

Until this testing is done, objective seven is only partly achieved.

6.1.8 Answers to the Research Questions

Research question	Answer from this study
RQ1: How does the pharmacy plan and order medicines?	Through periodic manual reviews of stock records. Five gaps were identified, pending staff confirmation.
RQ2: How accurately can ML forecast daily demand?	To within 21.3 units a day on average, 45.9% better than a naive forecast.
RQ3: Can a classifier detect stockout risk early enough to act?	Yes. It caught 80.2% of stockouts before the next delivery was due.
RQ4: How can suppliers be ranked fairly?	With a weighted score and clustering. The ranking was stable when weights changed.
RQ5: Can the models form one usable system?	Yes, technically. Acceptance by PGH staff is still to be tested.

Table 6.5: Answers to the research questions

6.2 Limitations and Constraints

Several limitations affect how far these results can be applied.

- **Simulated data.** Real PGH records were confidential, so the models were trained on simulated data. The results show what the system can do. They do not yet prove how it will perform at PGH.
- **Short history.** Only one year of demand data was available. The models could not learn how demand changes from year to year.
- **Hidden demand.** On stockout days, true demand was not recorded. The engine now ignores those days, but the true demand on them is still unknown.
- **Forecast horizon.** Accuracy was measured one day ahead. Longer forecasts feed predictions back in and are less certain.

- **Few suppliers.** Only twelve suppliers were scored, which limits the strength of the clustering results.
- **Assumed costs.** The ordering cost, holding cost and buffer days were assumptions. Real PGH figures may change the order sizes.
- **User testing.** Acceptance testing with PGH staff has not yet been carried out.
- **Time and resources.** The project had one academic year, a single developer and free cloud tools.
- **Local conditions.** Power cuts and slow internet affected development. The test laptop was also slow to start the server.

These limits do not undo the findings. They mean the results should be read as a tested proof of concept. A pilot on real PGH data must come before the system guides real orders.

6.3 Recommendations for Further Research

The following recommendations come directly from the experience of this project.

1. Pilot the system on real PGH data. Retrain each model on two or more years of records. Then compare the results with this study.
2. Record unmet demand. Staff could note how many patients were turned away during a stockout. This would reveal the demand that the current data hides.
3. Add disease data. Malaria case reports from the national health information system could give earlier warning.
4. Link to national systems. Links to eLMIS, Impilo and NatPharm would remove manual data loading. They would also support stock sharing.
5. Retrain regularly. Models should be retrained monthly, or when live error rises 20% above the test error.
6. Strengthen security. Multi-factor login for administrators and encrypted server disks should be added before full use.
7. Send alerts by SMS. Critical alerts could reach pharmacists by text message. SMS for Life did this in Tanzania (Barrington et al., 2010).

8. Train users continuously. Mbonane et al. (2023) found that training alone did not ensure good use of stock tools. Regular refresher sessions are needed.
9. Confirm the cost-benefit estimates. Real PGH expiry and purchase records should test the savings estimated in Chapter Two.

6.4 Conclusion

This study aimed to help the PGH pharmacy make evidence-based stock decisions. It developed and tested an Intelligent Decision Support System with four decision models. The demand model cut forecast error by almost half. The stockout model catches four in five stockouts before the next delivery. The supplier model ranks suppliers in a stable and explainable way. The expiry rule flags batches that will not be used in time.

The study's main contribution is how these models work together. One decision engine turns their outputs into a clear action for each medicine. Every recommendation shows its reasons, and a pharmacist must approve it. The system runs on ordinary computers and works without internet access. These features suit the conditions in Zimbabwean public hospitals.

The study also showed the value of honest evaluation. A rule was kept where machine learning did not improve on it. Hidden demand on stockout days was found and corrected. A security weakness was found in testing and fixed before deployment.

Much remains to be done before the system can guide real orders at PGH. It must be tested with pharmacy staff and retrained on real hospital data. Even so, the results show that machine learning can help Zimbabwe's public pharmacies. It can reduce stockouts, cut waste and support better use of scarce funds. In this way, it supports Vision 2030 and reliable access to essential medicines.

REFERENCES

African Union Commission. (2015). Agenda 2063: The Africa we want. African Union Commission.

AIDSFree Project. (2019). Zambia eLMIS endline evaluation report. JSI Research & Training Institute.

Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. In Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (pp. 2623–2631). ACM. https://doi.org/10.1145/3292500.3330701

Barrington, J., Wereko-Brobby, O., Ward, P., Mwafongo, W., & Kungulwe, S. (2010). SMS for Life: A pilot project to improve anti-malarial drug supply management in rural Tanzania using standard technology. Malaria Journal, 9, 298. https://doi.org/10.1186/1475-2875-9-298

Braun, V., & Clarke, V. (2006). Using thematic analysis in psychology. Qualitative Research in Psychology, 3(2), 77–101. https://doi.org/10.1191/1478088706qp063oa

Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32. https://doi.org/10.1023/A:1010933404324

Carbonneau, R., Laframboise, K., & Vahidov, R. (2008). Application of machine learning techniques for supply chain demand forecasting. European Journal of Operational Research, 184(3), 1140–1154. https://doi.org/10.1016/j.ejor.2006.12.004

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (pp. 785–794). ACM. https://doi.org/10.1145/2939672.2939785

Codd, E. F. (1970). A relational model of data for large shared data banks. Communications of the ACM, 13(6), 377–387. https://doi.org/10.1145/362384.362685

Creswell, J. W., & Creswell, J. D. (2018). Research design: Qualitative, quantitative, and mixed methods approaches (5th ed.). SAGE.

de Santis, R. B., de Aguiar, E. P., & Goliatt, L. (2017). Predicting material backorders in inventory management using machine learning. In 2017 IEEE Latin American Conference on Computational Intelligence (LA-CCI) (pp. 1–6). IEEE.

Dickson, G. W. (1966). An analysis of vendor selection systems and decisions. Journal of Purchasing, 2(1), 5–17.

Elmasri, R., & Navathe, S. B. (2016). Fundamentals of database systems (7th ed.). Pearson.

Etikan, I., Musa, S. A., & Alkassim, R. S. (2016). Comparison of convenience sampling and purposive sampling. American Journal of Theoretical and Applied Statistics, 5(1), 1–4. https://doi.org/10.11648/j.ajtas.20160501.11

Fourkiotis, K. P., & Tsadiras, A. (2024). Applying machine learning and statistical forecasting methods for enhancing pharmaceutical sales predictions. Forecasting, 6(1), 170–186. https://doi.org/10.3390/forecast6010010

Fowler, M. (2004). UML distilled: A brief guide to the standard object modeling language (3rd ed.). Addison-Wesley.

Gane, C., & Sarson, T. (1979). Structured systems analysis: Tools and techniques. Prentice Hall.

Government of Zimbabwe. (2018). Vision 2030: Towards a prosperous and empowered upper middle income society by 2030. Government of Zimbabwe.

Government of Zimbabwe. (2020). National Development Strategy 1: January 2021 – December 2025. Government of Zimbabwe.

Government of Zimbabwe. (2021). Cyber and Data Protection Act [Chapter 12:07] (Act No. 5 of 2021). Government Printer.

Harris, F. W. (1913). How many parts to make at once. Factory, The Magazine of Management, 10(2), 135–136.

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. MIS Quarterly, 28(1), 75–105. https://doi.org/10.2307/25148625

Ho, W., Xu, X., & Dey, P. K. (2010). Multi-criteria decision making approaches for supplier evaluation and selection: A literature review. European Journal of Operational Research, 202(1), 16–24. https://doi.org/10.1016/j.ejor.2009.05.009

Hubert, L., & Arabie, P. (1985). Comparing partitions. Journal of Classification, 2(1), 193–218. https://doi.org/10.1007/BF01908075

Hyndman, R. J., & Athanasopoulos, G. (2021). Forecasting: Principles and practice (3rd ed.). OTexts. https://otexts.com/fpp3/

International Organization for Standardization. (2022). ISO/IEC 27001:2022 Information security, cybersecurity and privacy protection: Information security management systems: Requirements. ISO.

Ivanov, D. (2020). Predicting the impacts of epidemic outbreaks on global supply chains: A simulation-based analysis on the coronavirus outbreak (COVID-19/SARS-CoV-2) case. Transportation Research Part E: Logistics and Transportation Review, 136, 101922. https://doi.org/10.1016/j.tre.2020.101922

Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT) (RFC 7519). Internet Engineering Task Force. https://doi.org/10.17487/RFC7519

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. In Advances in Neural Information Processing Systems 30 (pp. 3146–3154).

Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. In Advances in Neural Information Processing Systems 30 (pp. 4765–4774).

Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2022). M5 accuracy competition: Results, findings, and conclusions. International Journal of Forecasting, 38(4), 1346–1364. https://doi.org/10.1016/j.ijforecast.2021.11.013

Mbonane, H., Sibanda, M., Godman, B., Meyer, J. C., & Matlala, M. (2023). Knowledge, attitudes and practices of healthcare professionals on the use of an electronic stock visibility and management tool in a middle-income country: Implications for the future. Exploratory Research in Clinical and Social Pharmacy, 9, 100233.

Mbonyinshuti, F., Nkurunziza, J., Niyobuhungiro, J., & Kayitare, E. (2022). The prediction of essential medicines demand: A machine learning approach using consumption data in Rwanda. Processes, 10(1), 26. https://doi.org/10.3390/pr10010026

Mehrabi, N., Morstatter, F., Saxena, N., Lerman, K., & Galstyan, A. (2021). A survey on bias and fairness in machine learning. ACM Computing Surveys, 54(6), 1–35. https://doi.org/10.1145/3457607

Ministry of Health and Child Care. (2021). National health strategy for Zimbabwe 2021–2025. Government of Zimbabwe.

Mugauri, H. D., & Chimsimbe, M. (2025). Electronic health record systems in limited resource settings: A comprehensive evaluation of the Impilo platform. Acta Informatica Pragensia, 14(3), 393–407. https://doi.org/10.18267/j.aip.265

Nielsen, J. (1993). Usability engineering. Academic Press.

Office of the Auditor-General. (2025). Report of the Auditor-General on the management of medicines and medical supplies in public hospitals. Office of the Auditor-General of Zimbabwe.

Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. Journal of Management Information Systems, 24(3), 45–77. https://doi.org/10.2753/MIS0742-1222240302

Power, D. J. (2002). Decision support systems: Concepts and resources for managers. Quorum Books.

Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. Journal of Computational and Applied Mathematics, 20, 53–65. https://doi.org/10.1016/0377-0427(87)90125-7

Saabas, A. (2014, October 19). Interpreting random forests. Diving into Data. https://blog.datadive.net/interpreting-random-forests/

Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. PLoS ONE, 10(3), e0118432. https://doi.org/10.1371/journal.pone.0118432

Schwaber, K., & Sutherland, J. (2020). The Scrum guide: The definitive guide to Scrum. Scrum.org.

Settanni, E., Harrington, T. S., & Srai, J. S. (2017). Pharmaceutical supply chain models: A synthesis from a systems view of operations research. Operations Research Perspectives, 4, 74–95. https://doi.org/10.1016/j.orp.2017.05.002

Silver, E. A., Pyke, D. F., & Thomas, D. J. (2017). Inventory and production management in supply chains (4th ed.). CRC Press.

Simon, H. A. (1960). The new science of management decision. Harper & Row.

Sommerville, I. (2016). Software engineering (10th ed.). Pearson.

Strubell, E., Ganesh, A., & McCallum, A. (2019). Energy and policy considerations for deep learning in NLP. In Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (pp. 3645–3650). ACL. https://doi.org/10.18653/v1/P19-1355

United Nations. (2015). Transforming our world: The 2030 Agenda for Sustainable Development (A/RES/70/1). United Nations.

USAID | DELIVER PROJECT. (2015). Evaluation of the Zimbabwe Assisted Pull System. USAID | DELIVER PROJECT, Task Order 4.

Uthayakumar, R., & Priyan, S. (2013). Pharmaceutical supply chain and inventory management strategies: Optimization for a pharmaceutical company and a hospital. Operations Research for Health Care, 2(3), 52–64. https://doi.org/10.1016/j.orhc.2013.08.001

Wirth, R., & Hipp, J. (2000). CRISP-DM: Towards a standard process model for data mining. In Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining (pp. 29–39).